

プログラミング言語 Egison 入門

Egison 開発チーム

2026 年 2 月 8 日

はじめに

Egison は、「人間の頭の中の認識を、コンピュータ向けに翻訳することなしに、表現できる言語を作りたい」という動機で作られたプログラミング言語である。より幅広いアルゴリズムの簡潔な記述を可能にする新しいプログラミング言語は、既知のコンピュータ科学の問題のプログラミングを簡単にするだけでなく、従来プログラミングが難しいために敬遠されていた問題を扱いやすくし、コンピュータ科学の扱う範囲を広げる可能性を持つため重要である。

Egison に実装されているこれらのアイデアのうち、最初に実装された機能は本書の第一部の主題であるパターンマッチである。Egison のパターンマッチの特徴は、ユーザーにより拡張可能でかつ、非線形パターン（パターン変数に束縛した値を同じパターン内で参照するパターン）をバックトラッキングにより効率的に処理することである。本書の第一部は、Egison を通して提唱されているパターンマッチ指向プログラミングというプログラミング・パラダイムをできるだけコンパクトに紹介することを目的として執筆された。パターンマッチまわりの Egison の構文と、それらの構文を使ったプログラミングテクニックを紹介する。

このパターンマッチを応用して、Egison には数式処理システムが実装されている。この数式処理システムには、任意のユーザー定義関数に対してテンソルの添字記法が適用できるという特徴がある。この機能によって、数学のなかでも特に計算が大変な微分幾何という分野に現れる計算を紙の上の数式に近い形でプログラムとして記述できる。そのほかにも実験的な新しい機能が Egison 上の数式処理システムには実装されており、第二部はこれらについて紹介する。

本書がきっかけとなって、プログラミング言語、とくにパターンマッチや数式処理システムの研究をする研究仲間が読者の中から現れてくれたらとてもうれしい。

2021 年 5 月 13 日

江木 聡志

目次

はじめに	ii
第 1 章 プログラミング Egison とは	1
1.1 プログラミング言語全体の中での Egison の位置づけ	1
1.2 第 I 部 パターンマッチ指向プログラミング	3
1.3 第 II 部 数式処理システムとしての Egison	5
第 I 部 パターンマッチ指向プログラミング	7
第 2 章 Egison 早巡り	8
2.1 matchAll 式によるパターンマッチ	8
2.2 値パターンと述語パターンによる非線形パターンの表現	10
2.3 バックトラッキングによる効率的な非線形パターンマッチ	11
2.4 マッチャーによるパターンの拡張性と多相性	11
2.5 matchAllDFS 式によるパターンマッチ結果の順序の制御	12
2.6 and パターン・or パターン・not パターン	13
2.7 ループ・パターン	14
2.8 シーケンシャル・パターン	15
2.9 パターン関数によるパターンのモジュール化	17
2.10 マッチャー合成による新しいマッチャーの生成	17
第 3 章 パターンマッチ指向プログラミング入門	19
3.1 パターンマッチによるリスト・プログラミング	19
3.2 多重集合プログラミング	21
3.3 タプル・パターンによるデータの比較	24
3.4 ループ・パターンによる再帰的パターン	25
第 4 章 パターンマッチ指向プログラミングの効果	32
4.1 SAT ソルバーの実装	32

4.2	2 種類のループの分離	34
第 5 章	Egison パターンマッチの仕組み	35
5.1	パターンマッチ・アルゴリズムの概略	35
5.2	matchAll・matchAllDFS による探索木のトラバース	37
5.3	and パターン・or パターン・not パターンの実装	39
5.4	パターン関数の実装	40
第 6 章	マッチャーを定義しよう	42
6.1	inductive pattern 宣言	42
6.2	マッチャーの定義の基礎	43
6.3	eq マッチャーの定義	46
6.4	multiset マッチャーの定義	47
6.5	soretedList マッチャーの定義	48
6.6	原始ワイルドカードによる最適化	49
6.7	assocMultiset マッチャー	50
第 7 章	関数型プログラミング言語としてのいくつかの機能	51
7.1	無名パラメーター関数	51
7.2	無名 matchAll 関数・無名 match 関数	52
7.3	中置演算子の定義	52
7.4	型クラス	54
7.5	I/O 入出力	56
7.6	組み込み関数	58
7.7	seq 式	59
第 8 章	Sweet Egison - Egison パターンマッチの Haskell ライブラリ	65
8.1	Sweet Egison の使い方	65
8.2	Sweet Egison の仕組み	67
8.3	Sweet Egison の最適化	68
8.4	マッチャーとパターンの型	70
8.5	Sweet Egison のマッチャー定義	70
8.6	ユーザによる探索戦略の追加	71
第 II 部	数式処理システムとしての Egison	73
第 9 章	数式処理システム入門	74
9.1	シンボルの宣言	74

9.2	関数適用のシンボル化	75
9.3	数式の簡約	75
9.4	二次方程式を解くプログラム - 数式を処理するアルゴリズムの記述	76
9.5	微分するプログラム - 数式に対するパターンマッチ	77
9.6	ベクトルや行列の計算 - 添字記法	79
9.7	数式の出力形式	79
第 10 章	数式データの扱い	81
10.1	数式データの内部表現	81
10.2	バッククオート (‘) による式展開の制御	82
10.3	シングルクオート (’) による関数適用の制御	83
10.4	withSymbols 式によるローカルシンボルの宣言	84
10.5	数式データに対するパターンマッチ	84
第 11 章	テンソル計算	87
11.1	テンソルとは	87
11.2	テンソルの添字記法	88
11.3	添字記法をプログラミングに導入するためのアイデア	89
11.4	スカラー仮引数とテンソル仮引数	90
11.5	三種類の添字	92
11.6	添字の簡約規則	93
11.7	テンソル関数の適用	94
11.8	省略された添字の補完	94
11.9	tensorMap 自動挿入の仕組み	95
11.10	添字の反転と flipIndices 関数	96
11.11	generateTensor 式によるテンソルの生成	97
11.12	テンソルの宣言	98
11.13	添字付加のために便利な構文	99
11.14	テンソルの対称性・歪対称性の宣言	100
11.15	tensorMap 挿入の詳細な仕組み	103
第 12 章	関数シンボル	108
12.1	関数シンボルの基本的な使い方	108
12.2	引数への値の代入	109
12.3	合成関数の微分 (連鎖律)	109
12.4	関数シンボルを成分にもつテンソル	110
12.5	関数シンボルの内部表現とパターンマッチ	110

12.6	関数シンボルの実装方法	111
第 13 章	Egison で計算する微分幾何	112
13.1	微分幾何と Egison	112
13.2	リーマン曲率テンソルの計算	113
13.3	曲率形式の計算 - 微分形式を使った計算 (1)	115
13.4	ホッジ・ラプラシアン of 計算 - 微分形式を使った計算 (2)	118
13.5	その他の微分形式についての演算子 - 内部積・リー微分	119
第 III 部	Egison の今後	120
第 14 章	Egison 開発の目標と方針	121
14.1	処理系の開発	121
14.2	アプリケーションの提示・開発	125
14.3	Egison の先にある研究	127
付録 A	パターンマッチ指向プログラミング問題集	130
付録 B	数式処理システムとしての Egison の演習問題	134
参考文献		136
索引		137

第 1 章

プログラミング Egison とは

本章では、Egison がどのような言語であるのかを紹介する。Egison とそれが提唱しているパターンマッチ指向プログラミングについて、その大体のイメージをつかんでもらいたい。

1.1 プログラミング言語全体の中での Egison の位置づけ

プログラミング言語の機能には大きく分けて 2 種類ある。コンピュータを含むさまざまな機械の操作を簡潔に記述するための機能と、人間の頭のなかにある抽象的な概念をプログラムとして表現するための機能である。コンピュータは物理的なデバイスであるため、前者の機能は必須である。この機能には、たとえばキャッシュやメモリ管理ための操作を自動化する機能などが含まれる。後者の機能は、プログラマの頭のなかにあるアルゴリズムの認識を、コンピュータ向けに翻訳することなく記述できるようにすることを目指す。コンピュータに解かせたい問題のなかには、その解法に抽象的な概念が関係する問題が多くある。人間には簡単に理解できる抽象化でも、コンピュータが理解できる形で表現することは難しいことが多い。Egison は後者の機能の拡充に注力してつくられている。

アルゴリズムを簡潔に記述することを目指す主流の流派は、関数型プログラミング言語である。関数型プログラミング言語によるプログラムの記述の大きな特徴は、関数を他の組み込みデータ型と同じようにプログラマが扱えることである。具体的にいうと、関数を別の関数の引数として渡したり、関数を返す関数を定義することができる。そのおかげで、関数型プログラミング言語以外の言語ではモジュール化がむずかしい処理をモジュール化できることが多々ある。関数型プログラミングでは、物理的なデバイスであるコンピュータの操作は、多くの場合、コンパイラによってプログラマから隠される。

しかし、関数型プログラミング言語でも、頭のなかのアルゴリズムのイメージを、プログラムとして記述できるように翻訳する必要がある場合がある。非自由データ型を扱うアルゴリズムは、その典型的な例である。非自由データ型とは、同じデータにたいして複数の同値な表現形があるデータ型のことをいう。たとえば、多重集合（要素の重複を許す集合）は非自由データ型である。 $\{a, a, b\}$ という多重集合は、 $\{a, b, a\}$ や $\{b, a, a\}$ と表現できるからである。ほかには、グラフや

数式なども非自由データ型である。

非自由データ型にたいするパターンマッチを可能にすることによって、簡潔に記述できるアルゴリズムの範囲を広げることを目指して Egison は開発された。Egison には非自由データ型のデータをパターンマッチするための機能が実装されている。そして、このパターンマッチを活かしたプログラミング・パラダイムであるパターンマッチ指向プログラミングを提唱している。

また、このパターンマッチ機能を活かして Egison の上に数式処理システムが実装されている。数式処理システムとは、 $x + x = 2x$ や $(x + y)^2 = x^2 + 2xy + y^2$ のようにシンボリックな計算ができるプログラミング言語のことをいう。Egison を使えば数式に対するパターンマッチが簡単に定義できるため、数式処理システムが他の言語にくらべて簡単に実装できる。この数式処理システムの簡潔な実装による拡張性を活かして、Egison にはいくつか数式処理システムとして新しい機能が実装されている。特にテンソルについての計算を記述するための機能が発展しており、従来ユーザーが定義することが困難であった微分幾何学の演算子を簡単に定義できるようになっている。

■新しい表記法の追求と科学研究 既存の言語では、我々の頭の中のイメージをそのまま表現できないことがある。このような不都合は、我々が頭の中でおこなっている抽象化を、言語では表現できないときに生じる。このような抽象化を表現可能にする新しい構文を発見して、言語に組み込むことができれば、この問題は解決される。実際、プログラミング言語について、このような抽象化をひとつ見つけて、作りはじめたのが Egison である。

そもそも、すべてのイメージを表現できるような言語を作ることが本当にできるのかという問題は非常に難しい。というのは、我々の頭の中のイメージは、対象に対する我々の理解が深まると同時に変化するものであるからである。その変化と同時に、我々にとって便利な表現も自然と変わっていく。

このことを実感するために、ある極端な例を考えてみる。

数の概念はすでに知っているけれども、たし算やかけ算の概念は知らないひとがいるとする。そのひとにとっては、10 進法は理解不能な概念であり、10 進表記もまったく理解不能な表記である。しかし、たし算やかけ算の概念とその性質をある程度知っているひとにとっては、10 進表記は非常に便利で、直感的な数の表現である。

たし算・かけ算を知らないというのは極端な例に思ふかもしれない。しかし、我々がまだ知らない重要な概念は科学史をふりかえってみるとまだまだあってもおかしくない。このような概念が発見され、自然に対する我々の認識が深まれば、それを表現するために新しい表記法が言語に生まれる。逆に、このようにして生まれた新しい表記法は我々が自然をより深く理解することを助け、その結果、さらなる新しい表記法が生まれる。このように、表記法の進化と科学の発展は表裏一体の関係にある。

ゆえに、表記法について研究することには大きな意義があるだろう。「新しい表記法を見つ

けるための一般的な方法はあるのだろうか?」, 「よりよい表記法が満たす一般的法則はあるのだろうか?」, 「表記法の表現力を測る一般的な方法はあるのだろうか?」など興味深い問題はいくつも存在する。これらの問題の答えが見えたとき, まったく新しい世界観が生まれると筆者は期待している。新しいプログラミング言語を作ることとおして, これらの問題の答えに近づきたいというのが, Egison を作った動機である。

■独自のプログラミング言語を用意する理由 Egison に実装されている新しいプログラミング言語の機能を, 独自のプログラミング言語 Egison にではなく, 既存のプログラミング言語を拡張して実装するという手段もある。このような手段を選ばず, 独自のプログラミング言語 Egison を作る理由は, そのほうが新しい機能をすばやく確実に実装できるためである。既存のプログラミング言語に新しく発明した機能を追加するには, その新機能が既存の機能と競合し, 言語の実装の広範囲に影響を与える場合があるために, 新機能本体の実装以外のための大規模な調査や実装が必要になってしまう場合がある。実際, Egison のパターンマッチとテンソルの添字記法は両者ともそのような機能である。たとえば, Haskell に Egison のパターンマッチを実装しようとしたら, パターンマッチ機能の実装に加えて型システムの設計と実装もせねばならない。また, Haskell にテンソルの添字記法を実装する場合も, シンボリックな添字を扱うためのシンボル計算の機能や, シンボリックな添字をもつテンソルを扱うためのデータ構造の設計や型システムの設計・実装も必要となる。たいして, 自身で発明した新機能を実装するために設計した独自のプログラミング言語を用意すれば, 新機能の実装を最低限の労力で実装できる。ただし, 独自のプログラミング言語を用意することには, 既知の機能を再実装する必要があったり, 新規ユーザーを得るのが難しいというデメリットもある。

なお, Egison は 2026 年 1 月の開発により静的型付けシステムを備えるようになった (Egison version 5)。型推論により多くの場合型アノテーションを省略できるが, 必要に応じて明示的に型を指定することもできる。この型システムは, 非自由データ型のパターンマッチやテンソル計算など, Egison の高度な機能と統合されている。静的型システムの導入により, プログラムの信頼性が向上し, 特に数式処理システムとしての実用性が大幅に高まった。

2019 年からは既存のメジャーなプログラミング言語に Egison の機能を実装することにも取り組み始めている。他言語への Egison の機能の移植については, [14.1.2 節](#)で言及する。

1.2 第 I 部 パターンマッチ指向プログラミング

Egison が提唱する新しいプログラミング・パラダイムであるパターンマッチ指向プログラミングについて, 第 I 部は解説する。

1.2.1 パターンマッチ指向プログラミングとは

本節では、いくつかの例をみることによって、パターンマッチ指向プログラミングとはどのようなプログラミング・パラダイムであるのかそのイメージを紹介する。

パターンマッチ指向プログラミングによってプログラムの記述が直感的になっていることのわかりやすい例に `intersect` 関数の実装がある。 `intersect` は 2 つのリストを引数にとり、それらのリストに共通する要素のリストを返す関数である。 `Egison` を使って引数のリストをそれぞれ要素の順序を無視する集合としてパターンマッチすると、2 つのリストの共通要素にマッチするパターンを記述することにより、 `intersect` を記述できる。プログラムの読み方は次章以降で紹介する。

```
def intersect xs ys :=
  matchAllDFS (xs, ys) as (set eq, set eq) with
  | ($x :: _, #x :: _) -> x
```

対して、 `Egison` のパターンマッチを使わずに関数型プログラミングのスタイルで `Haskell` で記述すると、リストに対する関数を組み合わせて共通要素を抜き出すための方法を記述する必要がある。

```
intersect xs ys = filter (\x -> any (== x) ys) xs
```

このプログラムは、リスト内包表記を使って以下のように書き直すこともできる。

```
intersect xs ys = [x | x <- xs, any (== x) ys]
```

`Egison` のパターンマッチを使ったプログラムは、2 つのリストの共通要素にマッチするパターンを記述しているだけであるのに対し、関数型プログラミングでは、どうやってその共通要素を取り出すかその方法をプログラマが考えて記述する。前者のように「何を計算したいか (what to do)」を記述するプログラミングスタイルは宣言型プログラミング、後者のように「どのように計算するか (how to do)」を記述するプログラミングスタイルは手続き型プログラミングと呼ばれている。「何を計算したいか」から「どのように計算するか」が明らかである場合は、「何を計算したいか」を記述する宣言型プログラミングのほうがプログラムが読みやすく簡潔になる。`intersect` の例の場合は、 `Egison` が集合のパターンマッチアルゴリズムをモジュール化できるおかげで、宣言的なプログラミングが可能になっている。

少し毛色の違う例として、 `concat` 関数をパターンマッチ指向プログラミングで定義する。リストのリストの要素にマッチするパターンを記述することにより、 `concat` を定義できる。

```
def concat xss :=
  matchAllDFS xss as list (list something) with
  | _ ++ (_ ++ $x :: _) :: _ -> x

concat [[1,2],[3],[4,5]]
-- [1,2,3,4,5]
```

さらにもう 1 つのパターンマッチ指向プログラミングの例として、`unique`関数を定義する。後方に自身と同じ値が含まれない要素にマッチするパターンを記述することにより `unique`を定義できる。

```
def unique xs :=
  matchAllDFS xs as list eq with
  | _ ++ $x :: !(_ ++ #x :: _) -> x

unique [1,2,3,2,4]
-- [1,3,2,4]
```

次章以降で、上記のプログラムで使われている Egison の構文や、機能、プログラミングテクニックの解説をしていく。

1.2.2 第 I 部の構成

第 I 部は可能な限りコンパクトに、パターンマッチ指向プログラミングとそのための Egison の機能の解説をまとめることを目指した。そのため、本書は前提知識として、関数型プログラミング (Lisp 系言語や、OCaml, Haskell を使ったプログラミング) の知識を仮定している。既存の関数型言語にも存在する簡単な機能については、登場したときに軽く解説するにとどめた。

第 I 部の構成は以下の通りである。第 2 章では、パターンマッチに関係する Egison の構文を一通り紹介する。第 3 章では、パターンマッチ指向プログラミングで頻出するテクニックを紹介する。第 4 章では、Egison のパターンマッチを活かしたプログラミングスタイルであるパターンマッチ指向プログラミングとはなにか解説する。第 5 章では、Egison 内部のパターンマッチの仕組みも解説する。第 6 章では、ユーザーによるマッチャー定義の方法を説明する。第 7 章では、実際にプログラミングするときに便利な、無名パラメータ関数や IO 入出力するための機能について解説する。第 8 章では、Egison のパターンマッチを Haskell で使うためのライブラリ Sweet Egison の使い方を紹介する。

ユーザーとして Egison に興味がある読者は、第 2 章、第 3 章、第 4 章、第 5 章 5.1 節、第 6 章を順に読んでいくのがよい。Egison の設計・実装に興味がある読者は、第 2 章の前半部分を読んだ後、第 5 章、第 6 章を読むことができる。

1.3 第 II 部 数式処理システムとしての Egison

数式処理システムとして Egison がもつ機能を第 II 部は紹介する。

1.3.1 数式処理システムとしての Egison とその特徴

数式処理システムとは、 $x + x = 2x$ のようにシンボリックな計算ができるプログラミング言語のことである。このようなシンボリックな計算は、我々が日頃おこなっている計算の大きな部分

を占める．実際，中学校以降の数学で現れる計算のほとんどはシンボリックな計算を含む．そのため，シンボリックな計算をできるプログラミング言語を作ることは重要である．

数式処理システムを作ることがむずかしい原因は，シンボリックな変数を含む数式が一つの定まった形を持たない非自由データ型であることである．たとえば， $(x + y)^2$ という数式は， $x^2 + 2xy + y^2$ という形の数式とも同値である．Egison の非自由データ型に対しても適用可能なパターンマッチ機能を使うと数式の書き換え処理を簡潔に実装できるために数式処理システムを，既存のプログラミング言語よりもかなり少ないコード量で実装することができる．このような考えをもとに 2016 年に Egison で数式処理システムは実装された．この数式処理システムは Egison 処理系と統合されており，Egison 上で使うことができる．

この数式処理システムの実装は，ほかの数式処理システムの実装にくらべてシンプルである．そのために機能拡張がしやすいというメリットがある．実際，このおかげで，Egison 独自の重要な特徴であるテンソルの添字記法をプログラミングへの導入についても，簡単に実装することができた．テンソルの添字記法をプログラミング言語に導入する手法は，この数式処理システムを使って微分幾何の計算をするプログラムのサンプルを作っている最中に発案され，数週間で実装された．

プログラミング言語に数式処理システムの機能を実装することには，実用的なものが作りやすいという利点がある．数式処理システムは，手で計算するより速ければ喜ばれるケースが多く，通常のプログラミング言語とくらべて，実行速度がシビアに求められない．実行速度よりも，普段手で書いている数学記法をプログラムでも記述できることのほうが数学の研究者にとってはうれしいことが多い．そのため，数式処理システムの実装は，新しい記法のプログラミングへの導入を目指す Egison にとって格好の課題といえる．

1.3.2 第 II 部の構成

第 II 部の最初に数式処理システムの使い方について解説したあと，Egison のもつ数式処理システムとしての機能を解説する．

第 II 部の構成は以下の通りである．第 9 章では，数式処理システムの特徴であるシンボリックな計算の方法と，具体的なプログラムの例を紹介する．第 10 章では，Egison 内部で数式データがどう表現されているのか解説する．第 11 章では，テンソルの添字記法を使ったプログラムの記述の方法を解説する．第 12 章では，関数シンボルについて解説する．第 13 章では，ここまで解説した機能の実践的な応用として，微分幾何の計算をするプログラムを紹介する．

第Ⅰ部

パターンマッチ指向プログラミング

第 2 章

Egison 早巡り

本章は、パターンマッチ指向プログラミングのための Egison の機能を一通り紹介する。本章を理解すれば、Egison のパターンマッチの仕様はほぼ一通り理解したことになるはずである。

本章の前半（2.1 節，2.2 節，2.3 節，2.4 節，2.5 節）では、パターンマッチのための構文を紹介する。Egison のパターンマッチは、非自由データ型に対するパターンマッチを実現するために、複数のマッチ結果をもつ非線形パターン（パターン変数に束縛した値を同じパターン内で参照するパターン）を効率的に処理できるように設計されている。本章の前半では、この機能がどのように構文として表現できるのか解説する。

本章の後半（2.6 節，2.7 節，2.8 節，2.9 節，2.10 節）では、さまざまな組み込みパターンを紹介する。そのうち後半で紹介されるループ・パターンや、シーケンシャル・パターンは Egison 以外のプログラミング言語にはまだ実装されていないパターンである。これらの組み込みパターンは、最初は特殊でアドホックにみえるかもしれない。これらの組み込みパターンの意味は、初見で完全に理解できなくても問題ない。本書の後半で、これらのパターンの実用例が紹介されたときに、本章にもどってきてほしい。

2.1 matchAll 式によるパターンマッチ

パターンマッチを記述するためのいくつかの構文を Egison は提供している。そのうちもっとも基本的な構文は matchAll 式である。

```
matchAll [1,2,3] as list something with
| $x :: $ts -> (x, ts)
-- [(1,[2,3])]
```

matchAll 式は、**ターゲット**（上記の場合、`[1,2,3]`）、**マッチャー**（上記の場合、`list something`）、1 つ以上の**マッチ節**（上記の場合、`$x :: $xs -> (x, xs)`）から構成される。マッチ節は、**パターン**（上記の場合、`$x :: $xs`）と**ボディ**（上記の場合、`(x, xs)`）からなる。matchAll 式は、既存のプログラミング言語のマッチ式と同様に、ターゲットとパターンのパターンマッチを試みて、もしマッ

チしたらそのマッチ節のボディを評価する。

Egison の `matchAll` 式の特徴は、(1) 結果としてリストを返すことと、(2) マッチャーという追加の引数をとることにある。(1) の特徴は、複数の結果をもつパターンマッチをサポートするための特徴である。`matchAll` 式は複数のパターンマッチの結果すべてについてボディを評価して、その結果を集めてリストとして返す。上記の例のパターンに使われている `::` は **コンス・パターン** (*cons pattern*) と呼ばれる、リストを先頭の要素と残りのリストに分解するパターンコンストラクタである。そのため、上記の例の場合はパターンマッチの結果が一つであるので、長さが 1 のリストを返している。(2) の特徴は、パターンマッチアルゴリズムの拡張性とパターンの多相性を実現するための特徴である。マッチャーは Egison 以外の言語ではみられない独自のオブジェクトで、パターンマッチアルゴリズムを保持するためのオブジェクトである。マッチャーによるパターンの多相性については、2.4 節で扱う。--で始まる行はコメントである。本書において、プログラム直後のコメントはプログラムの実行結果を記述するものとする。

`matchAll` 式の文法の詳細についてももう少し説明する。`as` と `with` というキーワードでマッチャーは囲まれる。`matchAll` 式はマッチ節を複数とることができる。^{*1} マッチ節の先頭には `|` がつく。^{*2} `|` はマッチ節が複数行に渡る場合にプログラムの可読性を高める。マッチ節中のパターンとボディは `->` で区切られる。

```
matchAll ターゲット as マッチャー with
| パターン 1 -> ボディ 1
| パターン 2 -> ボディ 2
...
```

マッチ節が 1 つであるときは、下記のようにマッチ節の前の `|` を省略することができる。

```
matchAll ターゲット as マッチャー with パターン -> ボディ
```

複数の結果をもつパターンマッチの例を紹介する。下記の `matchAll` 式のパターンで使われている `++` は **ジョイン・パターン** (*join pattern*) と呼ばれる、リストを先頭部分のリストと残りのリストに分解するパターンコンストラクタである。ジョイン・パターンによる複数の分解のすべてについて、`matchAll` 式はボディ式を評価する。

```
matchAll [1,2,3] as list something with
| $hs ++ $ts -> (hs, ts)
-- [([], [1, 2, 3]), ([1], [2, 3]), ([1, 2], [3]), ([1, 2, 3], [])]
```

^{*1} `matchAll t as m with c1 c2 ...` は、`(matchAll t as m with c1) ++ (matchAll t as m with c2) ++ ...` と同値である。

^{*2} マッチ節が複数ある場合、最初のマッチ節の前にも `|` は必須である。

2.2 値パターンと述語パターンによる非線形パターンの表現

matchAll式は、非線形パターンと組み合わせたときにその本領を発揮する。たとえば、以下の非線形パターンは、対象のコレクションが同値な要素のペアを含む場合にパターンマッチに成功する。

```
matchAll [1,2,3,2,4,3] as list integer with
| _ ++ $x :: _ ++ #x :: _ -> x
-- [2,3]
```

値パターン (*value pattern*) は非線形パターンを表現するために重要な役割を果たす。値パターンは、値パターンの中身とターゲットが等しいかどうかチェックする。値パターンの先頭には、#が付加される。#の後ろには任意の式を書くことができる。#に続く式は、その値パターンの左側に現れたパターン変数に束縛された値を参照しながら評価される。この動作を実現するため、Egisonのパターンは左から右に順番に処理されることが決まっている。その結果、 $x :: \#x :: _$ のようなパターンは妥当であるが、 $\#x :: x :: _$ は正しく動かない。(どうしてもパターンの右側に現れるパターン変数の値を参照したいという場合は、2.8節で紹介するシーケンシャル・パターンが使える。)

ほかの非線形パターンの例として、双子素数のパターンマッチを紹介する。双子素数とは、差が2であるような素数のペアのことをいう。primesには素数の無限列が束縛されている。^{*3} このmatchAll式は、素数の無限列からすべての双子素数を順番に抜き出す。Egisonは静的型付けシステムを備えているが、型推論により型アノテーションを省略できる。必要に応じてdef twinPrimes : [(Integer, Integer)] := ...のように型を明示的に指定することもできる。

```
def twinPrimes := matchAll primes as list integer with
| _ ++ $p :: #(p + 2) :: _ -> (p, p + 2)

take 8 twinPrimes
-- [(3, 5), (5, 7), (11, 13), (17, 19), (29, 31), (41, 43), (59, 61), (71, 73)]
```

パターンマッチのときに、同値性よりも一般的な条件を使いたいことがある。**述語パターン** (*predicate pattern*) はそのために用意されている組み込みパターンである。述語パターンは、中身の述語にターゲットを適用した結果がTrueである場合、パターンマッチに成功するパターンである。述語パターンの先頭は、?からはじまり、?のあとには1引数の述語が続く。

```
def twinPrimes := matchAll primes as list integer with
| _ ++ $p :: ?(\q -> q = p + 2) :: _ -> (p, p + 2)
```

^{*3} primesはEgisonの組み込みライブラリで定義されている。

2.3 バックトラッキングによる効率的な非線形パターンマッチ

Egison 内部のパターンマッチアルゴリズムは、非線形パターンを効率的に処理するためにバックトラッキングを使う。

```
matchAll [1..n] as list integer with _ ++ $x :: _ ++ #x :: _ -> x
-- 0(n^2) で [] を返す
matchAll [1..n] as list integer with _ ++ $x :: _ ++ #x :: _ ++ #x :: _ -> x
-- 0(n^2) で [] を返す
```

上記の2つのマッチ式は、1から n までの整数のリストから同じ要素の2つ組、3つ組をそれぞれ抽出する。同じ要素の2つ組も3つ組もターゲットのリストには含まれていないため、これらのmatchAll式は両方とも空リストを返す。2つ目のmatchAll式が評価される時、Egison 処理系は2つ目の#xのパターンマッチまで行き着かない。1つ目の#xでパターンマッチが失敗するからである。(2.2節で述べたようにEgisonのパターンマッチはパターンを左から右へ順番に処理する。)それゆえ、両方のmatchAll式の時間計算量は同じである。Egison 内部のパターンマッチアルゴリズムについては、第5章で詳しく解説する。

2.4 マッチャーによるパターンの拡張性と多相性

パターンの拡張性以外のマッチャーがもたらすメリットに、**パターンのアドホック多相性**がある。パターンのアドホック多相性とは、`::`や`++`などといったパターン・コンストラクタを、リストや多重集合などといった複数のマッチャーに対して、同じ名前で使うことができる性質のことである。パターンのアドホック多相性は、さまざまな非自由データ型のパターンマッチを許すEgisonにおいて非常に重要である。なぜなら、1つのデータがプログラムの複数の箇所ですべて違う非自由データ型としてパターンマッチされることが多々あるためである。たとえば、リストは多重集合、集合としてパターンマッチされることがある。パターンの多相性によって、パターンコンストラクタの名前の数を大幅に減らすことができる。

下記のmatchAll式は、**コレクション** (*collection*) [1,2,3]をターゲットとして、それぞれ違うマッチャーを使って同じコンス・パターンでパターンマッチしている。ここでコレクションという言葉を使ったが、これは今までリストと呼んでいたもののことを指している。コレクションは、リストとしてパターンマッチされることもあれば、多重集合や集合としてパターンマッチされることがある。そのため、リストや多重集合、集合などとして扱われることがあるデータを総称して以降コレクションと呼ぶ。マッチャーがリストの場合、コンス・パターンは、先頭の要素と残りの要素のコレクションに分解する。マッチャーが多重集合の場合、コンス・パターンは、ある要素と残りの要素のコレクションに分解する。マッチャーが集合の場合、コンス・パターンは、ある要素とターゲットのコレクション自身に分解する。集合のこの挙動は、集合をすべての要素を無限個含むコレクションとしてとらえれば、自然な仕様と考えることができる。 $(\infty - 1 = \infty$ と考える。)

```
matchAll [1,2,3] as list something with $x :: $xs -> (x,xs)
-- [(1,[2,3])]
matchAll [1,2,3] as multiset something with $x :: $xs -> (x,xs)
-- [(1,[2,3]),(2,[1,3]),(3,[1,2])]
matchAll [1,2,3] as set something with $x :: $xs -> (x,xs)
-- [(1,[1,2,3]),(2,[1,2,3]),(3,[1,2,3])]
```

パターンの多相性は特に値パターンを表現するときに便利である。コンス・パターンなどのコンストラクタパターンと同様に値パターンの挙動もマッチャーによって異なる。たとえば、`[1,2,3] == [2,1,3]`のようなコレクション同士の同値性は、コレクションをリストとしてみなすか多重集合としてみなすかによって変わってくる。しかし Egison では、パターンのアドホック多相性のおかげで、両者の同値性を同じパターンでチェックできる。値パターンの多相性によって、非自由データ型を扱うプログラムが大幅に読みやすくなる。

```
matchAll [1,2,3] as list integer with #[2,1,3] -> "Matched" -- []
matchAll [1,2,3] as multiset integer with #[2,1,3] -> "Matched" -- ["Matched"]
```

2.5 matchAllDFS 式によるパターンマッチ結果の順序の制御

`matchAll`式は可算無限個の結果すべてを列挙するように内部のパターンマッチアルゴリズムが設計されているが、その列挙の仕方は処理系によって決められている。場合によっては、この順序が重要であることがある。

典型的な例を紹介する。以下の `matchAll`式はすべての自然数のペアを列挙する。`take`関数を使って先頭 8 個のペアを取り出している。`matchAll`はパターンマッチの探索木をすべてのノードをたどるために幅優先探索する。^{*4} その結果、パターンマッチの結果の順序は以下のようになる。

```
take 8 (matchAll [1..] as set something with
  | $x :: $y :: _ -> (x,y))
-- [(1,1),(1,2),(2,1),(1,3),(2,2),(3,1),(2,3),(3,2)]
```

上記の順序は、無限に大きい可能性がある探索木をすべてたどる場合に適している。しかし、この順序が好ましくない場面もある。パターンマッチの探索木を深さ優先探索する `matchAllDFS`は、このような場面のために用意されている。

```
take 8 (matchAllDFS [1..] as set something with
  | $x :: $y :: _ -> (x,y))
-- [(1,1),(1,2),(1,3),(1,4),(1,5),(1,6),(1,7),(1,8)]
```

^{*4} この幅優先探索の詳細については、Egison パターンマッチの設計についての論文 [7] の 5.2 節にて詳しく解説されている。

たとえば、以下のようにパターンマッチにより `concat`関数を定義する場合、`matchAllDFS`式を使うことで正しい順番で結果のコレクションを生成できる。

```
def concat xss := matchAllDFS xss as list (list something) with
  | _ ++ ( _ ++ $x :: _ ) :: _ -> x
```

もし、`matchAllDFS`の代わりに`matchAll`を使ってしまうと、以下のように、引数のリストのリストの要素を交互に列挙してしまう。

```
take 10 (matchAll [[1..], map neg [1..]] as list (list something) with
  | _ ++ ( _ ++ $x :: _ ) :: _ -> x)
-- [1,2,-1,3,-2,4,-3,5,-4,6]
```

2.6 and パターン・or パターン・not パターン

and パターンや *or* パターン、*not* パターンといった論理パターンは、パターンの表現力を広げるために重要な役目を果たす。and パターンは、2つのパターンをとり、両方のパターンがマッチする場合、その and パターン自体がパターンマッチに成功する。or パターンは、2つのパターンをとり、どちらか1つのパターンがマッチする場合、その or パターン自体がパターンマッチに成功する。not パターンは、1つのパターンをとり、そのパターンのマッチに失敗した場合、not パターン自体のパターンマッチに成功する。

and パターンと or パターンの使用例として、三つ子素数を抽出するパターンマッチを紹介する。三つ子素数とは、 $(p, p+2, p+6)$ または $(p, p+4, p+6)$ の形で表される素数の三つ組のことをいう。or パターン $(\#(p+2) \mid \#(p+4))$ は、 $p+2$ と $p+4$ の両方にマッチするために使われている。and パターン $(\#(p+2) \mid \#(p+4)) \ \& \ \m は、 $p+2$ または $p+4$ にマッチした場合、その値を $\$m$ に束縛するために使われている。この and パターンの使い方は、Haskell に提供されている *as* パターンの使い方に似ている。

```
def primeTriples := matchAll primes as list integer with
  | _ ++ $p :: ((#(p+2) \mid \#(p+4)) \& $m) :: \#(p+6) :: _
  -> (p, m, p+6)

take 6 primeTriples -- [(5,7,11),(7,11,13),(11,13,17),(13,17,19),(17,19,23),(37,41,43)]
```

not パターンは、その名前が示すとおり、ターゲットがパターンとマッチしない場合にパターンマッチに成功する。not パターンの先頭には`!`が付加され、`!`の後に任意のパターンが続く。以下の `matchAll` は双子素数でない隣り合う素数のペアを列挙する。not パターン $\! \#(p+2)$ は、 $p+2$ 以外の値にマッチするパターンを表現している。

```
take 10 (matchAll primes as list integer with
  | _ ++ $p :: (!\#(p+2) \& $q) :: _ -> (p, q))
-- [(2,3),(7,11),(13,17),(19,23),(23,29),(31,37),(37,41),(43,47),(47,53),(53,59)]
```

2.7 ループ・パターン

ループ・パターン (*loop pattern*) はパターンの複数回の繰り返しを表現するための組み込みパターンである。ループ・パターンは正規表現のクリーネスター演算子 (*) の拡張である。

ターゲットのコレクションの 2 つの要素の組み合わせを列挙するパターンマッチを考えることから始める。これは以下のような `matchAll` 式で記述できる。

```
def comb2 xs := matchAll xs as list something with
  | _ ++ $x_1 :: _ ++ $x_2 :: _ -> [x_1, x_2]

comb2 [1,2,3,4] -- [[1,2],[1,3],[2,3],[1,4],[2,4],[3,4]]
```

Egison はこの例の中の `$x_1` と `$x_2` のように、パターン変数に添字を付加することを許す。^{*5}これらは**添字付き変数**と呼ばれ、 x_1 や x_2 のような数式に対応する。_のあとに続く式は、添字と呼ばれ、整数に評価される必要がある。 `x_i_j_k` のように任意個の添字を変数に付加することができる。添字付き変数 `$x_i` に値が束縛されたとき、もし変数 `x` にまだ何も束縛されていなかった場合に、Egison 処理系は、連想配列を生成し、`x` に束縛する。この連想配列のキーは整数 `i` であり、それに対応する値は添字付き変数 `$x_i` にマッチした値である。変数 `x` にすでに連想配列が束縛されている場合には、新しいキーとバリューのペアがこの連想配列に追加される。

上記の `comb2` の 2 を `n` に一般化してみよう。ここで、ループ・パターンを使うことができる。

```
def comb n xs := matchAll xs as list something with
  | loop $i                -- 添字変数
    (1, n)                 -- 添字範囲
    (_ ++ $x_i :: ...)     -- 繰り返しパターン
    _                      -- 終端パターン
  -> map (\i -> x_i) [1..n]

comb 2 [1,2,3,4] -- [[1,2],[1,3],[2,3],[1,4],[2,4],[3,4]]
comb 3 [1,2,3,4] -- [[1,2,3],[1,2,4],[1,3,4],[2,3,4]]
```

ループ・パターンは、**添字変数**、**添字範囲**、**繰り返しパターン**、**終端パターン**を引数にとる。添字変数（上記の場合、`$i`）は、現在の繰り返し回数を保持する変数である。添字範囲（上記の場合、`(1, n)`）は、添字変数が動く範囲を指定するために使われる。添字範囲は、**開始値**と**終了値**のペアである。繰り返しパターン（上記の場合、`(_ ++ $x_i :: ...)`）は、添字変数が添字範囲のなかにいる間、繰り返されるパターンである。終端パターン（上記の場合、`_`）は、添字変数が添字範囲の外に動いたときに展開されるパターンである。繰り返しパターンの中では、**三点リーダーパターン**...を使うことができる。繰り返しパターンや終端パターンは、三点リーダーパターンの場所に展

^{*5} このため、Egison では `snake_case` によって命名された変数を使うことができない。

開される．三点リーダーパターンで繰り返しパターン展開されるとき，添字変数の値がインクリメントされる．たとえば， $n = 3$ のとき，上記のループ・パターンは，以下のように展開される．

```
1 (loop $i (1, 3) (_ ++ $x_i :: ...)) _
2 _ ++ $x_1 :: (loop $i (2, 3) (_ ++ $x_i :: ...)) _
3 _ ++ $x_1 :: _ ++ $x_2 :: (loop $i (3, 3) (_ ++ $x_i :: ...)) _
4 _ ++ $x_1 :: _ ++ $x_2 :: _ ++ $x_3 :: (loop $i (4, 3) (_ ++ $x_i :: ...)) _
5 _ ++ $x_1 :: _ ++ $x_2 :: _ ++ $x_3 :: _
```

上記のループ・パターンの繰り返し回数は定数だった．しかし，添字範囲の終了値を整数値でなくパターンにすることにより，ターゲットによりループ・パターンの繰り返し回数が変わることができる．添字範囲の終了値がパターンである場合，三点リーダーパターンは繰り返しパターンと終端パターンの両方に展開される．三点リーダーパターンが終端パターンに展開されたときの繰り返し回数が，添字範囲の終了値のパターンとパターンマッチされる．以下のループ・パターンは，ターゲットのコレクションの先頭部分を列挙する．

```
matchAll [1,2,3,4] as list something with
| loop $i (1, $n) ($x_i :: ...) _ -> map (\i -> x_i) [1..n]
-- [[], [1], [1,2], [1,2,3], [1,2,3,4]]
```

上記，2通りの添字範囲の指定を紹介したが，これらは下記のように，より一般的な添字範囲の指定方法に展開される． $(1, n)$ は下記の3つ目のケースに， $(1, \$n)$ は下記の4つ目のケースに対応する．

(開始値)	$\Leftrightarrow$ (開始値, [開始値..], _)
(開始値, 終了値のリスト)	$\Leftrightarrow$ (開始値, 終了値のリスト _)
(開始値, 終了値)	$\Leftrightarrow$ (開始値, [終了値] _)
(開始値, 終端パターン)	$\Leftrightarrow$ (開始値, [開始値..], 終端パターン)

一般に，添字範囲は，開始値と終了値のリスト，終端パターンの3つからなる．開始値は省略できない．終了値のリストが省略された場合は，開始値から始まる無限リストが終了値のリストとなる．終了値のリストがリストでなく，整数値であった場合，その整数値だけを含むリストに変換される．終端パターンが省略された場合，ワイルドカードが補完される．たとえば， $(1, n)$ は $(1, [n], _)$ ， $(1, \$n)$ は $(1, [1..], \$n)$ にそれぞれ展開される．

ループ・パターンは，ツリーやグラフのパターンマッチをするときに特によく使われる．第3章 3.4 節でそのような例を紹介する．形式的なループ・パターンの構文と意味論は，ループ・パターンの論文 [5] で解説されている．

2.8 シーケンシャル・パターン

Egison 処理系はパターンを左から右に順番に処理する．しかし，パターンの右側で束縛されるパターン変数の値を参照したいなど，この処理の順番を変えたいことがある．シーケンシャル・パ

ターン (*sequential pattern*) はそのために用意された組み込みパターンである。

シーケンシャル・パターンは、パターンマッチの処理の順番をユーザーが変えることを許す。シーケンシャル・パターンは、パターンのリストとして表現される。パターンマッチは、リストの先頭から順番に実行される。以下のシーケンシャルパターンは、リストの3つ目の要素、1つ目の要素、2つ目の要素の順番でターゲットのリストをパターンマッチする。

```
matchAll [2,3,1,4,5] as list integer with
| { @ :: @ :: $x :: _,
    (#(x + 1), @ ),
    #(x + 2)}
-> "Matched" -- ["Matched"]
```

シーケンシャル・パターン中に現れる@は、**後回しパターン変数** (*later pattern variable*) と呼ばれる。後回しパターン変数に束縛されたターゲットは、シーケンシャル・パターンの次の要素のパターンでパターンマッチされる。複数の後回しパターン変数が現れた場合、次のシーケンスはそれらをまとめてタプルとしてパターンマッチする。

シーケンシャル・パターンのこの特徴は、パターンの離れた複数の部分についてまとめて not パターンを適用することを可能にする。たとえば、以下のシーケンシャル・パターンは、ターゲットのコレクションのペアが、共通の要素をちょうど1つだけ含む場合に、パターンマッチに成功する。シーケンシャル・パターンは、それぞれのコレクションに共通の要素があることをチェックした後に、それぞれの残りの要素のコレクションにこれ以上共通要素がないことをチェックすることを可能にする。シーケンシャル・パターンと not パターンの組み合わせは、数学的なアルゴリズムを記述するときに現れることがある。たとえば、第4章4.1節でも現れる。

```
def singleCommonElem xs ys := match (xs, ys) as (multiset eq, multiset eq) with
| {($x :: @, #x :: @),
   !($y :: _, #y :: _)} -> True
| _ -> False
```

シーケンシャル・パターンと同等のことが matchAll 式のネストにより表現できるのではと考えた読者がいるかもしれない。実は、少なくとも2つの理由でこれは不可能である。1つ目の理由は、ネストした matchAll 式は、Egison 内部の幅優先探索を壊すことに由来する。外側の matchAll 式の2番目の結果は、外側の matchAll 式1つ目の結果について内側の matchAll 式の結果をすべて評価した後に計算される。2つ目の理由は、後回しパターン変数がターゲットだけでなく、マッチャーの情報も保持することである。matchAll の引数のマッチャーが、関数の引数に由来するパラメーターであることがある。それゆえ、内側の matchAll 式で使うべきマッチャーが構文的に導けないことがある。

2.9 パターン関数によるパターンのモジュール化

コンス・パターンやジョイン・パターンのようなパターン・コンストラクタは、第 6 章で解説されるようにマッチャーで定義される。これらのパターン・コンストラクタを組み合わせて、新しいパターン・コンストラクタをつくることができる。そのためには、パターンを引数にとってパターンを返す**パターン関数** (*pattern function*) を使う。

パターン関数を定義する構文はラムダ式の構文と似ているが、矢印に $\rightarrow$ の代わりに $\Rightarrow$ を使う点異なる。以下で定義されているパターン関数 `twin` は、それぞれの第一引数が同じ値である場合にマッチするような、二重にネストしたコンス・パターンをモジュール化している。パターン関数の仮引数は**変数パターン** (*variable pattern*) と呼ばれる。この例の変数パターンは、`pat1` と `pat2` である。パターン関数のボディで、変数パターンの中身を参照するときは、`~`を変数パターンの先頭に付加する。これは変数パターンをパターン・コンストラクタと区別するためである。

```
def twin := \pat1 pat2 => (~pat1 & $x) :: #x :: ~pat2
```

この `twin` パターン関数は、以下のように動作する。

```
match [1, 1, 2, 3] as list integer with
| twin $n $ns -> [n, ns]
-- [1, [2, 3]]
```

2.10 マッチャー合成による新しいマッチャーの生成

ここまでに登場したマッチャーは、Egison 唯一の組み込みマッチャー `something` を除いて、すべてユーザーが定義することができる。マッチャーを定義するには、第 6 章で解説する `matcher` 式を使うことが基本であるが、既存のマッチャーを組み合わせて新しいマッチャーをつくることもできる。この方法で、たとえば、多重集合のタプルのマッチャーや多重集合の多重集合のマッチャーを定義することができる。

タプルに対するマッチャーは、マッチャーのタプルにより表現される。**タプル・パターン**はそのようなマッチャーを使ったパターンマッチのときに使われる。たとえば、以下は、第 1 章 1.2 節でも登場した `intersect` 関数の定義である。2 つの集合のタプルに対するマッチャーを使って、コレクションの共通要素をパターンマッチで取り出している。

```
def intersect xs ys := matchAll (xs,ys) as (set eq, set eq) with
| ($x :: _, #x :: _) -> x
```

上記で使われている `eq` マッチャーは、同値性が定義されたデータ型^{*6}のパターンマッチに使える

^{*6} 現在の Egison には静的な型はないが、Haskell というと Eq 型クラスに属するデータ型

ユーザー定義マッチャーである。eqマッチャーに対して、値パターンが使われた場合、同値性のチェックによりパターンマッチがおこなわれる。

また、タプル・マッチャーと、マッチャーを引数にとってマッチャーを返す関数を組み合わせると、さまざまな非自由データ型に対するマッチャーが定義できる。たとえば、グラフの各ノードを整数、グラフの辺を2つのノードのペアとして表現したとき、有向グラフを表すマッチャーは次のように辺の多重集合として定義できる。

```
def graph := multiset (integer, integer)
```

隣接リストにより表現したグラフのマッチャーも定義できる。隣接リストによるグラフは、整数と整数の多重集合のタプルの多重集合として定義される。ここでも整数によりノードのIDを表現している。

```
def adjacencyGraph := multiset (integer, multiset integer)
```

代数的データ型に対するマッチャーはmatcher式によっても定義できるが、代数的データ型に対するマッチャーを簡単な記述で定義できる特別な構文 algebraicDataMatcher式を Egison は提供している。algebraicDataMatcher式は糖衣構文で、matcher式に脱糖される。algebraicDataMatcher式を使えば、たとえば二分木に対するマッチャーは以下のように定義できる。

```
def binaryTree a := algebraicDataMatcher
  | bLeaf a
  | bNode a (binaryTree a) (binaryTree a)
```

代数的データ型に対するマッチャーと非自由データ型に対するマッチャーも組み合わせることができる。たとえば、任意の数の子供をもつツリーで子供の順番を無視するマッチャーは以下のように定義できる。第3章3.4節でこのツリーに対するパターンマッチを紹介する。

```
def tree a := algebraicDataMatcher
  | leaf a
  | node a (multiset (tree a))
```


第 3 章

パターンマッチ指向プログラミング 入門

本章は、第 2 章でみてきたパターンマッチのための構文やさまざまな組み込みパターンを使って、どのようなプログラミングができるのかを解説する。そのために、Egison パターンマッチを活用したプログラミングにおいて頻出するパターンを紹介していく。本章で Egison のパターンマッチを使ったプログラミングの具体的なイメージをつかんでいてもらいたい。

3.1 パターンマッチによるリスト・プログラミング

本節は、リスト・プログラミングが Egison のパターンマッチによってどのように変わるのか観察する。ここでリスト・プログラミングとは、関数型プログラミング言語を含む一般のプログラミング言語のリスト・ライブラリで提供されているような関数を定義するためのプログラミングのことをいう。 `_ ++ $x :: _` という 2 引数目がコンス・パターンのジョイン・パターンは、リスト・プログラミングにおいて頻出する。そのためこのパターンに名前をつけ、**ジョイン・コンス・パターン** (*join-cons pattern*) と呼んでいる。本節で示すように、リストに対する関数の多くがジョイン・コンス・パターンを使って定義できる。

3.1.1 単一のジョイン・コンス・パターン — map 関数とその仲間

list マッチャーについて、パターン `_ ++ $x :: _` は、ターゲットのそれぞれの要素にマッチする。その結果、下記の `matchAll` 式は、`xs` のそれぞれの要素にマッチし、それらに関数 `f` を適用した結果を返す。これはまさに `map` 関数の定義である。型注釈により、`map` 関数が任意の型 `a` のリストを受け取り、型 `b` のリストを返すことを示している。

```
def map (f: a -> b) (xs: [a]) : [b] := matchAll xs as list something with
  | _ ++ $x :: _ -> f x
```

この `matchAll` 式を変更して、さまざまな関数を定義できる。たとえば、`filter` 関数は述語パター

ンを挿入して定義できる。型注釈により、述語関数 `pred` が型 `a` の値を受け取り `Bool` を返すことを示している。

```
def filter (pred: a -> Bool) (xs: [a]) : [a] := matchAll xs as list something with
| _ ++ (?pred & $x) :: _ -> x
```

`member` 関数は値パターンを挿入することにより定義できる。`member` 関数は第一引数の要素が第二引数のコレクションに含まれているかどうか判定する述語である。`member` 関数は、`match` 式を使って定義する。`match` 式は、`car (matchAll ...)`^{*1} のエイリアスとして実装されている。この実装が可能であるのは、Egison が `matchAll` 式を遅延評価するためである。型注釈により、`member` 関数が任意の型 `a` の値とリストを受け取り、`Bool` を返すことを示している。

```
def member (x: a) (xs: [a]) : Bool := match xs as list eq with
| _ ++ #x :: _ -> True
| _ -> False
```

`delete` 関数は、`member` 関数をすこし編集して実装できる。`delete` 関数は、第一引数 `x` の最初の出現を第二引数のコレクション `xs` から取り除く。

```
def delete (x: a) (xs: [a]) : [a] := match xs as list eq with
| $hs ++ #x :: $ts -> hs ++ ts
| _ -> xs
```

述語 `any` と `every` も同様に `match` 式により実装できる。`any` は第二引数のコレクションの要素のうちに第一引数の述語を満たすものがあるかどうか判定する述語である。`every` は第二引数のコレクションの要素のすべてが第一引数の述語を満たすかどうか判定する述語である。

```
def any (pred: a -> Bool) (xs: [a]) : Bool := match xs as list something with
| _ ++ ?pred :: _ -> True
| _ -> False

def every (pred: a -> Bool) (xs: [a]) : Bool := match xs as list something with
| _ ++ !?pred :: _ -> False
| _ -> True
```

3.1.2 ジョイン・コンス・パターンのネスト — `unique`・`concat` 関数

複数のジョイン・コンス・パターンを組み合わせることで、さらに強力なパターンをつくることができる。その例として `unique` 関数を紹介する。`unique` 関数をパターンマッチ指向プログラミングにより定義すると以下ようになる。型注釈により、`unique` 関数が任意の型 `a` のリストを受け取り、同じ型のリストを返すことを示している。

^{*1} `car` はコレクションの先頭要素を取り出す関数である。

```
def unique (xs: [a]) : [a] := matchAllDFS xs as list eq with
  | _ ++ $x :: !(_ ++ #x :: _) -> x
```

not パターンが、 x が x のあとにそれ以上現れないことを表現するために使われている。その結果このパターンは、それぞれの要素の最後の出現にマッチする。

```
unique [1,2,3,2,4] -- [1,3,2,4]
```

述語パターンを上手に使うと、それぞれの要素の最初の出現からなるコレクションを返す unique 関数を定義することも可能である。最初の出現だけにマッチするために、その要素より前に同じ要素が出現しないときにマッチするパターンを書く必要がある。Egison のパターンマッチはパターンを左から右に処理するため、そのようなパターンは、コンス・パターンとジョイン・パターンを単純に組み合わせるだけでは書けない。

```
def unique2 (xs: [a]) : [a] := matchAllDFS xs as list eq with
  | $hs ++ (!?(\x -> member x hs) & $x) :: _ -> x

unique2 [1,2,3,2,4] -- [1,2,3,4]
```

もうひとつのもっとエレガントな方法に、シーケンシャル・パターンを使う方法がある。同じ意味のパターンを、ジョイン・パターンの第一引数に後回しパターン変数を使うシーケンシャル・パターンにより表現できる。

```
def unique (xs: [a]) : [a] := matchAllDFS xs as list eq with
  | { @ _ ++ $x :: _ ,
      !(_ ++ #x :: _) }
  -> x
```

ネストしたジョイン・コンス・パターンのもうひとつの例に、第 2 章 2.5 節で紹介した concat 関数がある。マッチャー合成 (第 2 章 2.10 節) とジョイン・コンス・パターンを組み合わせることにより、concat をパターンマッチ指向プログラミングにより定義できる。型注釈により、concat 関数がリストのリストを受け取り、フラット化されたリストを返すことを示している。

```
def concat (xss: [[a]]) : [a] := matchAllDFS xss as list (list something) with
  | _ ++ (_ ++ $x :: _) :: _ -> x
```

3.2 多重集合プログラミング

多重集合を直接パターンマッチできることは、Egison の大きな特徴である。多重集合のパターンマッチを活かしたプログラミングのコツを本節では紹介したい。これ以降の本書に出てくるパターンマッチはほぼ多重集合のパターンマッチである。

3.2.1 多重集合のコンス・パターン

multisetマッチャーのコンス・パターンは、要素の順序を無視してコレクションのパターンマッチをするのに便利である。数学のアルゴリズムを記述するとき、このような状況はよく現れる。

簡単な例からはじめる。連想リストに対する lookup関数は、単一のコンス・パターンで定義できる。単一の多重集合のコンス・パターンは、リストのジョイン・コンス・パターンと置き換えることもできる。型注釈により、lookup関数がキー kと連想リストを受け取り、対応する値を返すことを示している。

```
def lookup (k: a) (ls: [(a, b)]) : b := match ls as multiset (eq, something) with
| (#k, $x) :: _ -> x
```

ただし、ネストした多重集合のコンス・パターンは、リストのジョイン・コンス・パターンと置き換えることができない。 k 重にネストした多重集合のコンス・パターンは $P(n, k) = \frac{n!}{(n-k)!}$ 通りの k 個の要素のパーミュテーションを列挙するために使えるのに対し、 k 重にネストしたリストのジョイン・コンス・パターンは $C(n, k) = \frac{n!}{k!(n-k)!}$ 通りの k 個の要素の組み合わせを列挙するために使える。

```
matchAll [1, 2, 3] as list something with
| _ ++ $x :: _ ++ $y :: _ -> (x, y)
-- [(1,2),(1,3),(2,3)]

matchAll [1, 2, 3] as multiset something with
| $x :: $y :: _ -> (x, y)
-- [(1,2),(1,3),(2,1),(2,3),(3,1),(3,2)]
```

上記と同等なプログラムは、従来の関数型プログラミングの一般的な方法では、リスト内包表記によって以下のように記述できる。tailsは、引数のリストの後方部分からなる部分リストを返す関数である。splitsは、引数のリストを先頭部分のリストと残りのリストに分解したものを返す関数である。

```
[ (x, y) | x : ts <- tails [1, 2, 3], y <- ts ]
-- [(1,2),(1,3),(2,3)]

[ (x, y) | (hs, x : ts) <- splits [1, 2, 3], y <- hs ++ ts ]
-- [(1,2),(1,3),(2,1),(2,3),(3,1),(3,2)]
```

多重集合のためのライブラリは、従来のプログラミング言語にも用意されているものの、多重集合として扱いたいコレクションをリストとして捉え直す必要がある場面は多い。実際、上記のリスト内包表記による $C(n, 2)$ 個の要素の組み合わせの列挙では、splits関数と ++関数という 2 つのリストを処理するための関数を使っており、対象のコレクションをリストとして捉え直した処理を記述している。多重集合に対するパターンマッチは、このようなコレクションをリストと

して捉え直す処理をパターンの定義（この例の場合，multisetマッチャーの::パターンの定義）に隠すことによって，多重集合として扱いたいコレクションを直接多重集合として扱える機会を広げる．本書のこれ以降の部分は，多重集合に対するさまざまなパターンを，第2章で紹介したパターンと組み合わせて記述していく．

3.2.2 ポーカーの役判定

多重集合に対する非線形パターンの応用例としてポーカーの役判定を紹介する．多重集合に対する非線形パターンにより，ポーカーのすべての役はそれぞれ1つのパターンとして表現できる．*2“ $[p_1, p_2, \dots, p_n]$ ”というパターンは，“ $p_1 : p_2 : \dots : p_n : []$ ”に展開される糖衣構文である．

```
def poker (cs: [Card]) : String :=
  match cs as multiset card with
  | card $s $n :: card #s #(n-1) :: card #s #(n-2) :: card #s #(n-3) :: card #s #(n-4) ::
    -
    -> "Straight flush"
  | card _ $n :: card _ #n :: card _ #n :: card _ #n :: _ :: []
    -> "Four of a kind"
  | card _ $m :: card _ #m :: card _ #m :: card _ $n :: card _ #n :: []
    -> "Full house"
  | card $s _ :: card #s _ :: card #s _ :: card #s _ :: card #s _ :: []
    -> "Flush"
  | card _ $n :: card _ #(n-1) :: card _ #(n-2) :: card _ #(n-3) :: card _ #(n-4) :: []
    -> "Straight"
  | card _ $n :: card _ #n :: card _ #n :: _ :: _ :: []
    -> "Three of a kind"
  | card _ $m :: card _ #m :: card _ $n :: card _ #n :: _ :: []
    -> "Two pair"
  | card _ $n :: card _ #n :: _ :: _ :: _ :: []
    -> "One pair"
  | _ :: _ :: _ :: _ :: _ :: _ :: [] -> "Nothing"
```

この poker関数は以下のように動く．

```
poker [Card Spade 5, Card Spade 6, Card Spade 7, Card Spade 8, Card Spade 9]
-- "Straight flush"
poker [Card Spade 5, Card Diamond 5, Card Spade 7, Card Club 5, Card Heart 5]
-- "Four of a kind"
poker [Card Spade 5, Card Diamond 5, Card Spade 7, Card Club 5, Card Heart 7]
-- "Full house"
```

*2 著者が Egison のマッチ式を設計したとき，ポーカーの役判定をするプログラムを簡潔に記述できることを必要条件として設計した．

```

poker [Card Spade 5, Card Spade 6, Card Spade 7, Card Spade 13, Card Spade 9]
-- "Flush"
poker [Card Spade 5, Card Club 6, Card Spade 7, Card Spade 8, Card Spade 9]
-- "Straight"
poker [Card Spade 5, Card Diamond 5, Card Spade 7, Card Club 5, Card Heart 8]
-- "Three of a kind"
poker [Card Spade 5, Card Diamond 10, Card Spade 7, Card Club 5, Card Heart 10]
-- "Two pair"
poker [Card Spade 5, Card Diamond 10, Card Spade 7, Card Club 5, Card Heart 8]
-- "One pair"
poker [Card Spade 5, Card Spade 6, Card Spade 7, Card Spade 8, Card Diamond 11]
-- "Nothing"

```

パターンコンストラクタ (card) は小文字から始まり、データコンストラクタ (Card) は大文字から始まるというルールが Egison にはある。poker関数の型注釈により、カードのリストを受け取り、役を表す文字列を返すことを示している。

なお、上の例でカードのマッチャーは以下のように代数的データマッチャーとして定義されている。

```

def suit := algebraicDataMatcher
  | spade
  | heart
  | club
  | diamond

def card := algebraicDataMatcher
  | card suit (mod 13)

```

3.3 タプル・パターンによるデータの比較

複数のデータをくらべたいという場面は、プログラミングにおいて頻出する。このような場面で、とくに not パターンと一緒に使われるタプル・パターンが役に立つことがおおい。たとえば、difference関数は、第 2 章 2.10 節に登場した intersect関数の実装に not パターンを挿入することにより定義できる。型注釈により、2 つの集合を受け取り、差集合を返すことを示している。

```

def difference (xs: [a]) (ys: [a]) : [a] := matchAll (xs, ys) as (set eq, set eq) with
  | ($x :: _, !(#x :: _)) -> x

```

!(\$x :: _, #x :: _)のように、not パターンの挿入位置を変えることにより、2 つのコレクションに共通要素が 1 つもない場合にパターンマッチに成功するパターンも記述できる。

これらのパターンをシーケンシャル・パターンと組み合わせることにより、さらに複雑なパターンを記述することができる。シーケンシャル・パターンは、not パターンを、パターンの複数の離

れた部分にまとめて適用することを可能にするからである。たとえば、第 2 章 2.8 節では、2 つのコレクションに共通要素が 1 つしかないことをチェックする `singleCommonElem` 関数をパターンマッチにより定義した。シーケンシャル・`not` パターンは、数学のアルゴリズムを記述するときに必要なことがある。たとえば、第 4 章 4.1 節で紹介する SAT ソルバーの実装でもシーケンシャル・`not` パターンは現れる。型注釈により、2 つのリストを受け取り、真偽値を返すことを示している。

```
def singleCommonElem (xs: [a]) (ys: [a]) : Bool := match (xs, ys) as (multiset eq,
    multiset eq) with
    | [($x :: @, #x :: @),
      !($y :: _, #y :: _)] -> True
    | _ -> False
```

シーケンシャル・パターンをループ・パターンと組み合わせることも可能である。たとえば、2 つのリストの共通の先頭部分にマッチするパターンを、シーケンシャル・ループ・パターンにより記述できる。

```
match (xs, ys) as (list eq, list eq) with
| loop $i (1,$n)
  [($x_i :: @, #x_i :: @), [...]]
  !($y :: _, #y :: _)
-> map (\i -> x_i) [1..n]
```

3.4 ループ・パターンによる再帰的模式

ループ・パターンは、パターンの繰り返しを表現するために使われる。ループ・パターンは、シンプルなパターンコンストラクタを組み合わせることで複雑なパターンを構築したり、パターン変数の数がパラメーターによって変わるパターンを記述するときに役に立つ。そのような状況では、複雑な再帰を記述する必要がでてくる。ループ・パターンを使うと、それらの再帰をパターンに押し込め、直感的な記述ができる。本節では、そのような例を紹介していく。

3.4.1 N クイーン問題

本節は、N クイーン問題を解くプログラムをみせることにより、技巧的であるがトリッキーなループ・パターンの例を紹介する。N クイーン問題とは、 $n \times n$ のチェス盤に n 個のチェスのクイーンの駒をお互いが攻撃しあえないように配置する問題である。チェスのクイーンは、縦横斜め何マス離れている駒でも攻撃することができる。将棋でいうと、飛車と角行を合わせた動きをチェスのクイーンはできる。

4 クイーン問題をとくプログラムからはじめよう。このプログラムでは、4 つのクイーンの配置をリストで表現する。リストの n 番目の要素は、 n 行目のクイーンが何列目の位置にあるかを表

現する．このとき，解は，コレクション [1,2,3,4] の要素の順番が並び替わったコレクションである必要がある．2 つのクイーンを同じ行や列に配置できないためである．それゆえ，コレクション [1,2,3,4] を整数の多重集合としてパターンマッチする．クイーン同士が斜めのラインを共有できないという条件は， $a_1 \pm 1 \neq a_2$ ， $a_1 \pm 2 \neq a_3$ ， $a_2 \pm 1 \neq a_3$ ， $a_1 \pm 3 \neq a_4$ ， $a_2 \pm 2 \neq a_4$ ， $a_3 \pm 1 \neq a_4$ という条件により表現する．

```
matchAll [1,2,3,4] as multiset integer with
  $a_1 ::
    (!#(a_1 - 1) & !#(a_1 + 1) & $a_2) ::
    (!#(a_1 - 2) & !#(a_1 + 2) & !#(a_2 - 1) & !#(a_2 + 1) & $a_3) ::
    (!#(a_1 - 3) & !#(a_1 + 3) & !#(a_2 - 2) & !#(a_2 + 2) & !#(a_3 - 1) & !#(a_3 + 1) &
      $a_4) ::
    [] -> [a_1,a_2,a_3,a_4]
-- [[2,4,1,3],[3,1,4,2]]
```

上記のプログラムを n クイーン問題のために一般化するには，二重にネストしたループ・パターンを使うことができる．外側のループ・パターンの添字変数 i が，内側のループ・パターン添字範囲にて，内側のループの繰り返し回数の違いを表現するために使われている．また，前の繰り返しパターンで束縛された値，たとえば， a_j に束縛された値が， $\#(a_j - (i - j))$ や $\#(a_j + (i - j))$ のように参照されていることにも注意してほしい．このように，添字付きパターン変数の非線形性（パターンの左側で添字付きパターン変数に束縛した値が参照できること）がうまく機能している．型注釈により，`nQueens` 関数が整数 n を受け取り，整数のリストのリストを返すことを示している．

```
def nQueens (n: Integer) : [[Integer]] :=
  matchAll [1..n] as multiset integer with
  | $a_1 ::
    (loop $i (2,n)
      ((loop $j (1, i - 1)
        (!#(a_j - (i - j)) & !#(a_j + (i - j)) & ...)
        $a_i) :: ...)
      [] -> map (\i -> a_i) [1..n])

nQueens 4 -- [[2,4,1,3],[3,1,4,2]]
```

3.4.2 ツリーのパターンマッチ

本節は，ツリーのパターンマッチをみせることにより，ループ・パターンの実例を紹介する．本節のツリーのノードは，XML のように任意の個数の部分ツリーを子供としてもつ．また，これらのサブツリーは多重集合として扱われる．このようなツリーに対するマッチャーは第 2 章 2.10 節にて定義した．このマッチャーを本節では使う．

プログラミング言語をカテゴリー分けしたツリーにたいしてパターンを書く．`treeData` がその

カテゴリーツリーを定義している。たとえば, "Egison"は, "pattern-match-oriented" (パターンマッチ指向) カテゴリーと, "Functional programming" (関数型プログラミング) カテゴリーの "Dynamically typed" (動的型付け) サブカテゴリーに属している。データコンストラクタ Nodeと Leafは大文字から始まる。

```
def treeData :=
  Node "Programming language"
    [Node "pattern-match-oriented" [Leaf "Egison"],
     Node "Functional language"
       [Node "Strictly typed" [Leaf "OCaml", Leaf "Haskell", Leaf "Curry", Leaf "Coq"],
        Node "Dynamically typed" [Leaf "Egison", Leaf "Lisp", Leaf "Scheme", Leaf "Racket"
                                   "]],
     Node "Logic programming" [Leaf "Prolog", Leaf "Curry"],
     Node "Object oriented" [Leaf "C++", Leaf "Java", Leaf "Ruby", Leaf "Python", Leaf "
OCaml"]]
```

下記の matchAll式は, ある言語が属するカテゴリーを列挙する。ツリーの末端は任意の深さでありうるため, 添字範囲の終了値がパターン (下記の場合 \$n) であるループ・パターンが使われている。このループ・パターンの三点リーダーパターンは, 繰り返しパターンの末尾でない場所に位置している。繰り返しの場所をユーザーが指定できることも, 正規表現のクリーネスター演算子にはないループ・パターンの強みのひとつである。パターンコンストラクタ nodeと leafは小文字から始まる。型注釈により, ancestors関数が文字列 xとツリー tを受け取り, 祖先ノードのパスのリストを返すことを示している。

```
def ancestors (x: String) (t: Tree String) : [[String]] := matchAll t as tree string with
  | loop $i (1,$n)
    (node $c_i (... :: _))
    (leaf #x)
  -> map (\i -> c_i) [1..n]

ancestors "Egison" treeData
-- ["Programming language", "pattern-match-oriented"], ["Programming language", "
Functional language", "Dynamically typed"]]
```

あるサブカテゴリーに属する言語をすべて列挙するようなパターンを記述することも可能である。二重にネストしたループ・パターンを使えば, そのサブカテゴリーが任意の深さにあってもよいようにできる。以下のパターンは, 特定のカテゴリーに属するすべての言語を列挙する。ツリーのなかで現れる順番と同じ順番で言語を列挙するために, matchAllDFSが使われている。型注釈により, descendants関数が文字列 xとツリー tを受け取り, 子孫ノードのリストを返すことを示している。

```
def descendants (x: String) (t: Tree String) : [String] := matchAllDFS t as tree string
  with
```

```

| loop _ (1,_)
  (node _ (... :: _))
  (node #x ((loop _ (1,_)
                (node _ (... :: _))
                (leaf $y)) :: _))
-> y

descendants "Functional language" treeData
-- ["OCaml", "Haskell", "Curry", "Coq", "Egison", "Lisp", "Scheme", "Racket"]

```

ツリーのパターンマッチができる DSL (domain-specific language) としては、XML path がある。ユーザー定義可能な数少ないパターンコンストラクタとループ・パターンを組み合わせる幅広いパターンを記述できることが Egison の利点である。XML path は、たとえば ancestor コマンドなど、多くの組み込みコマンドを使ってパターンを記述する。

3.4.3 グラフのパターンマッチ

本節では、グラフに対するパターンマッチを紹介する。辺の集合として表現したグラフと、隣接リストとして表現したグラフの両方についてのパターンマッチを紹介する。

辺の集合としてのグラフ

本節は、辺の集合として表現したグラフに対してパターンマッチをおこなう。まず、以下のようなマッチャーとグラフを定義する。

```

def graph := set edge
def edge := algebraicDataMatcher
  | edge integer integer

def graphData :=
  [ Edge 1 2, Edge 2 1, Edge 2 3, Edge 2 4, Edge 3 4, Edge 4 5, Edge 4 6, Edge 4 7
  , Edge 5 4, Edge 5 6, Edge 5 7, Edge 6 4, Edge 6 5, Edge 6 7, Edge 7 4, Edge 7 5
  , Edge 7 6, Edge 7 8, Edge 9 10, Edge 10 7 ]

```

パターンコンストラクタ edge は小文字から始まり、データコンストラクタ Edge は大文字から始まっている。図 3.1 は、上記のグラフを描画している。本節は、このグラフに対していくつかのパターンマッチを紹介する。

あるノード s から 2 辺でたどり着けるノードを列挙するパターンは以下のように書ける。

```

let s := 1 in
  matchAll graphData as graph with
  | edge (#s & $x_1) $x_2 :: edge #x_2 $x_3 :: _
  -> x
-- [1,3,4,5,6,7]

```

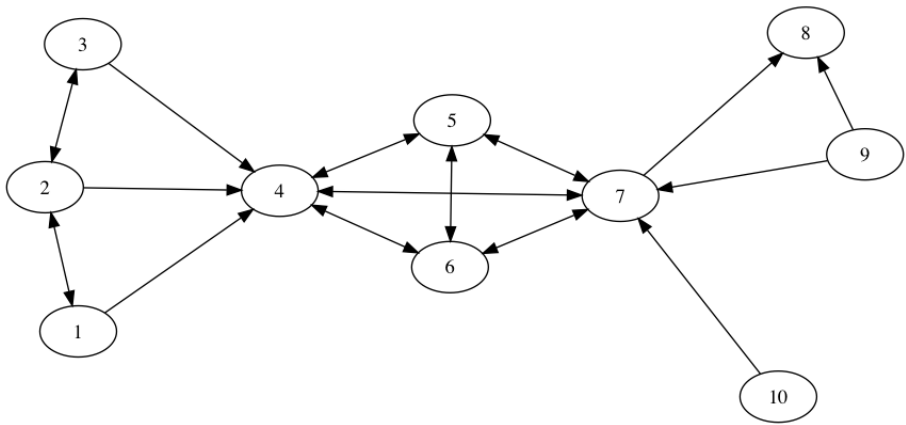


図 3.1: graphDataの描画

ノード s を始点とする辺でノード s とつながっているが、自身を始点とする辺がノード s に対してでないノードを列挙するパターンは `not` パターンを使って以下のように書ける。

```
let s := 1 in
  matchAll graphData as graph with
    | edge #s $x :: !(edge #x #s :: _)
    -> x
-- [4]
```

ノード s からノード e へのパスすべてを列挙するパターンはループ・パターンを使って記述できる。Egison は、パターン中で `let` 式を使うことを許している。この `let` 式は、 x_1 に s を束縛するために使われている。この `let` 式のおかげで、ループ・パターンの最初の繰り返しパターンを特別扱いする必要がなくなっている。

```
let (s, e) := (1, 8) in
  matchAll graphData as graph with
    | let x_1 := s in
      loop $i (2, $n)
        (edge #x_(i - 1) $x_i :: ...)
        (edge #x_(n - 1) (#e & $x_n) :: _)
    -> map (\i -> x_i) [1..n]
-- [[1,4,7,8], ...]
```

サイズ n のクリーク（完全グラフになっている部分グラフ）を列挙するパターンは以下のように書ける。二重にネストしたループ・パターンを使う。

```
let n := 4 in
  matchAll graphData2 as graph with
    | edge $x_1 $x_2 :: loop $i (3, n)
```

```

        (edge #x_1 $x_i :: loop $j (2, i - 1)
          (edge #x_j #x_i :: ...)
          ...)
      -
-> map (\i -> x_i) [1..n]
-- [[4,5,6,7],...]

```

本節は、有向グラフのパターンマッチを紹介した。上記の有向グラフに対するパターンを無向グラフに対するパターンに変更することは簡単で、Edge a bと Edge b aを同一視するようなマッチャーを使えばよい。そのような無向グラフに対するマッチャーは、第 6 章 6.2 節で紹介する `unordered pair` に対するマッチャーを使えば定義できる。

隣接グラフ

本節は、重み付き隣接リストにより表現したグラフのパターンマッチをおこなう。重み付き隣接リストに対するマッチャーは、マッチャー合成（第 2 章 2.10 節）により定義できる。下記のプログラム中の `graphData` は隣接リストにより空路による都市間のネットワークを重み付き隣接リストにより表現している。2つの都市間の移動にかかる時間が、重みとして整数により表現されている。

```

def graph := multiset (string, multiset (string, integer))

def graphData :=
  [("Berlin", [("New York", 14), ("London", 2), ("Tokyo", 14), ("Vancouver", 13)]),
   ("New York", [("Berlin", 14), ("London", 12), ("Tokyo", 18), ("Vancouver", 6)]),
   ("London", [("Berlin", 2), ("New York", 12), ("Tokyo", 15), ("Vancouver", 10)]),
   ("Tokyo", [("Berlin", 14), ("New York", 18), ("London", 15), ("Vancouver", 12)]),
   ("Vancouver", [("Berlin", 13), ("New York", 6), ("London", 10), ("Tokyo", 12)])]

```

以下のプログラム中のパターンは、ベルリンを出発して、すべての都市をまわって、ベルリンに戻るルートにマッチする。このパターンを使って、巡回セールスマン問題を解いている。非線形ループ・パターンを効果的に使っている。型注釈により、`trips`関数がグラフデータから経路のリストを計算し、総距離と都市のリストのタプルのリストを返すことを示している。

```

def trips : [(Integer, [String])] :=
  let n := length graphData in
  matchAll graphData as graph with
  | (#"Berlin", ((s_1, p_1) :: _)) ::
    loop $i (2, n - 1)
      ((s_(i - 1), (s_i, p_i) :: _) :: ...)
      ((s_(n - 1), ("Berlin" & s_n, p_n) :: _) :: [])
  -> (sum (map (\i -> p_i) [1..n]), map (\i -> s_i) [1..n])

head (sortBy (\(_, x), (_, y) -> compare x y) trips)

```

```
-- (46, ["London", "New York", "Vancouver", "Tokyo", "Berlin"])
```

ツリー (3.4.2 節) と同様に、グラフに対するパターンマッチのための DSL がいくつかグラフ・データベースに対するクエリー言語として開発されている。いくつかのユーザー定義パターン・コンストラクタと、ループ・パターンをはじめとする少数の組み込みパターンを組み合わせで多様なパターンを表現できることが、Egison のこれらの DSL にたいする利点である。

第 4 章

パターンマッチ指向プログラミングの効果

本章の目的は、パターンマッチ指向プログラミングがどのような場面でどのように役に立つか明確にすることである。前章までの例は、すべて単一の `matchAll` や `match` で記述した例ばかりを紹介してきたが、本章ではより大きなプログラムの記述の中で Egison のパターンマッチがどのような役割を果たすのかみていく。

4.1 SAT ソルバーの実装

ある程度大きなプログラムを書く中で、パターンマッチ指向プログラミングの効果を見るために、SAT ソルバーを実装する。SAT ソルバーは、与えられた命題論理式が真になる論理変数の割り当てが存在するかどうか判定するプログラムである。SAT ソルバーが入力にとる論理式は、連言標準形 (*conjunctive normal form*) であることが多い。論理式が連言標準形であるとは、論理式がリテラル (*literal*) の選言からなる節の連言になっていることをいう。リテラルとは、 p または $\neg p$ という形をした論理式のことをいう。たとえば、 $(p \vee q) \wedge (\neg p \vee r) \wedge (\neg p \vee \neg r)$ は、 $p = \text{False}$, $q = \text{True}$, $r = \text{True}$ という割り当てをすれば真になる連言標準形の論理式である。

4.1.1 Davis-Putnam アルゴリズム

ここでは Davis-Putnam アルゴリズム [8] という、SAT 問題を解くアルゴリズムのなかでもシンプルなアルゴリズムを実装する。本節の実装では、連言標準形の命題論理式をリテラルのコレクションのコレクションとして表現する。 $\wedge$ と $\vee$ は可換な演算子であるため、このコレクションのコレクションは、リテラルの多重集合の多重集合としてパターンマッチできる。さらに、リテラルを整数として表現し、 p の形のリテラルは正整数、 $\neg p$ の形のリテラルは p に対応する正整数の -1 倍の負整数となるようにする。そうすると、これらの論理式に対するマッチャーは *multiset* (*multiset integer*) と定義できる。以下のプログラムは、Davis-Putnam アルゴリズム

の核の実装である。dp関数は、論理変数のリストと論理式を引数にとり、解がある場合はTrueを、そうでない場合はFalseを返す。型注釈により、dp関数が整数のリスト（論理変数）と整数のリストのリスト（論理式）を受け取り、Boolを返すことを示している。

```

1 def dp (vars: [Integer]) (cnf: [[Integer]]) : Bool :=
2   match (vars, cnf) as (multiset integer, multiset (multiset integer)) with
3   | (_, []) -> True
4   | (_, [] :: _) -> False
5   -- 1-literal rule
6   | (_, ($l :: []) :: _) -> dp (delete (abs l) vars) (assignTrue l cnf)
7   -- pure literal rule (positive)
8   | ($v :: $vs, !((#(neg v) :: _) :: _)) -> dp vs (assignTrue v cnf)
9   -- pure literal rule (negative)
10  | ($v :: $vs, !((#v :: _) :: _)) -> dp vs (assignTrue (neg v) cnf)
11  -- otherwise
12  | ($v :: $vs, _) ->
13    dp vs
14    ((resolveOn v cnf) ++ (deleteClausesWith v (deleteClausesWith (neg v) cnf)))

```

1つ目のマッチ節（3行目）は、入力された論理式が空だった場合、解を持つことを表現している。2つ目のマッチ節（4行目）は、入力された論理式が空節を含んでいた場合、解が存在しないことを表現している。3つ目のマッチ節（6行目）は、1-リテラル・ルール（*1-literal rule*）というルールを表現している。入力の論理式が1つのリテラルからなる節を持つとき、そのリテラルがTrueになる割り当てを即座にできる。4つ目のマッチ節（8行目）は、ある論理変数の否定がリテラルとして全く現れない場合、その論理変数に即座にTrueを割り当てることができることを表現している。たとえば、 $(p \vee q) \wedge (\neg p \vee r) \wedge (\neg p \vee \neg r)$ は論理変数 q の否定を含まないため、 q にTrueを割り当てることができる。5つ目のマッチ節（10行目）は、4つ目のマッチ節と逆のことを表現している。このマッチ節は、ある論理変数の否定のみが入力の論理式に含まれる場合、その論理変数にFalseを割り当てることができることを表現している。最後のマッチ節（12-14行目）は導出原理（*resolution principle*）を適用している。このマッチ節は、 $p \vee C$ と $\neg p \vee D$ という形のすべての節のペアを列挙し（ C と D はリテラルの選言とする）、 $C \vee D$ という形の節を生成する。

上記のdpの定義は、入力の論理式を簡約するためのすべてのルールを多重集合に対するパターンマッチをいかして記述している。同様のことをするために、伝統的な関数型プログラミングでは、複数のライブラリ関数を組み合わせたり、補助関数を定義する必要がある。Davis-Putnam アルゴリズムの OCaml による実装は [8] でみることができる。

4.1.2 導出原理

本節は、resolveOn関数の実装を紹介する。まずは、ナイーブな実装を紹介するところからはじめる。resolveOn関数は、matchAll式を使って以下のように定義できる。型注釈により、resolveOn関

数が整数 v と論理式 cnf を受け取り、導出された節のリストを返すことを示している。

```
def resolveOn (v: Integer) (cnf: [[Integer]]) : [[Integer]] := matchAll cnf as multiset (
  multiset integer) with
| (#v :: $xs) :: (#(negate v) :: $ys) :: _ -> unique (filter (\c -> not (tautology c)) (
  xs ++ ys))
```

$p \vee C$ と $\neg p \vee D$ の形の節のペアを列挙するパターンは、多重集合の多重集合に対するパターンマッチにより記述できる。ただし、これだけでは、 $C \vee D$ がトートロジーな節である場合 (C がリテラル q を含みかつ D がリテラル $\neg q$ を含む場合そうなる) にもマッチしてしまう。そのために、`filter`関数と `tautology`関数を使って、そのような節を取り除いている。シーケンシャル・`not`パターン (第3章 3.3 節) を使って `resolveOn`関数を定義すると、パターンにそのような操作を取り込める。

```
def resolveOn (v: Integer) (cnf: [[Integer]]) : [[Integer]] :=
  matchAll cnf as multiset (multiset integer) with
  | {(#v :: (@ & $xs)) :: (#(neg v) :: (@ & $ys)) :: _,
    !($l :: _, #(neg l) :: _)}
    -> unique (xs ++ ys)
```

4.2 2 種類のループの分離

4.1 節で実装した SAT ソルバーは、パターンマッチ指向プログラミングによって取りのぞけないループを含んでいた。そのループとは、`dp`関数の再帰である。この再帰は、SAT 問題を解くための探索空間を狭めるために本質的なループである。この探索空間の縮小は、単純なバックトラッキング・アルゴリズムでは不可能なものである。たいてい、その他のバックトラッキングにより実装できるループはすべてパターンの中に押し込められている。伝統的な関数型プログラミングでは、これら 2 種類のループのどちらも再帰を使って記述する必要があった。パターンマッチ指向プログラミングは、バックトラッキングにより実装できるループをパターンに閉じ込めることにより、時間的計算量を減らすための本質的なループのみの記述にプログラマが注力できるようにする。これにより、プログラムの記述しやすさ・読解しやすさが向上する。

第 5 章

Egison パターンマッチの仕組み

本章は、Egison 内部のパターンマッチの仕組みを解説する。まず、5.1 節で概略を説明する。5.2 節以降では、Egison パターンマッチを自身で実装するために必要な事項を解説する。単に Egison の使い方を知りたい読者は、5.2 節以降を読み飛ばして第 6 章に進んでも構わない。

5.1 パターンマッチ・アルゴリズムの概略

以下の `matchAll` 式を実行したときにどのような処理がなされるのかみていくことにより、Egison 内部のパターンマッチ・アルゴリズムの概略をつかむ。

```
matchAll [2,8,2] as multiset eq with
| $m :: #m :: _ -> m
-- [2,2]
```

上記の `matchAll` 式を受け取ると Egison 処理系は、以下のような **マッチング・ステート** (*matching state*) と呼ばれるオブジェクトを処理系内部で生成する。マッチング・ステートは、**マッチング・アトム** (*matching atom*) のスタックと、パターンマッチの途中結果 (下記の場合, `[]`), `matchAll` 式実行時の環境 (下記の場合, `env`) からなる。マッチング・アトムは、パターン、マッチャー、ターゲットからなる 3 つ組である。初期マッチング・ステートは、`matchAll` 式のパターン、マッチャー、ターゲットから構成されるマッチング・アトム 1 つからなるスタックから構成される。Egison 内部のパターンマッチ・アルゴリズムは、マッチング・ステートの簡約プロセスとして定義されている。簡約の結果、マッチング・アトムのスタックが空になるとパターンマッチに成功する。

```
MState [($m :: #m :: _, multiset eq, [2,8,2])]
      [] env
```

このマッチング・ステートは、次のステップで以下の 3 つのマッチング・ステートに簡約される。この簡約は `multiset` マッチャーのコンス・パターンの定義 (第 6 章 6.4 節にて解説する) にもとづいて実行される。

```
MState [($m, eq, 2)]
```

```

, (#m :: _, multiset eq, [8,2])
[] env

MState [($m, eq, 8)
, (#m :: _, multiset eq, [2,2])]
[] env

MState [($m, eq, 2)
, (#m :: _, multiset eq, [2,8])]
[] env

```

このように1つのマッチング・ステートを1ステップ簡約すると複数のマッチング・ステートが生成される。生成されるマッチング・ステートの数が0個であることも、無限個ある場合もある。これらのマッチング・ステートがどのような順番で簡約されるのかは、5.2節で論じる。ここでは、1つ目のマッチング・ステートの簡約をみる。1つ目のマッチング・ステートは、次のステップで以下のように簡約される。スタックの先頭のマッチング・アトムのマッチャーがeqからsomethingに変わる。この簡約は、eqマッチャーに定義されている。(第6章6.3節でその定義をみる。)

```

MState [($m, something, 2)
, (#m :: _, multiset eq, [8,2])]
[] env

```

somethingはEgison 唯一の組み込みマッチャーであり、パターンマッチの途中結果に新しい束縛を追加することができる。ここでは、変数mに2を束縛している。

```

MState [(#m :: _, multiset eq, [8,2])]
[(m, 2)] env

```

再び、multisetのコンス・パターンの定義にもとづき、マッチング・ステートが簡約される。

```

MState [(#m, eq, 8)
, (_, multiset eq, [2])]
[(m, 2)] env

MState [(#m, eq, 2)
, (_, multiset eq, [8])]
[(m, 2)] env

```

上記、1つ目のマッチング・ステートは値パターン#mのパターンマッチに失敗して消える。(簡約された結果0個のマッチング・ステートを生成するともいうことができる。)2つ目のマッチング・ステートは、先頭のマッチング・アトムが解決されて以下ようになる。

```

MState [(_, multiset eq, [8])]
[(m, 2)] env

```

このマッチング・ステートも、先頭のマッチング・アトムが解決されて以下ようになる。

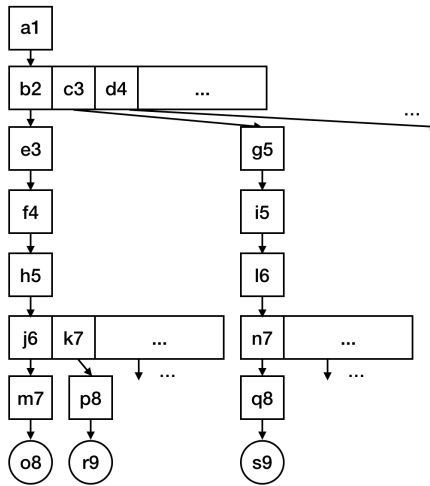


図 5.1: matchAllDFS式の探索木

```
MState []
  [(m, 2)] env
```

最終的にマッチング・アトムが上記のように空になると、このパターンマッチの途中結果が最終結果としてボディの評価に使われる。

5.2 matchAll・matchAllDFS による探索木のトラバース

本節では、matchAll式と matchAllDFS式を実行したときに内部でどのような探索木がトラバースされるのかを解説する。ある 1 つのマッチング・ステートを簡約すると 0 個から無限個までを含む複数のマッチング・ステートが生成される。そのため、Egison の内部のパターンマッチ・アルゴリズムは、初期マッチング・ステートを根とする木の探索と考えることができる。

まずは、以下のような matchAllDFS式の探索木をみる。

```
take 8 (matchAllDFS [1..] as set integer with
  | $m :: $n :: _ -> (m, n))
-- [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (1, 8)]
```

この matchAllDFS式の探索木は図 5.1 のようになる。図中の四角は、1 つのマッチング・ステートを表現している。図中の丸は、マッチング・アトムのスタックが空になったマッチング・ステート、パターンマッチの最終結果を表現している。matchAllDFS式はこの探索木を深さ優先探索する。

そのため、マッチング・ステートは、a1, b2, e3, f4, h5, j6, m7, o8, k7, p8, r9, ... という順番で簡約されていく。マッチング・ステート h5 は無限個のマッチング・ステート (j6, k7, ...) に簡約される。そのため、深さ優先探索では、マッチング・ステート c3 の簡約に行き着くこと

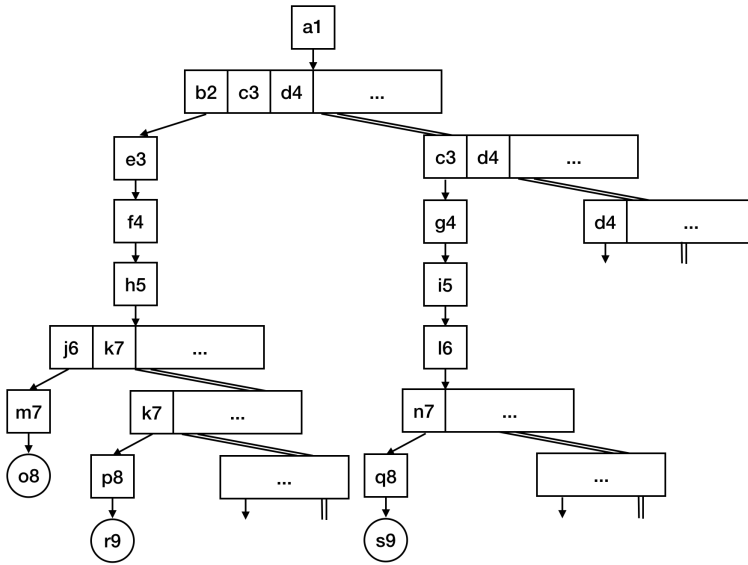


図 5.2: matchAll1式の探索木

がない。このように matchAll1DFSでは、すべてのマッチング・ステートをたどれず、可算無限のパターンマッチ結果をすべて列挙できない場合がある。

matchAll1式は、可算無限のパターンマッチ結果をすべて列挙するために、この探索木に変形を加えて幅優先探索をおこなう。図 5.2 は、下記の matchAll1式を実行したときの探索木である。

```
take 8 (matchAll1 [1..] as set integer with
  | $m :: $n :: _ -> (m, n))
-- [(1, 1), (1, 2), (2, 1), (1, 3), (2, 2), (3, 1), (1, 4), (2, 3)]
```

図 5.1 の探索木については、1つのマッチング・ステートが1つのノードをあらわしていたが、図 5.2 の探索木については、一連なりのマッチング・ステートのリストを1つのノードとする。たとえば、a1 は1つのマッチング・ステートからなるノード、b2, c3, d4, ... は無限個のマッチング・ステートからなる1つのノードである。このように探索木をとらえると、この探索木は二分木になっている。探索木が二分木であるため、すべてのノードの子が有限であるため、幅優先探索をすれば、すべてのノードをたどることができる。

図 5.1 の探索木を斜めにとらえることによって、図 5.2 の二分木への変形はできる。b2, c3, d4, ... からなるノードに注目しよう。このノードの子は、e3 単独からなるノードと、c3, d4, ... からなるノードである。一般に、あるノードの子は、そのノードの先頭のマッチング・ステートを簡約した結果と、そのノードの先頭の要素を取り除いたマッチング・ステートのリストとなる。

5.3 and パターン・or パターン・not パターンの実装

本節では、組み込みパターンの実装の例として、and パターン・or パターン・not パターン（第2章 2.6 節）の実装を説明する。

and パターンの実装からみていく。以下のような and パターンを含むパターンマッチを考える。

```
matchAll [1, 2, 3] as list integer with
| $n :: (_ :: _ & $rs) -> (n, rs)
-- [(1, [2, 3])]
```

このパターンマッチを処理する過程で、以下のようなマッチング・ステートの簡約にいきつく。

```
MState [( _ :: _ & $rs, [2, 3], list integer)]
      [(n, 1)] env
```

スタックの先頭のマッチング・アトムのパターンが and パターンであった場合、Egison 処理系は以下のように簡約する。

```
MState [( _ :: _, [2, 3], list integer)
      , ($rs, [2, 3], list integer)]
      [(n, 1)] env
```

and パターンのそれぞれの引数について、同じターゲットとマッチャーからつくったマッチング・アトムがスタックに追加される。

次に or パターンの実装をみる。以下のような or パターンを含むパターンマッチを考える。

```
matchAll [1, 1, 2] as list integer with
| $m :: ([ ] | #m :: _) -> m
-- [1]
```

このパターンマッチを処理する過程で、以下のようなマッチング・ステートの簡約にいきつく。

```
MState [([ ] | #m :: _), [1, 2], list integer)]
      [(m, 1)] env
```

スタックの先頭のマッチング・アトムのパターンが or パターンであった場合、Egison 処理系は以下のように簡約する。

```
MState [([ ], [1, 2], list integer)]
      [(m, 1)]

MState [(#m :: _, [1, 2], list integer)]
      [(m, 1)] env
```

or パターンのそれぞれの引数について、同じターゲットとマッチャーからつくったマッチング・アトムをスタックの先頭にもつマッチング・ステートを生成する。

最後に not パターンの実装をみる。以下のような not パターンを含むパターンマッチを考える。

```
matchAll [2, 8, 2] as multiset integer with
| $m :: (!(#m :: _) & $rs) -> (m, rs)
-- [(8, [2,2])]
```

このパターンマッチを処理する過程で、以下のようなマッチング・ステートの簡約にいきつく。

```
MState [(!(#m :: _), [2, 2], multiset integer)
, ($rs, [2, 2], multiset integer)]
[(m, 8)] env
```

スタックの先頭のマッチング・アトムのパターンが not パターンであった場合、Egison 処理系は以下のように、スタックの先頭のマッチング・アトムから not パターンの not をのぞいたマッチング・アトムだけをスタックにもつマッチング・ステートを生成する。

```
MState [(#m :: _), [2, 2], multiset integer)
[(m, 8)] env
```

このマッチング・ステートの簡約した結果、1 つもパターンマッチに成功するマッチング・ステートがなければ、以下のような先ほどのマッチング・ステートから not パターンをもつ先頭のマッチング・アトムをのぞいたマッチング・ステートを簡約する。

```
MState [($rs, [2, 2], multiset integer)]
[(m, 8)] env
```

5.4 パターン関数の実装

本節は、パターン関数の実装を解説する。以下のようなパターン関数の適用を含むパターンマッチを考える。

```
def twin := \p1 p2 => (~p1 & $x) :: #x :: ~p2

matchAll [1, 2, 1, 3] as multiset integer with
| $m :: twin $n _ -> (m, n)
-- [(2, 1), (2, 1), (3, 1), (3, 1)]
```

このパターンマッチを処理する過程で、以下のようなマッチング・ステートの簡約にいきつく。

```
MState [(twin $n _, [1, 1, 3], multiset integer)]
[(m, 2)]
```

このようにスタックの先頭のマッチング・アトムのパターンが、パターン関数の適用であった場合、パターン関数が展開される。この展開と同時に、MNode というほとんど MState と同じ構造のデータにマッチング・アトムは変換される。MNode は、パターンマッチの途中結果（下記の場合、

[]) と、パターン関数を定義したときの環境（下記の場合、env1）、パターン関数の引数に束縛されたパターンの環境（下記の場合、[(p1, \$n), (p2, _)]）をもつ。このように MState がネストしたような構造をつくるのは、パターンの静的スコープを実現するためである。

```
MState [MNode [(~p1 & $x) :: #x :: ~p2, [1, 1, 3], multiset integer]]
      []
      env1
      [(p1, $n), (p2, _)]
      [(m, 2)] env
```

このマッチング・ステートを簡約していくと、スタックの先頭のパターンがパターン関数の仮引数（下記の場合、p1）であるマッチング・ステートにいきつく。

```
MState [MNode [(p1, 1, integer),
               (#x :: ~p2, [1, 3], multiset integer)]
      [(x, 1)]
      env1
      [(p1, $n), (p2, _)]
      [(m, 2)] env
```

パターン関数の引数に束縛されたパターンの環境（MNode の 3 つ目の引数）から p1 に束縛されているパターンを取り出し、マッチング・アトムのパターンをそのパターンに展開する。同時に、このマッチング・アトムを MNode から MState のマッチング・アトムに持ち上げる。

```
MState [($n, 1, integer),
      MNode [(#x :: ~p2, [1, 3], multiset integer)]
      [(x, 1)]
      env1
      [(p1, $n), (p2, _)]
      [(m, 2)] env
```

第 6 章

マッチャーを定義しよう

本章は、ユーザーが自身でマッチャーを定義する方法を解説する。

6.1 inductive pattern 宣言

マッチャーを定義する際には、そのマッチャーが処理するパターンコンストラクタをあらかじめ *inductive pattern 宣言* (*inductive pattern declaration*) で宣言しておく必要がある。

inductive pattern 宣言は、パターンコンストラクタの型を、データコンストラクタやマッチャーの定義から独立して宣言するための構文である。たとえば、リスト型 `[a]` に対するパターンコンストラクタは以下のように宣言される。

```
1 inductive pattern [a] :=  
2   | []  
3   | (::) a [a]  
4   | (++) [a] [a]
```

この宣言は、パターンコンストラクタ `[]`, `::`, `++` がいずれもリスト型 `[a]` を対象とするパターンコンストラクタであることを宣言している。各パターンコンストラクタの後に並ぶ型は、そのパターンコンストラクタの引数の型、すなわちパターンマッチの成功時にネクスト・ターゲットとして渡される値の型を表す。

inductive pattern 宣言の重要な特徴は、`::` というパターンコンストラクタが `list` マッチャー、`multiset` マッチャー、`set` マッチャーなど、複数のマッチャーにわたって共有される点にある。それぞれのマッチャーは `::` に対して異なる分解方法を実装するが、そのパターンコンストラクタの型は inductive pattern 宣言によって一元的に管理される。

```
matchAll [1, 2, 3] as list integer with  
  | $x :: $xs -> (x, xs)  
-- [(1,[2,3])]  
  
matchAll [1, 2, 3] as multiset integer with
```



```
| $x :: $xs -> (x, xs)
-- [(1,[2,3]),(2,[1,3]),(3,[1,2])]
```

同じパターン `$x :: $xs` でも、使うマッチャーによって意味（分解方法）が変わる。

この inductive pattern 宣言とマッチャー定義の関係は、型クラスとインスタンス定義の関係に類似している。型クラス宣言がメソッドの型を宣言し、インスタンス定義がその具体的な実装を提供すると同様に、inductive pattern 宣言がパターンコンストラクタの型を宣言し、マッチャー定義がそのパターンコンストラクタに対する具体的な分解方法を定義する。

型クラス	パターンマッチング
型クラス宣言	inductive pattern 宣言
インスタンス定義	マッチャー定義
メソッド名	パターンコンストラクタ名
メソッドの型	パターンコンストラクタの型
インスタンスディスパッチ（静的）	マッチャーディスパッチ（動的）

ただし、型クラスのインスタンス選択がコンパイル時に静的に決定されるのとは異なり、マッチャーの選択は `match` 式や `matchAll` 式においてユーザーが実行時に明示的に指定する（動的選択）。

6.2 マッチャーの定義の基礎

本節は、非自由データ型のなかでもっとも単純な `unordered pair`（要素の順番を無視するペア）のマッチャー定義をみていくことにより、マッチャー定義の方法の概略を解説する。

まずは、整数の `unordred pair` に対するマッチャー `unorderedIntegerPair` マッチャーの挙動を確認しよう。`unorderedIntegerPair` に対して、`pair` パターンを使うと以下のように 2 通りの要素の順番を両方パターンマッチする。

```
matchAll (1, 2) as unorderedIntegerPair with
| pair $x $y -> (x, y)
-- [(1,2),(2,1)]
```

そのおかげで、`pair` の第一引数がターゲットの 2 番目の要素である場合にも、パターンマッチに成功する。

```
matchAll (1, 2) as unorderedIntegerPair with
| pair #2 $y -> y
-- [1]
```

パターンがパターン変数であった場合には、ターゲットをそのまま束縛する。

```
matchAll (1, 2) as unorderedIntegerPair with
| $p -> p
```

```
-- [(1,2)]
```

上記のような動作をする `unorderedIntegerPair` マッチャーは以下のように定義できる。まず、`pair` パターンコンストラクタを `inductive pattern` 宣言で宣言しておく必要がある。

```
1 inductive pattern (Integer, Integer) :=
2   | pair Integer Integer
```

この宣言は、`pair` が整数のペア型 (`Integer, Integer`) を対象とするパターンコンストラクタで、`Integer` 型の引数を 2 つ取ることを宣言している。次に、型注釈により整数のペアに対するマッチャーであることを示しながら、`unorderedIntegerPair` マッチャーを以下のように定義する。

```
1 def unorderedIntegerPair : Matcher (Integer, Integer) := matcher
2   | pair $ $ as (integer, integer) with
3     | ($x, $y) -> [(x, y), (y, x)]
4   | $ as something with
5     | $tgt -> [tgt]
```

`matcher` は、マッチャーを生成するための組み込み構文である。

```
matcher
| 原始パターンパターン as ネクスト・マッチャー式 with
| 原始データパターン -> ボディ
...
...
```

`matcher` 式は、複数のマッチャー節 (*matcher clause*) をとる。マッチャー節は、**原始パターンパターン** (*primitive pattern-pattern*)、**ネクスト・マッチャー式** (*next matcher expression*)、複数の**原始マッチ節** (*primitive match clause*) からなる。原始マッチ節は、**原始データパターン** (*primitive data pattern*) とボディからなる。

`unorderedIntegerPair` の 1 つ目のマッチャー節 (2-3 行目) をみていく。まず、このマッチャー節の**原始パターンパターン**は `pair $ $` である。**原始パターンパターン** はパターンに対するパターンである。この**原始パターンパターン** は `pair` パターンにマッチし、このマッチャー節は `pair` パターンに対するパターンマッチの方法を定義している。`pair` のように、**原始パターンパターン** 中にあらわれるパターンコンストラクタは、**原始パターンパターンコンストラクタ** (*primitive pattern-pattern constructor*) と呼ばれる。`pair` パターンの引数としてあらわれている `$` は **パターン・ホール** (*pattern hole*) と呼ばれる**原始パターンパターン**の構成要素で、任意のパターンがマッチする。**パターン・ホール** にマッチしたパターンは、**ネクスト・パターン** (*next pattern*) と呼ばれる。**原始パターンパターン** のパターンマッチは、一般の関数型プログラミング言語の代数的データ型に対するパターンマッチとほぼ同じようにおこなわれる。次に、このマッチャー節の**ネクスト・マッチャー式**は、(`integer, integer`) である。これは、上記の**パターン・ホール** に束縛された 2 つのパターン (`pair` パターンの引数) がそれぞれ `integer` マッチャーを使って再帰的に

パターンマッチされることを表現している。このマッチャー節は、1つの原始マッチ節（3行目）をとる。原始データパターンは、ターゲットとパターンマッチされる。原始データパターンのパターンマッチも、一般の関数型プログラミング言語の代数的データ型に対するパターンマッチとほぼ同じようにおこなわれる。原始マッチ節のボディは、ターゲットの分解結果を返す。 (x, y) と (y, x) はネクスト・ターゲット (*next target*) と呼ばれ、ネクスト・パターンとネクスト・マッチャーを使って再帰的にパターンマッチされる。

2つ目のマッチャー節（4-5行目）の原始パターンパターンは、パターン・ホールである。パターン・ホールは任意のパターンにマッチするため、1つ目のマッチャー節でマッチしなかったパターンはすべて2つ目のマッチ節で処理される。1つ目のマッチャー節でマッチしない `unorderedIntegerPair` に対するパターンは、ワイルドカードかパターン変数のみであるので、このマッチャー節はこれらのパターンを処理する。このマッチャー節は、パターンとターゲットを変えず、ただマッチャーを `something` に変換するだけである。パターンとターゲットは `something` を使ってパターンマッチされる。`something` は唯一の組み込みマッチャーであり、以下の3種類のパターンを処理する。

- **ワイルドカード**：つねにパターンマッチに成功し、束縛は生じない。
- **パターン変数 $\$x$** ：パターンマッチに成功し、ターゲットの値を x に束縛する。
- **値パターン $\#e$** ：式 e を評価した値とターゲットの値を **組み込み構造的同値判定**（データコンストラクタの構造による等価比較）で比較し、等しければパターンマッチに成功する。

値パターンを `something` が直接処理するようになったのは、静的型付けの導入にともなう設計変更である（詳しくは後述の 6.3 節を参照）。

以下の `unorderedPair` のように、ペアの要素のデータ型に対するマッチャーを受け取る `unordered pair` に対するマッチャーを定義することができる。

```
matchAll (1, 2) as unorderedPair integer with
| pair $x $y -> (x, y)
-- [(1,2),(2,1)]
```

そのためには、まず `inductive pattern` 宣言を型パラメータを使った汎用的な形に変更する。

```
inductive pattern {a} (a, a) :=
| pair a a
```

そして、`unorderedPair` をマッチャーを引数にとってマッチャーを返す関数として定義すればよい。`pair` パターンに対するマッチャー節のネクスト・マッチャー式で `unorderedPair` の引数である m を指定している。型注釈により、型パラメータ `{a}` と引数のマッチャー (m : `Matcher a`) を受け取り、ペア型 `(a, a)` に対するマッチャーを返すことを示している。

```
def unorderedPair {a} (m: Matcher a) : Matcher (a, a) := matcher
| pair $ $ as (m, m) with
| ($x, $y) -> [(x, y), (y, x)]
```

```
| $ as something with
| $tgt -> [tgt]
```

6.3 eq マッチャーの定義

eq マッチャーも `matcher` 式でユーザーが定義することができる。eq マッチャーは値パターンとワイルドカード・パターン変数のみを処理するため、特別なパターンコンストラクタを持たない。したがって、eq マッチャー自体には `inductive pattern` 宣言は不要である。型注釈により、型制約 `{Eq a}` を持つ型パラメータ `a` に対するマッチャーであることを示している。

```
1 def eq {Eq a} : Matcher a := matcher
2   | # $val as () with
3     | $tgt -> if val == tgt then [()] else []
4   | $ as something with
5     | $tgt -> [tgt]
```

eq マッチャーの 1 つ目のマッチャー節 (2-3 行目) は値パターンに対するマッチャー節である。 `# $val` は値パターンにマッチする原始パターンパターンである。このような原始パターンパターンは原始値パターン (*primitive value pattern*) と呼ばれる。変数 `val` には値パターンの中身が束縛される。このマッチャー節の原始パターンパターンは、パターン・ホールを含んでいないため、ネクスト・マッチャー式は空タプルである。原始マッチ節で、値パターンの中身とターゲットの値が等しいかどうかチェックしている。

■ **something と eq の違い。** `something` は値パターンを組み込みの構造的等値判定で処理するのに対し、`eq` は `Eq` 型クラスのメソッドであるユーザ定義の `==` 演算子を使って比較する。この 2 つの使い分けは、マッチャーを引数にとるマッチャー (マッチャーパラメータ化マッチャー) の設計と深く関わっている。

たとえば、`multiset` のような引数マッチャー `m` を受け取るマッチャーで値パターンを処理するとき、再帰的な比較の基底ケースをどのマッチャーに委ねるかが問題になる。もし `something` が値パターンを処理できないとすると、`eq` マッチャーに委ねるしかなく、`eq` は `Eq` 型クラス制約を必要とするため、`multiset` の型が `multiset : {Eq a} => Matcher a -> Matcher [a]` のように `Eq` 制約を必要とするものになってしまう。しかし `multiset` の多くの用途は値パターンを使わないため、この制約は過度に制限的である。

この問題を解決するために、`something` に組み込みの構造的等値判定による値パターン処理を組み込んだ。これにより、マッチャーパラメータ化マッチャーは、型クラス制約を課すことなく `something` を基底ケースとして使え、`multiset : Matcher a -> Matcher [a]` という制約のない型を持つことができる。一方、ユーザ定義の等値比較 (`==`) を使いたい場合には、明示的に `eq` マッチャーを指定することで対応できる。

原始パターンパターンは、ここまでに出てきた 3 つの構成要素であるパターン・ホール、原始

パターンパターンコンストラクタの適用, 原始値パターンと, 原始ワイルドカードからなる.

```
原始パターンパターン ::= $ | c 原始パターンパターン* | # $ID | _
```

原始ワイルドカードについては, 6.6 節で用例を紹介する.

原始データパターンは, ワイルドカード, パターン変数, 原始データパターンコンストラクタの適用からなる.

```
原始データパターン ::= _ | $ID | C 原始データパターン*
```

6.4 multiset マッチャーの定義

本節は, multiset マッチャーの定義を解説する. multiset マッチャーは, 6.1 節で示したリスト型 [a] の inductive pattern 宣言で宣言されたパターンコンストラクタ ([], ::, ++) を処理するマッチャーである. 型注釈により, 型パラメータ {a} と引数のマッチャー (m: Matcher a) を受け取り, リスト型 [a] に対するマッチャーを返すことを示している.

```
1 def multiset {a} (m: Matcher a) : Matcher [a] := matcher
2   | [] as () with
3     | $tgt -> match tgt as (mutiset m) with
4       | [] -> [()]
5       | _ -> []
6   | $ :: $ as (m, multiset m) with
7     | $tgt -> matchAll tgt as list m with
8       | $hs ++ $x :: $ts -> (x, hs ++ ts)
9   | # $val as () with
10    | $tgt -> match (val, tgt) as (list m, multiset m) with
11      | ([], []) -> [()]
12      | ($x :: $xs, #x :: #xs) -> [()]
13      | (_, _) -> []
14   | $ as something with
15     | $tgt -> [tgt]
```

1 目と 4 目のマッチャー節はほぼ自明であるので, 2 目と 3 目のマッチャー節をみていく.

2 目のマッチャー節 (6-8 行目) をみていこう. 原始プリミティブパターンは $\$:: \$$ で, ネクスト・マッチャー式は $(a, \text{multiset } a)$ である. a は multiset の引数のマッチャーである. ネクスト・ターゲットは, matchAll 式を使って定義されている. この matchAll 式は, ターゲットが $[1, 2, 3]$ である場合, $[(1, [2, 3]), (2, [1, 3]), (3, [1, 2])]$ を返す. このリストに含まれるそれぞれのネクスト・ターゲットは, ネクスト・パターンとネクスト・マッチャーを使って再帰的にパターンマッチされる. たとえば, ネクスト・ターゲット 1 と $[2, 3]$ は, ネクスト・マッチャー a と multiset a を使って, コンス・パターンの第 1 引数と第 2 引数のパターンとパターンマッチされる.

3 目のマッチ節をみていこう. このマッチャー節の原始パターンパターンは, 原始値パター

ン#\$valである。このマッチャー節は、値パターンを処理する。このマッチャー節は、値パターンの中身 (val) とターゲット (tgt) が等しいかどうかくらべる。この match 式の 1 つ目と 3 つ目のマッチ節は自明であるので説明を省略する。2 つ目のマッチ節のパターンは、val の先頭の要素を tgt から取り出す。そして、val と tgt から同じ要素を 1 つ抜いたコレクションを \$xs と #xs というパターンを使って再帰的にパターンマッチしている。#xs のパターンマッチに、このマッチャー節自身が再帰的に呼び出されるが、コレクションの長さが 1 つずつ短くなっていくため、最終的に 1 つ目か 3 つ目のマッチ節どちらかにいきつく。

6.5 sortedList マッチャーの定義

二重にネストしたジョイン・コンス・パターンを使って $(p, p + 6)$ という形の素数のペアを列挙するプログラムは、後方の素数のペアになるほど列挙に時間がかかるようになる。その理由は、二重にネストしたジョイン・コンス・パターンは、すべての素数の組み合わせを検査するためである。たとえば、このパターンは、 $(3, 5)$, $(3, 7)$, $(3, 11)$, $(3, 13)$, $(3, 17)$, $(3, 19)$ のような素数のペアすべてを検査する。しかし、 $(3, 11)$ 以降の素数のペア、差が 6 より大きくなる最初の素数のペア、以降のペアについては、 $(p, p + 6)$ であるか検査する必要はない。

```
take 10 (matchAll primes as sortedList integer with
  | _ ++ $p :: (_ ++ #(p + 6) :: _) -> (p, p + 6))
-- [(5,11),(7,13),(11,17),(13,19),(17,23),(23,29),(31,37),(37,43),(41,47),(47,53)]
```

ソート済みリストに特化したマッチャーを使うことにより、この不必要な探索は避けることができる。通常の list マッチャーに、 $\$ ++ \#\$px : \$$ を原始パターンパターンにもつマッチャー節を追加すれば、ソート済みリストに特化したマッチャーをつくることができる。このマッチャー節が、 $(p, p + 6)$ という形の素数のペアにマッチする二重にネストしたジョイン・コンス・パターンの計算量を $O(n^2)$ から $O(n)$ に減らす。型注釈により、型制約 {Ord a} を持つ型パラメータ a と マッチャー引数 (m: Matcher a) を受け取り、ソート済みリスト型 [a] に対するマッチャーを返すことを示している。

```
def sortedList {Ord a} (m: Matcher a) : Matcher [a] := matcher
  | $ ++ #\$px :: $ as (sortedList m, sortedList m) with
    | \tgt -> matchAll tgt as list m with
      | loop $i (1, $n)
        ((?(\x -> x < px) & $h_i) :: ...)
        (#px :: $ts)
        -> (map (\i -> h_i) [1..n], ts)
    ...
```

このようにネストしたパターンに対して直接パターンマッチのアルゴリズムを定義することによる最適化のことを、**パターン・フュージョン** (*pattern fusion*) と呼ぶ。

6.6 原始ワイルドカードによる最適化

パターンがワイルドカードを含む場合、ワイルドカードにマッチするターゲットの計算を省くことにより、パターンマッチの処理を高速化できる。原始パターンパターン中で、原始ワイルドカードを使うことにより、Egison ではこの最適化をおこなえる。本節は、6.4 節で紹介した multiset マッチャーについて、そのような最適化を紹介する。このような最適化のことを、**ワイルドカード最適化** (*wildcard optimization*) と呼ぶ。

本節で紹介する最適化の対象は、以下のようなコンス・パターンの第二引数がワイルドカードである場合である。この場合、対象のコレクションから要素を 1 つ除いた残りのコレクションを計算する必要はない。原始ワイルドカードを使うことにより、この計算を省くように multiset マッチャーに記述することができる。

```
1 matchAll [1, 2, 3, 4, 5] as multiset eq with
2 | $x :: _ -> x
3 -- [1, 2, 3, 4, 5]
```

このような最適化は、以下の 2-3 行目のマッチャー節を 6.4 節の multiset マッチャーの定義に追加すればできる。このマッチャー節の原始パターンパターンで原始ワイルドカードが使われている。パターン・ホールはコンス・パターンの第一引数だけであるので、ネクスト・マッチャーは m だけになり、ネクスト・ターゲットは tgt のそれぞれの要素であるために、tgt をそのまま返すようになっている。

```
1 def multiset {a} (m: Matcher a) : Matcher [a] := matcher
2 | $ :: _ as m with
3   | $tgt -> tgt
4 | $ :: $ as (m, multiset m) with
5   | $tgt -> matchAll tgt as list m with
6     | $hs ++ $x :: $ts -> (x, hs ++ ts)
7   ...
```

6.5 節の sortedList マッチャーも同様の最適化ができる。下記の 2-7 行目のマッチャー節は、ジョイン・パターンの第一引数を計算することを省略する役割を果たす。

```
1 def sortedList {Ord a} (m: Matcher a) : Matcher [a] := matcher
2 | _ ++ #px :: $ as sortedList m
3   | \tgt -> matchAll tgt as list m with
4     | loop $i (1, _)
5       (?(\x -> x < px) :: ...)
6       (#px :: $ts)
7     -> ts
8 | $ ++ #px :: $ as (sortedList m, sortedList m)
9 | \tgt -> matchAll tgt as list m with
```

```

10         | loop $i (1, $n)
11           ((?(\x -> x < px) & $h_i) :: ...)
12           (#px :: $ts)
13           -> (map (\i -> h_i) [1..n], ts)
14     ...

```

6.7 assocMultiset マッチャー

Egison には、多重集合に対するマッチャーとして、`assocMultiset`が用意されている。本節はその使い方と定義を紹介する。

多重集合は、要素のその個数の連想リストとして表現できる。`assocMultiset`マッチャーは、このように連想リストとして表現された多重集合に対するマッチャーである。たとえば、以下のプログラムは 1 を 4 個、2 を 3 個、3 を 1 個もつ多重集合に対してパターンマッチをしている。ターゲットの多重集合に 3 個以上現れる要素を取り出すパターンが表現されている。

```

1 matchAll [(1, 4), (2, 3), (3, 1)] as assocMultiset eq with
2   | $x ^ #3 :: $rs -> (x, rs)
3 -- [(1, [(1, 1), (2, 3), (3, 2)]), (2, [(1, 4), (3, 2)])]

```


第 7 章

関数型プログラミング言語としてのいくつかの機能

本章は、関数型のプログラムの記述を便利にするための Egison の機能をいくつか紹介する。

7.1 無名パラメーター関数

無名パラメーター関数 (anonymous parameter function) を使うと引数に名前をつけずに関数を定義できる。ラムダ式を使うと関数の命名を省略できる。引数の命名まで省略できる無名パラメーター関数は、これをさらにおしすすめたものと考えることができる。たとえば、第一引数を 10 倍して第二引数と足し合わせる無名パラメーター関数は以下のように定義できる。

```
2#(%1 * 10 + %2) 1 2
-- 12
```

#の前の 2 は 2 引数の関数を定義していることを意味する。関数のボディの中で現れる %1 と %2 はそれぞれ第一引数と第二引数を表している。

無名パラメータ関数には、引数にタプルをとるものとリストをとるものの 2 つの変種がある。この 2 つの変種は #の前に指定する引数の数を括弧で囲むことによって定義する。#の前の数を () で囲むとタプルを引数に取る無名パラメーター関数を定義できる。

```
(2)#(%1, %2) (10, 20)
-- (10, 20)
```

#の前の数を [] で囲むとリストを引数に取る無名パラメーター関数を定義できる。

```
[2]#(%2, %1) [10, 20]
-- (20, 10)
```

7.2 無名 matchAll 関数・無名 match 関数

関数定義のとき、仮引数をすぐにパターンマッチすることが多い。するとその仮引数の名前がそのパターンマッチのターゲットとしてしか役に立たない。この問題を解決するために無名 matchAll 関数と無名 match 関数がある。

まず、無名 matchAll 関数を紹介する。\`\`にスペースを入れずに matchAll を続けることで無名 matchAll 関数は定義される。このときパターンマッチのターゲットは省略される。この無名 matchAll 関数の引数がターゲットとなるからである。

```
\matchAll as list something with
| $hs ++ $ts -> (hs, ts)
```

無名 match 関数は、無名 matchAll 関数の matchAll を match に変えるだけで定義できる。

```
\match as list something with
| nil -> True
| _ -> False
```

7.3 中置演算子の定義

Egison はユーザーが新しく中置演算子を定義する機能を提供している。中置演算子を宣言するには、infix、または infixl、infixr を使う。infix、infixl、infixr はそれぞれ結合性のない演算子、左結合の演算子、右結合の演算子を定義するのに使う。infix、infixl、infixr は共通して三つの引数をとる。第一引数には、これから定義する中置演算子が関数であるのか、パターンコンストラクタであるのか指定するために、expression か pattern というキーワードをとる。第二引数には、これから定義する中置演算子の優先度を整数値でとる。第三引数には、これから定義する中置演算子の名前をとる。

中置演算子として使う関数を定義する例として論理積 `&&` を定義すると以下ようになる。型注釈により、2 つのブール値を引数にとりブール値を返すことを示している。

```
infixr expression 5 &&

def (&&) (a: Bool) (b: Bool) : Bool := match (a, b) as (eq, eq) with
| (#True, #True) -> True
| _ -> False
```

中置演算子として使うパターンコンストラクタを定義する例としてジョイン・パターン `++` を定義すると以下ようになる。型注釈により、型パラメータ `{a}` とマッチャー引数 `(m: Matcher a)` を受け取り、リスト型 `[a]` に対するマッチャーを返すことを示している。

```
infixl pattern 7 ++
```

```
def list {a} (m: Matcher a) : Matcher [a] := matcher
  | $ ++ $ as (list m, list m) with
    ...
```

演算子	演算子の内容	優先度
^	べき乗	8
*	掛け算	7
/	割り算	7
%	割り算の余り	7
.	テンソルの掛け算	7
+	足し算	6
-	引き算	6
::	コンス	5
++	アペンド	5
=, <= >=, <, >	比較	4
&&	論理積	3
	論理和	2
\$	関数適用	0

表 7.1: 中置演算子一覧（関数）

演算子	演算子の内容	優先度
^	べき乗	8
*	掛け算	7
/	割り算	7
+	足し算	6
::	コンス	5
++	アペンド	5
&	and パターン	3
	or パターン	2

表 7.2: 中置演算子一覧（パターン）

7.4 型クラス

Egison は型クラス (type class) をサポートしている。型クラスは、複数の型に共通する振る舞いを抽象化するための機構である。Egison の型クラスは Haskell の型クラスに似た構文と意味論をもつ。

7.4.1 型クラスの定義

型クラスは `class` キーワードを使って定義する。型クラスの定義には、型変数と、その型変数に対して要求される関数のシグネチャを含める。たとえば、等値性を比較できる型を表す `Eq` 型クラスは以下のように定義される。

```
class Eq a where
  (==) (x: a) (y: a) : Bool
  (/=) (x: a) (y: a) : Bool
```

この定義は、「型 `a` が `Eq` のインスタンスであるためには、2 つの `a` 型の値を受け取り `Bool` を返す `==` と `/=` という演算を提供しなければならない」ことを意味する。

もう一つの例として、数値型を表す `Num` 型クラスは以下のように定義される。

```
class Num a where
  (+) (x: a) (y: a) : a
  (-) (x: a) (y: a) : a
  (*) (x: a) (y: a) : a
  (/) (x: a) (y: a) : a
```

7.4.2 型クラスのインスタンス

具体的な型を型クラスのインスタンスにするには `instance` キーワードを使う。たとえば、`Integer` 型を `Eq` のインスタンスにするには以下のように書く。

```
instance Eq Integer where
  (==) x y := x == y
  (/=) x y := not (x == y)
```

`MathExpr` 型 (数式を表す型) を `Num` のインスタンスにする例は以下のとおりである。

```
instance Num MathExpr where
  (+) x y := plusForMathExpr x y
  (-) x y := minusForMathExpr x y
  (*) x y := multForMathExpr x y
  (/) x y := divForMathExpr x y
```

型クラスのインスタンス定義には型クラス制約を含めることもできる。たとえば、Tensor a型をEqのインスタンスにするとき、要素型aもEqのインスタンスである必要がある場合は以下のよう
に書く。

```
instance {Eq a} Eq (Tensor a) where
  (==) t1 t2 := t1 == t2
  (/=) t1 t2 := not (t1 == t2)
```

7.4.3 型クラス制約の使い方

型クラスを使うことで、多相的な関数に制約を課することができる。型クラス制約は波括弧{}で囲んで関数の型シグネチャに記述する。たとえば、2つの値の等値性を比較する関数は以下のように定義できる。

```
def eqAs {Eq a} (m: Matcher a) (x: a) (y: a) : Bool := ...
```

この型シグネチャは「型aがEqのインスタンスであるとき、eqAsはMatcher a, a, aを受け取りBoolを返す」ことを意味する。

複数の型クラス制約を課することもできる。たとえば、順序付けと等値性の両方を必要とする関数は以下のように書ける。

```
def compare {Eq a, Ord a} (x: a) (y: a) : Ordering := ...
```

型クラス制約は、関数だけでなくマッチャーの定義にも使える。たとえば、sortedListマッチャーは、要素型が順序付け可能(Ordのインスタンス)であることを要求する。

```
def sortedList {Ord a} (m: Matcher a) : Matcher [a] := matcher
...
```

7.4.4 型クラスの実装：辞書渡しスタイル

Egisonの型クラスは**辞書渡しスタイル**(dictionary-passing style)で実装されている。これは、型クラス制約をもつ関数が、実際にはその型クラスのメソッドを含む辞書を引数として受け取る関数に変換されることを意味する。

たとえば、以下のような型クラス制約をもつ関数は

```
def double {Num a} (x: a) : a := x + x
```

内部的には以下のように辞書パラメータを受け取る関数に変換される。

```
def double (dict_Num_a) (x: a) : a :=
  (dict_Num_a."plus") x x
```

ここで `dict_Num_a` は `Num a` インスタンスのメソッド (`+`, `-`, `*`, `/`) を含むハッシュ (辞書) である。

具体的な型に対して関数を呼び出す場合、適切なインスタンスの辞書が自動的に渡される。たとえば、`double 1` は以下のように変換される。

```
double numInteger 1
```

ここで `numInteger` は `Num Integer` インスタンスの辞書である。

この辞書渡しスタイルにより、型クラスの動的ディスパッチ (実行時の型に応じたメソッドの選択) が実現される。また、型クラス制約をもつ関数同士の呼び出しも正しく処理される。たとえば、以下のような関数では

```
def myPlus {Num a} (x: a) (y: a) : a := x + y
def myPlus2 {Num a} (x: a) (y: a) : a := myPlus x y
```

`myPlus2` が受け取った辞書が `myPlus` に正しく渡される。

7.5 IO 入出力

遅延評価を基本の評価戦略とする Egison は、副作用をもつプログラムを記述するための特別な構文をもつ。Egison は静的型システムをもつ言語であり、Haskell と同じような方法で副作用をもつプログラムを記述する。

7.5.1 main 関数

たとえば、"Hello world!" という文字列を出力するプログラムは以下のように書ける。型注釈により、コマンドライン引数の文字列リストを受け取り、IO 処理を実行することを示している。

```
def main (args: [String]) : IO () := write "Hello world!\n"
```

コマンドラインオプション `-t` がない場合は、Egison は `main` 関数を実行する。上記のプログラムは以下のように実行できる。

```
$ cat hello.egi
def main (args: [String]) : IO () := write "Hello world!\n"
$ egison hello.egi
Hello world!
```

`write` は Egison に組み込みで実装されている IO 関数である。`write` は第一引数に文字列をとり、その文字列を標準出力に印字する。IO 関数には `write` 以外にもいくつかもある。図 7.3 にその一覧をまとめた。

`main` の第一引数にはコマンドライン引数がある。コマンドライン引数は文字列として `main` の第一引数に渡される。

```
$ cat args.egi
def main (args: [String]) : IO () := write (show args)
$ egison args.egi hello world 1
["hello", "world", "1"]
```

Egison でつくったスクリプトをコマンドにしたい場合はシェバン (shebang) を使えばよい.

```
$ cat args
#!/usr/local/egison

def main (args: [String]) : IO () := write (show args)
$ args hello world 1
["hello", "world", "1"]
```

7.5.2 do 式

do式を使うと、複数の IO 関数を一つにまとめ、順番に実行することができる. do式は Haskell の do記法に対応している. たとえば、以下のプログラムはユーザーの入力を一行読み取り、それを出力する.

```
def main (arg: [String]) : IO () := do
  let line := readLine ()
  write line
```

do式と再帰関数を組み合わせることができる. 以下はインタプリタの REPL (Read-Eval-Print Loop) の Eval をぬいたプログラムである.

```
def main (arg: [String]) : IO () := repl

def repl : IO () := do
  write "> "
  flush ()
  let line := readLine ()
  write line
  flush ()
  repl
```

Haskell と同様に、オフサイドルールを使わない do記法の構文もサポートしている.

```
do { print "foo" ; print "bar" ; print "baz" }
```

なお、Egison の do式は IO モナドに対してのみ使用可能であり、Haskell のようにリストモナドやその他のモナドに対して使うことはできない.

7.5.3 io 関数

io関数は Haskell の `unsafePerformanceIO`関数に対応する組み込み関数である。io関数を使うと IO 関数でない通常の関数のなかで IO 関数を呼び出せるようになる。これは `printf` デバッグをしたりするのに便利である。

以下のプログラムは二つのサイコロをふりそれらの目の合計値を返す。

```
def dice : Integer := io (rand 1 6)

dice + dice
```

7.6 組み込み関数

Egison には多数の組み込み関数 (primitive functions) が用意されている。これらの関数は言語処理系に組み込まれており、ユーザーが直接利用できる。本節では主要な組み込み関数をカテゴリ別に紹介する。

7.6.1 算術演算関数

整数演算と浮動小数点演算のための組み込み関数が提供されている。表 7.4 に主要な算術演算関数を示す。

また、浮動小数点数に対する数学関数も提供されている (表 7.5)。

数学定数として、円周率 `f.pi` ($\pi \approx 3.14159$) と自然対数の底 `f.e` ($e \approx 2.71828$) が定義されている。

7.6.2 文字列操作関数

文字列を操作するための組み込み関数が提供されている (表 7.6)。

7.6.3 型変換・型チェック関数

型変換と型チェックのための組み込み関数が提供されている (表 7.7)。

注: Egison では `MathExpr`, `Integer`, `Rational` が同一視されるため, `isInteger` と `isRational` は数式処理システムで数値の性質を判定するために使われる。

7.6.4 テンソル関連関数

テンソルの情報を取得するための組み込み関数が提供されている (表 7.8)。

7.6.5 デバッグ・テスト関数

プログラムのデバッグとテストのための組み込み関数が提供されている (表 7.9).

7.7 seq 式

seq式は、プログラムを部分的に遅延評価ではなく正格評価したいときに使う。遅延評価は便利であるが、ときに、メモリを無駄に確保し効率が悪いことがある。部分的にプログラムを正格評価することで、プログラムの効率が向上することがある。seq式は第一引数のプログラムを正格評価したあと、第二引数のプログラムを評価する。たとえば、foldlは以下のように seq式を使うと効率が向上することが知られている。型注釈により、畳み込み関数、初期値、リストを受け取り、累積結果を返すことを示している。

```
def foldl {a, b} (fn: b -> a -> b) (init: b) (ls: [a]) : b :=
  match ls as list something with
  | [] -> init
  | $x :: $xs ->
    let z := fn init x
    in seq z (foldl fn z xs)
```

関数名	関数の内容
<code>return x</code>	x を IO 関数の結果として返す.
<code>openInputFile $path$</code>	パス $path$ のファイルを読み込みモードで開き, そのファイルへの入力ポートを返す.
<code>openOutputFile $path$</code>	パス $path$ のファイルを書き込みモードで開き, そのファイルへの出力ポートを返す.
<code>closeInputPort $port$</code>	入力ポート $port$ を閉じる.
<code>closeOutputPort $port$</code>	出力ポート $port$ を閉じる.
<code>readChar</code>	標準入力から一文字読み取り, その文字を返す.
<code>readLine</code>	標準入力から一行読み取り, その文字列を返す.
<code>writeChar c</code>	標準出力に文字 c を書き込む.
<code>write s</code>	標準出力に文字列 s を書き込む.
<code>readCharFromPort $port$</code>	入力ポート $port$ から一文字読み取り, その文字を返す.
<code>readLineFromPort $port$</code>	入力ポート $port$ から一行読み取り, その文字列を返す.
<code>writeCharToPort c $port$</code>	文字 c を出力ポート $port$ に書き込む.
<code>writeToPort s $port$</code>	文字列 s を出力ポート $port$ に書き込む.
<code>isEof</code>	標準入力 EOF (ファイルの終端) に達しているか調べ, 達していれば <code>True</code> , そうでなければ <code>False</code> を返す.
<code>isEofPort $port$</code>	入力ポート $port$ が EOF に達しているか調べ, 達していれば <code>True</code> , そうでなければ <code>False</code> を返す.
<code>flush</code>	標準出力のバッファに溜まっている内容を即座に出力する.
<code>flushPort $port$</code>	出力ポート $port$ のバッファに溜まっている内容を即座に出力する.
<code>readFile $path$</code>	パス $path$ のファイルの全内容を文字列として読み込んで返す.
<code>rand n_1 n_2</code>	整数 n_1 から n_2 までの範囲の整数をランダムに返す (両端を含む).
<code>f.rand f_1 f_2</code>	浮動小数点数 f_1 から f_2 までの範囲の浮動小数点数をランダムに返す.
<code>newIORef</code>	変更可能な変数 (参照) を生成してその参照を返す. 初期値は未定義.
<code>writeIORef ref x</code>	参照 ref が参照する変更可能な変数の値を x に更新する.
<code>readIORef ref</code>	参照 ref が参照する変更可能な変数の現在の値を返す.
<code>readProcess cmd $args$ $input$</code>	コマンド cmd を引数リスト $args$ で実行し, 標準入力に文字列 $input$ を与え, その標準出力を文字列として返す.
<code>io $ioAction$</code>	IO 関数 $ioAction$ を純粋な関数の中で実行し, その結果を返す (<code>unsafePerformIO</code> に相当).

表 7.3: IO 関数一覧

関数名	関数の内容
<code>i.+ x y</code>	整数 x と y の和を返す.
<code>i.- x y</code>	整数 x と y の差を返す.
<code>i.* x y</code>	整数 x と y の積を返す.
<code>i./ x y</code>	整数 x を y で割った商を有理数として返す.
<code>i.modulo x y</code>	整数 x を y で割った剰余を返す (mod).
<code>i.quotient x y</code>	整数 x を y で割った商を返す (quot).
<code>i.% x y</code>	整数 x を y で割った剰余を返す (rem).
<code>i.power x y</code>	整数 x の y 乗を返す.
<code>i.abs x</code>	整数 x の絶対値を返す.
<code>i.neg x</code>	整数 x の符号を反転した値を返す.
<code>i.< x y</code>	整数 x が y より小さければ True, そうでなければ False を返す.
<code>i.<= x y</code>	整数 x が y 以下であれば True, そうでなければ False を返す.
<code>i.> x y</code>	整数 x が y より大きければ True, そうでなければ False を返す.
<code>i.>= x y</code>	整数 x が y 以上であれば True, そうでなければ False を返す.
<code>f.+ x y</code>	浮動小数点数 x と y の和を返す.
<code>f.- x y</code>	浮動小数点数 x と y の差を返す.
<code>f.* x y</code>	浮動小数点数 x と y の積を返す.
<code>f./ x y</code>	浮動小数点数 x を y で割った商を返す.
<code>f.abs x</code>	浮動小数点数 x の絶対値を返す.
<code>f.neg x</code>	浮動小数点数 x の符号を反転した値を返す.
<code>f.< x y</code>	浮動小数点数 x が y より小さければ True, そうでなければ False を返す.
<code>f.<= x y</code>	浮動小数点数 x が y 以下であれば True, そうでなければ False を返す.
<code>f.> x y</code>	浮動小数点数 x が y より大きければ True, そうでなければ False を返す.
<code>f.>= x y</code>	浮動小数点数 x が y 以上であれば True, そうでなければ False を返す.
<code>round x</code>	浮動小数点数 x を最も近い整数に丸める.
<code>floor x</code>	浮動小数点数 x 以下の最大の整数を返す (床関数).
<code>ceiling x</code>	浮動小数点数 x 以上の最小の整数を返す (天井関数).
<code>truncate x</code>	浮動小数点数 x の小数部分を切り捨てた整数を返す.

関数名	関数の内容
<code>f.sqrt x</code>	浮動小数点数 x の平方根を返す.
<code>f.exp x</code>	自然対数の底 e の x 乗を返す.
<code>f.log x</code>	浮動小数点数 x の自然対数を返す.
<code>f.sin x</code>	x のサイン (正弦) を返す.
<code>f.cos x</code>	x のコサイン (余弦) を返す.
<code>f.tan x</code>	x のタンジェント (正接) を返す.
<code>f.asin x</code>	x のアークサイン (逆正弦) を返す.
<code>f.acos x</code>	x のアークコサイン (逆余弦) を返す.
<code>f.atan x</code>	x のアークタンジェント (逆正接) を返す.
<code>f.sinh x</code>	x の双曲線サイン (双曲正弦) を返す.
<code>f.cosh x</code>	x の双曲線コサイン (双曲余弦) を返す.
<code>f.tanh x</code>	x の双曲線タンジェント (双曲正接) を返す.
<code>f.asinh x</code>	x の逆双曲線サイン (逆双曲正弦) を返す.
<code>f.acosh x</code>	x の逆双曲線コサイン (逆双曲余弦) を返す.
<code>f.atanh x</code>	x の逆双曲線タンジェント (逆双曲正接) を返す.

表 7.5: 数学関数一覧

関数名	関数の内容
<code>pack cs</code>	文字のリスト <i>cs</i> を文字列に変換する.
<code>unpack s</code>	文字列 <i>s</i> を文字のリストに変換する.
<code>unconsString s</code>	文字列 <i>s</i> の先頭文字と残りの文字列のタプルを返す.
<code>lengthString s</code>	文字列 <i>s</i> の長さ（文字数）を返す.
<code>appendString s₁ s₂</code>	文字列 <i>s₁</i> と <i>s₂</i> を連結した文字列を返す.
<code>splitString pat s</code>	文字列 <i>s</i> をパターン <i>pat</i> で分割し、部分文字列のリストを返す.
<code>regex pat s</code>	正規表現 <i>pat</i> で文字列 <i>s</i> をマッチし、マッチ前・マッチ部分・マッチ後の 3 つ組を返す.
<code>regexCg pat s</code>	正規表現 <i>pat</i> で文字列 <i>s</i> をマッチし、キャプチャグループを含む結果を返す.
<code>read s</code>	文字列 <i>s</i> を Egison の式として解析し、評価した結果を返す.
<code>readTsv s</code>	タブ区切り文字列 <i>s</i> を解析し、評価した結果を返す.
<code>show x</code>	値 <i>x</i> を文字列表現に変換する.
<code>showTsv x</code>	値 <i>x</i> をタブ区切り形式の文字列に変換する.

表 7.6: 文字列操作関数一覧

関数名	関数の内容
<code>itof n</code>	整数 <i>n</i> を浮動小数点数に変換する.
<code>rtof r</code>	有理数 <i>r</i> を浮動小数点数に変換する.
<code>ctoi c</code>	文字 <i>c</i> をその Unicode コードポイント（整数）に変換する.
<code>itoc n</code>	整数 <i>n</i> を対応する Unicode 文字に変換する.
<code>isInteger x</code>	値 <i>x</i> が整数であれば True, そうでなければ False を返す.
<code>isRational x</code>	値 <i>x</i> が有理数であれば True, そうでなければ False を返す.

表 7.7: 型変換・型チェック関数一覧

関数名	関数の内容
<code>tensorShape <i>t</i></code>	テンソル <i>t</i> の形状（各次元のサイズ）をリストとして返す.
<code>tensorToList <i>t</i></code>	テンソル <i>t</i> の要素をリストに変換する.
<code>dfOrder <i>t</i></code>	微分形式 <i>t</i> の階数を返す.
<code>addSubscript <i>sym sub</i></code>	シンボル <i>sym</i> に下付き添字 <i>sub</i> を追加する.
<code>addSuperscript <i>sym sup</i></code>	シンボル <i>sym</i> に上付き添字 <i>sup</i> を追加する.

表 7.8: テンソル関連関数一覧

関数名	関数の内容
<code>assert <i>label test</i></code>	テスト <i>test</i> が <code>True</code> でなければラベル <i>label</i> を含むアサーションエラーを発生させる.
<code>assertEqual <i>label actual expected</i></code>	値 <i>actual</i> と <i>expected</i> が等しくなければラベル <i>label</i> を含むアサーションエラーを発生させる.

表 7.9: デバッグ・テスト関数一覧

第 8 章

Sweet Egison - Egison パターンマッチの Haskell ライブラリ

Egison パターンマッチの実装には、本書でここまで紹介してきた Egison の言語実装のほかに、既存言語上の Domain Specific Language (DSL) として実装したライブラリ実装もある。既存言語のライブラリ実装には、deep embedding（インタプリタを埋め込むにより DSL を実現する手法）による実装である miniEgison 系と、shallow embedding（コンパイラを埋め込むことにより DSL を実現する手法）による実装である Sweet Egison 系の二種類がある。本章は、これらの実装のなかでもっとも高速な Sweet Egison の Haskell 実装の使い方を紹介する。Egison と異なり Haskell は静的型システムをもつ。そのため、Sweet Egison を使ったパターンマッチはコンパイル時に型検査がなされる。

8.1 Sweet Egison の使い方

Sweet Egison は Hackage を通して Haskell ライブラリとして公開されている。本節は Sweet Egison をすでにインストールしているものとして使い方を解説する。

Sweet Egison には Egison と同様に `matchAll` 式がある。`matchAll` は引数として探索戦略とターゲットマッチャー、マッチ節のリストをとる。Sweet Egison では `matchAll` は Haskell の関数として実装されており、`as` や `with` のようなキーワードは挟まない。

```
matchAll dfs [1, 2, 3] (List Something)
  [[mc| $x : $xs -> (x, xs) |]]
-- [(1, [2, 3])]
```

上記の `matchAll` 式は探索戦略として深さ優先探索 `dfs` が指定されている。Sweet Egison ではユーザが新しい評価戦略を定義できる。これは Egison にはない Sweet Egison の機能である。探索戦略の定義の方法は 8.6 節で解説する。上記の `matchAll` 式のマッチャーは `(List Something)` である。8.5 節で改めて解説するように、Sweet Egison ではマッチャーは Haskell のデータとして表現さ

れる。マッチ節は `mc` クアジ・クォーター (quasi-quoter) を使って表現される。クアジ・クォーターは Template Haskell が提供する機能のひとつで、`[mc|と|]` で囲まれた部分を文字列として受け取り、Haskell プログラムを返す関数を定義することで、ユーザは Haskell メタプログラミングすることができる。マッチ節がどのような Haskell プログラムに展開されるのかは、8.3 節で紹介する。コンス・パターンは Haskell と同様にコロンひとつで表現する。

ライブラリに最初から定義されている探索戦略には、深さ優先探索 `dfs` と幅優先探索 `bfs` がある。深さ優先探索 `dfs` は、無限にあるパターンマッチの結果をすべて列挙できないことがあるが、高速である。

```
take 10 (matchAll dfs [1..] (Set Something))
  [[mc| $x : $y : _ -> (x, y) |]]
-- [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (1, 8), (1, 9), (1, 10)]
```

幅優先探索 `bfs` は、無限にあるパターンマッチの結果をすべて列挙できるが、深さ優先探索 `dfs` にくらべて実行効率がよくない。

```
take 10 (matchAll bfs [1..] (Set Something))
  [[mc| $x : $y : _ -> (x, y) |]]
-- [(1, 1), (1, 2), (2, 1), (1, 3), (2, 2), (3, 1), (1, 4), (2, 3), (3, 2), (4, 1)]
```

`matchAll` 式がマッチ節のリストを引数にとるのは、複数のマッチ節を扱うためである。Egison と同様に “`matchAll t m [c1, c2, ...]`” というプログラムは、 “`matchAll t m [c1] ++ matchAll t m [c2] ++ ...`” と同値である。

```
matchAll [1, 2, 3] (List Something)
  [[mc| $x : $xs -> (x, xs) |]
  , [mc| _ : $x : $xs -> (x, xs) |]]
-- [(1, [2, 3]), (2, [3])]
```

Sweet Egison では、Egison と同様に `#` を使って値パターンを表現する。値パターンを使って非線形パターンを表現できる。

```
matchAll [1, 5, 2, 4] (Multiset Eql)
  [[mc| $x : #(x + 1) : _ -> (x, x + 1) |]]
-- [(1, 2), (4, 5)]
```

最初のパターンマッチの結果だけを返す `match` も Sweet Egison には実装されている。 `matchAll` を使って `match` は以下のように定義される。

```
match s tgt m cs = head (matchAll s tgt m cs)
```

`match` を使ってポーカーの役判定は図 8.1 のように記述できる。Sweet Egison では、ターゲットのデータ型とマッチャーの両方でデータコンストラクタを使うために、データコンストラクタの名前がかぶりやすい。そのため、マッチャーのほうに `CardM` のように末尾にマッチャーの `M` をつけて、名前の衝突を回避することがある。


```

data Suit = Spade | Heart | Club | Diamond deriving (Eq)
data Card = Card Suit Integer

poker :: [Card] -> String
poker cs =
  matchDFS cs (Multiset CardM)
    [[mc| [card $s $n, card #s #(n-1), card #s #(n-2), card #s #(n-3), card #s #(n-4)] ->
      "Straight flush" |],
    [mc| [card _ $n, card _ #n, card _ #n, card _ #n, _] -> "Four of a kind" |],
    [mc| [card _ $m, card _ #m, card _ #m, card _ $n, card _ #n] -> "Full house" |],
    [mc| [card $s _, card #s _, card #s _, card #s _, card #s _] -> "Flush" |],
    [mc| [card _ $n, card _ #(n-1), card _ #(n-2), card _ #(n-3), card _ #(n-4)] -> "
Straight" |],
    [mc| [card _ $n, card _ #n, card _ #n, _, _] -> "Three of a kind" |],
    [mc| [card _ $m, card _ #m, card _ $n, card _ #n, _] -> "Two pair" |],
    [mc| [card _ $n, card _ #n, _, _, _] -> "One pair" |],
    [mc| _ -> "Nothing" |]]

```

図 8.1: ポーカーの役判定をする Haskell プログラム

8.2 Sweet Egison の仕組み

Sweet Egison の実装のアイデアを説明するために、まず以下の `matchAll` 式がどのような Haskell プログラムに変換されるのかみる。 `cons $x _` は `$x : _` と同値のパターンである。Sweet Egison では中置演算子は組み込みで `:` と `++` のみが実装されている。説明をわかりやすくするために、本節では前置記法でコンス・パターンを記述する。

```
matchAll dfs [1, 2, 3] (Multiset Something) [[mc| cons $x _ -> x |]]
```

上記のプログラムは以下のようなリスト・モナドについての `do` 式に展開される。

```

(\ (matcher, target) -> do
  (x, _) <- cons matcher target
  let (_, _) = consM matcher target
  return x)
(Multiset Something, [1, 2, 3]) -- [1, 2, 3]

```

展開後のプログラムにあらわれる `cons` と `consM` はそれぞれネクスト・ターゲット、ネクスト・マッチャーを計算するための関数である。 `cons` と `consM` は Haskell で定義された関数である。 `cons` と `consM` は、マッチャーとターゲットを引数に取る。マッチャーを引数にとる理由はパターンの多相性を実現するためである。

```
cons (Multiset Something) [1, 2, 3] -- [(1, [2, 3]), (2, [1, 3]), (3, [1, 2])]
consM (Multiset Something) [1, 2, 3] -- (Something, Multiset Something)
```

```
matchAll dfs [1, 2, 3] (Multiset Something) [[mc| cons $x (cons $y _) -> (x, y) |]]
-- [(1, 2), (1, 3), (2, 1), (2, 3), (3, 1), (3, 2)]
```

```
(\ (matcher, target) -> do
  (x, target') <- cons matcher target
  let (_, matcher') = consM matcher target
  (y, _) <- cons matcher' target'
  let (_, _) = consM matcher' target'
  return (x, y))
(Multiset Something, [1, 2, 3])
```

値パターンを含むパターンの展開もみる。

```
matchAll dfs [1, 2, 3] (Multiset Eq1)
  [[mc| cons $x (cons #(x * 2) _) -> (x, y) |]]
-- [(1, 2), (1, 3), (2, 1), (2, 3), (3, 1), (3, 2)]
```

valuePatはHaskellで定義された関数であり、さきほど登場したcons関数と同様にHaskellで定義された関数である。

```
(\ (matcher, target) -> do
  (x, target') <- cons matcher target
  let (_, matcher') = consM matcher target
  (target'', _) <- cons matcher' target'
  let (matcher'', _) = consM matcher' target'
  valuePat (x * 2) matcher'' target''
  return (x, y))
(Multiset Something, [1, 2, 3])
```

8.3 Sweet Egison の最適化

実際のSweet Egisonは節で解説した展開よりも、最適化のために、もうすこし複雑な展開をする。特にワイルドカード最適化のために特別な工夫がある。本節ではこれらの工夫を紹介する。

8.3.1 ワイルドカードのパターンマッチの最適化

6.6節で述べたワイルドカード最適化をSweet Egisonでもおこなうことができる。ワイルドカード最適化とは、ワイルドカードを含むパターンについて、ワイルドカードにマッチするターゲットの計算を省くことにより、パターンマッチの処理を高速化することであった。本節は、6.6

節と同様に、多重集合に対するコンス・パターンの第二引数がワイルドカードであった場合の最適化の例を使って Sweet Egison でこの最適化を使う方法を説明する。

```
matchAll dfs [1, 2, 3] (Multiset Something) [[mc| $x : _ -> x |]]
-- [1, 2, 3]
```

上記の matchAll 式はワイルドカード最適化のために以下のように展開される。

```
(\ (matcher, target) -> do
  (x, _) <- cons (GP, WC) matcher target
  let (_, _) = consM matcher target
  return x)
(Multiset Something, [1, 2, 3])
```

2 行目の cons 関数が第一引数にパターンに対するパターンを追加で引数にとっている。パターンに対するパターンは、ワイルドカードとそれ以外のパターンを区別する。以下、PP はパターンに対するパターン (Patterns for Patterns)、WC はワイルドカード (Wildcard)、GP は一般のパターン (General Patterns) の略である。

```
data PP a = WC | GP
```

cons 関数は第一引数として追加でこのパターンに対するパターンを追加で引数にとる。以下のように cons パターンの第二引数がワイルドカードである場合は、その第二引数に対するネクスト・ターゲットとして undefined を返す。

```
cons (GP, WC) (Multiset Something) [1, 2, 3]
-- [(1, undefined), (2, undefined), (3, undefined)]
```

cons パターンの第二引数がワイルドカード以外である場合は、ネクスト・ターゲットとしてコレクションを計算して返す。

```
cons (GP, GP) (Multiset Something) [1, 2, 3]
-- [(1, [2, 3]), (2, [1, 3]), (3, [1, 2])]
```

8.3.2 パターン・フュージョン

6.5 節で解説した最適化であるパターン・フュージョンは、パターンのパターンマッチがでない Sweet Egison ではユーザーが追加することができない。リストに対するパターンマッチで頻出するジョイン・コンス・パターンについてのみ組み込みでパターン・フュージョンが定義されている。Sweet Egison はジョイン・コンス・パターン $p_1 ++ p_2 : p_3$ を $\text{joinCons } p_1 \ p_2 \ p_3$ に変換する。

```
matchAll dfs [1, 2, 3] (List Something) [[mc| _ ++ $x : _ -> x |]]
-- [1, 2, 3]
```

たとえば、上記の matchAll 式は以下のように展開される。

```
(\ (matcher, target) -> do
  (_, x, _) <- joinCons (WC, GP, WC) matcher target
  let (_, _, _) = joinConsM matcher target
  (List Something, [1, 2, 3])
```

8.4 マッチャーとパターンの型

マッチャーの型はクラスメソッドのない型クラスを使って表現される。

```
class Matcher m tgt
```

この型クラスを使って、たとえば Something マッチャーは下記のように定義される。データ Something をもつ型 Something を定義し、型 Something のデータは型 a のマッチャーであるという関係をインスタンス宣言で表現している。本来は、データ Something に Matcher a のような型を直接つけることが理想であるが、これは Haskell の型システムの制約のためできない。

```
data Something = Something
```

```
instance Matcher Something a
```

パターンは、8.3 節で述べたように、パターンに対するパターンと、マッチャー、ターゲットを引数にとり、ネクスト・ターゲットを返す関数として定義される。

```
type Pattern ps im it ot = ps -> im -> it -> [ot]
```

パターンの多相性を実現するためには、パターンを型クラスに属する関数として定義する。たとえば、コレクションに対するパターンは以下のような型クラスを使って定義される。

```
class CollectionPattern m t where
  type ElemM m
  type ElemT t
  nil :: Pattern () m t ()
  nilM :: m -> t -> ()
  cons :: Pattern (PP (ElemT t), PP t) m t (ElemT t, t)
  consM :: m -> t -> (ElemM m, m)
  join :: Pattern (PP t, PP t) m t (t, t)
  joinM :: m -> t -> (m, m)
  joinCons :: Pattern (PP t, PP (ElemT t), PP t) m t (t, ElemT t, t)
  joinConsM :: m -> t -> (m, ElemM m, m)
```

8.5 Sweet Egison のマッチャー定義

多重集合のマッチャーは以下のように定義される。

```

newtype Multiset m = Multiset m

instance Matcher m t => Matcher (Multiset m) [t]

instance Matcher m t => CollectionPattern (Multiset m) [t] where
  type ElemM (Multiset m) = m
  type ElemT [t] = t
  cons (_, WC) (Multiset _) xs = map (\x -> (x, undefined)) xs
  cons _      (Multiset _) xs = matchAll dfs xs (List Something)
    [[mc| $hs ++ $x : $ts -> (x, hs ++ ts) |]]
  consM (Multiset m) _ = (m, Multiset m)

```

8.6 ユーザによる探索戦略の追加

バックトラッキングによる探索アルゴリズムを記述することに使えるモナドをバックトラッキング・モナドという。代表的なバックトラッキング・モナドとしてリスト・モナドが知られている。リスト・モナドは以下のようにバックトラッキングを記述することに使える。リスト・モナドを使って 1 から 3 の整数のペアを列挙している。

```

do ns <- [1..3]
  x <- ns
  y <- ns
  return (x, y)
-- [(1,1),(1,2),(1,3),(2,1),(2,2),(2,3),(3,1),(3,2),(3,3)]

```

リスト・モナドは探索木を深さ優先探索の順番で探索する。そのため、探索木が無限に大きい場合、すべての結果を列挙できないことがある。

```

do ns <- [1..]
  x <- ns
  y <- ns
  return (x, y)
-- [(1,1),(1,2),(1,3),(1,4),(1,5),...]

```

ここで、深さ優先探索に対するリスト・モナドのように、深さ優先探索以外の探索戦略についても対応するモナドが存在するのかという疑問が生じる。Spivey によって幅優先探索について、リストのリストをベースのデータ構造にするモナドが対応することが発見されている [9]。このモナドは n 要素目のリストに、探索木の深さ n の場所にあるノードを保持する。

バックトラッキング・モナドは、fromList と toList という関数をもつ。これらはモナドのベースとなるデータ構造とリストとの間の変換関数である。深さ優先探索のモナドの場合は、ベースとなるデータ構造がリストであるため、どちらも id 関数として実装できる。幅優先探索のモナド

の場合は、ベースとなるデータ構造がリストのリストであるため、`toList`関数が `concat` になったりする。バックトラッキング・モナドの実装の詳細は、`backtracking` という名前の Haskell ライブラリとして Hackage で公開されているのでそちらを参照してほしい [3]。

バックトラッキング・モナドを使うと探索戦略を多相的に切り替えることができる。以下 `dfs` と `bfs` はそれぞれ深さ優先探索と幅優先探索のモナドのベースとなるデータ構造を初期化するための関数である。`dfs` と `bfs` を置き換えるだけで、探索戦略を制御できる。

```
toList (do
  ns <- dfs [1..]
  x <- fromList ns
  y <- fromList ns
  return (x, y))
-- [(1,1),(1,2),(1,3),(1,4),...]
```

```
toList (do
  ns <- bfs [1..]
  x <- fromList ns
  y <- fromList ns
  return (x, y))
-- [(1,1),(1,2),(2,1),(1,3),...]
```

`matchAll` の第一引数に渡されるのはこの探索戦略ごとのモナドを初期化するための関数である。

```
matchAll strategy [1..] (Set Something) [[mc| $x : $y : _ -> (x, y) |]]
```

`matchAll` の変換結果に適切に、この初期化関数と `fromList` と `toList` を挿入すれば、探索戦略を制御することができる。

```
toList
  ((\ (matcher, target) -> do
    (x, target') <- fromList (cons matcher target)
    let (_, matcher') = consM matcher target
    (y, _) <- fromList (cons matcher' target')
    let (_, _) = consM matcher' target'
    return (x, y))
  (strategy (Set Something, [1..])))
```

深さ優先探索や幅優先探索のためのモナドは Haskell で定義されている。ユーザーが新しく別の探索戦略のためのモナドを追加することも可能である。この機能は Egison 本体にはない Sweet Egison ならではの機能である。

第 II 部

数式処理システムとしての Egison

第 9 章

数式処理システム入門

数式処理システムとは、シンボリックな計算をサポートするプログラミング言語のことをいう。数式処理システムは、未束縛の変数をシンボルとして扱う。数式処理システムを使うと、たとえば、 $x + x \rightarrow 2x$ や $(x + y)^2 \rightarrow x^2 + 2xy + y^2$ のような計算ができる。Egison もこのようなシンボリックな計算をサポートしている。本章は数式処理システムを使うとどのような計算ができるのかみていく。

9.1 シンボルの宣言

数式処理システムは、未束縛の変数をシンボルとして扱う。Egison では、シンボルを使う前に `declare symbol` 文で宣言する必要がある。シンボルには型を指定でき、省略した場合は `MathExpr` 型が使われる。

```
declare symbol x, y, z : MathExpr
```

```
x -- x
```

数式処理システムとしての機能をもつ Egison には、シンボル同士の足し算や掛け算が組み込みで定義されている。そのため、たとえば、 $x + x \rightarrow 2x$ や $(x + y)^2 \rightarrow x^2 + 2xy + y^2$ のような計算ができる。

```
x + x -- 2x
```

```
(x + y)^2 -- x^2 + 2 * x * y + y^2
```

Egison は数式を自動で積和標準形に展開する。積和標準形とは、掛け算が内側に、足し算が外側になるような形の数式のことである。そのため、 $(x + y)^2$ は $x^2 + 2xy + y^2$ のように展開される。

出力の数式の項の因子として現れるシンボルの順番は、ユーザーが入力した数式に現れる順番と同じになるように実装されている。たとえば、上記の $(x + y)^2$ の x と y を入れ替えて $(y + x)^2$ を計算すると 2 項目は $2yx$ となる。

```
(y + x)^2 -- y^2 + 2 * y * x + x^2
```


9.2 関数適用のシンボル化

`sqrt x`や`exp x`のようにシンボルを引数にとる可能性がある関数について、関数適用を止めて、その式をシンボリックな値としてそのまま返すようにライブラリで定義されている。

```
declare symbol x : MathExpr
```

```
sqrt 4 -- 2
sqrt x -- sqrt x
exp 1 -- e
exp x -- exp x
```

`sqrt 2`のように評価を進められない関数適用についても、関数適用を止めて、その式をそのまま返すようにライブラリで定義されている。 $\sqrt{8} \rightarrow 2\sqrt{2}$ のような簡約もライブラリで定義されている。

```
sqrt 2 -- sqrt 2
sqrt 8 -- 2 * sqrt 2
```

これらの定義については、10.3 節で紹介するシングルクォートを使って定義される。

9.3 数式の簡約

$\sqrt{x} \cdot \sqrt{x} = x$ や $\sin^2 \theta + \cos^2 \theta = 1$ のような数式を紙の上で扱うときにおこなう簡約が、Egison を含む多くの数式処理システムには実装されている。

```
declare symbol x, θ : MathExpr
```

```
sqrt x * sqrt x -- x
(sin θ)^2 + (cos θ)^2 -- 1
```

Egison では、このような簡約のための機能は、Sweet Egison(Egison のパターンマッチ機能を提供する Haskell ライブラリ) を使って Haskell により言語に組み込みで実装されている。

上記以外にも、複素数などについての簡約ルールも実装されている。

```
i^2 -- -1
w^3 -- 1
w + w^2 -- -1
```

簡約ルールは <https://github.com/egison/egison/blob/master/hs-src/Language/Egison/Math/Normalize.hs> で定義されている。

数式の簡約の自動化は非常に難しく、Egison では簡潔なアルゴリズムしか用意されていない。たとえば、多項式の約分にはグレブナー基底についての理論に基づいたアルゴリズムが有用であ

るが、Egison には実装されていない。そのため、Egison は複雑な多項式の分数の適切に簡約できないことがある。

Wolfram 言語 (Mathematica) は、このような高度な数式の簡約ができる。“-M mathematica” オプションを追加して Egison 処理系を実行すると Wolfram 言語で読み込めるかたちで数式を出力する。この出力を Wolfram 言語の処理系に入力すれば、Wolfram 言語の高度な簡約機能を利用できる。

```
$ egison -M mathematica
> x + sqrt 2
#mathematica|x + Sqrt[2]|#
```

9.4 二次方程式を解くプログラム - 数式を処理するアルゴリズムの記述

本節では、二次方程式を解くアルゴリズムをプログラムとして書いていく。以下のように二次方程式を引数にとり、解を返す関数 qF を定義していく。

```
qF (x ^ 2 + x + 1) x
-- ((-1 + i * sqrt 3) / 2, (-1 - i * sqrt 3) / 2)

qF (x ^ 2 + b * x + c) x
-- ((- b + sqrt (b^2 - 4 * c)) / 2, (- b - sqrt (b^2 - 4 * c)) / 2)

qF (a * x ^ 2 + b * x + c) x
-- ((- b + sqrt (b^2 - 4 * a * c)) / (2 * a), (- b - sqrt (b^2 - 4 * a * c)) / (2 * a))

qF (a * x ^ 2 + 2 * b * x + c) x
-- ((- b + sqrt (b^2 - a * c)) / a, (- b - sqrt (b^2 - a * c)) / a)
```

qF は解の公式 $\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ をそのまま書くだけでも定義することができる。以下のプログラムは qF' という補助関数を使って qF を定義している。qF' は第一引数に二次の係数、第二引数に一次の係数、第三引数に定数項をとる。たとえば、qF' 1 0 2 は 2 次方程式 $x^2 + 2 = 0$ の解を返す。coefficients は多項式とシンボルを引数にとり、そのシンボルについて係数のリストを返す関数である。qF' は単純に $\frac{-b + \sqrt{b^2 - 4ac}}{2a}$ と $\frac{-b - \sqrt{b^2 - 4ac}}{2a}$ からなるタプルを返す関数として定義されている。

```
def qF f x :=
  match coefficients f x as list mathExpr with
  | [$a_0, $a_1, $a_2] -> qF' a_2 a_1 a_0

def qF' a b c :=
  ( ((- b) + sqrt (b ^ 2 - 4 * a * c)) / 2 * a
```

```
, ((- b) - sqrt (b ^ 2 - 4 * a * c)) / 2 * a )
```

しかし、上記の記述ではアルゴリズムっぽさがないため、今回は平方完成によりこの解の公式を求めるアルゴリズムを書いていく。平方完成とは、 x^2+bx+c のような二次式を $(x+b/2)^2-b^2/4+c$ のように変形する操作のことである。

```
1 def qF' a b c :=
2   match (a, b, c) as (mathExpr, mathExpr, mathExpr) with
3     | (#1, #0, _) -> (sqrt (- c), - sqrt (- c))
4     | (#1, _, _) ->
5       (2)#((- (b / 2)) + %1, (- (b / 2)) + %2)
6       (withSymbols [x, y]
7         qF (substitute [(x, y - b / 2)] (x ^ 2 + b * x + c)) y)
8     | (_, _, _) -> qF' 1 (b / a) (c / a)
```

平方完成を使って二次方程式を解くアルゴリズムを記述すると qF' は 3 つのマッチ節からなる関数となる。3 行目のマッチ節は、 $x^2 + c = 0$ のような形の二次方程式を処理する。このマッチ節は、 $\sqrt{c}$ と $-\sqrt{c}$ という結果を返す。4-7 行目のマッチ節は、 $x^2 + bx + c = 0$ のような二次の係数が 1 である形の二次方程式を処理する。このマッチ節が平方完成の操作をするこの関数の本体である。substitute は Egison のライブラリ関数である。substitute は第一引数で指定された代入を第二引数の式に対しておこなう。ここでは第一引数で、 $(x, y - b / 2)$ という代入が指定されているため、 $x = y - b/2$ という代入を式 $x^2 + bx + c$ に対しておこなう。同時に複数の代入を指定するために、substitute の第一引数はリストを引数に取るようになっている。 $x = y - b/2$ という代入をするとこの方程式は 3 行目のマッチ節でマッチする y についての二次方程式に変形される。そのため、qF 関数にこの変形された方程式を渡せば、 y についてこの二次方程式の解を得られる。この y についての解に $-\frac{b}{2}$ を加えることにより、もとの x についての解を得ることができる。withSymbols 式は、局所シンボルを生成する構文である。第一引数のリストで指定されたシンボル（上記の場合、 x と y ）を第二引数の式の中でシンボルとして使うことができる。withSymbols 式のこの機能のおかげで、もしプログラムの別の箇所で、 x や y にたとえば具体的な整数が束縛されていたとしても、withSymbols 式の内部では、その影響を考えなくてよくなる。8 行目のマッチ節は、二次の係数が 1 でない二次方程式を処理する。二次の係数で方程式を割ることにより、二次の係数が 1 の方程式に変換し、ふたたび qF 関数にわたす処理をしている。

二次方程式より一段と複雑になるが、三次方程式・四次方程式を解くプログラムを書いてみると面白い。これらのプログラムは Egison のソースコードの sample/math/algebra ディレクトリ以下に公開されている。

9.5 微分するプログラム - 数式に対するパターンマッチ

Egison には微分計算をするための関数 d/d がライブラリ関数として実装されている。この関数は第一引数の関数を第二引数のシンボルについて微分した結果を返す。関数の名前を d/d のように

することで数式に近い形で微分を表現できている。Egison は/を変数名に使うことを許している。

```
d/d x x -- 1
d/d (x^2) x -- 2 * x
d/d (exp x) x -- exp x
d/d (log x) x -- 1 / x
d/d (x * log x) x -- log x + 1
d/d (1 / log x) x -- -1 / (x * (log x)^2)
```

本節はこの関数の実装を紹介する。

d/dは以下のように定義される。

```
1 def d/d f x :=
2   match f as mathExpr with
3     -- シンボルの微分
4     | #x -> 1
5     | ?isSymbol -> 0
6     -- 関数適用の微分
7     | #'exp $g -> exp g * d/d g x
8     | #'log $g -> 1 / g * d/d g x
9     | #'sqrt $g -> 1 / (2 * sqrt g) * d/d g x
10    | #'(^) $g $h -> f * d/d (log g * h) x
11    | #'cos $g -> (- sin g) * d/d g x
12    | #'sin $g -> cos g * d/d g x
13    -- 定数の微分
14    | #0 -> 0
15    | _ * #1 -> 0
16    -- 項の微分
17    | #1 * $fx ^ $n -> n * fx ^ (n - 1) * d/d fx x
18    | $a * $fx ^ $n * $r -> a * d/d (fx ^ n) x * r + a * fx ^ n * d/d r x
19    -- 多項式の微分
20    | poly $ts -> sum (map 1#(d/d %1 x) ts)
21    -- 商の微分
22    | $p1 / $p2 ->
23      let p1' := d/d p1 x
24      p2' := d/d p2 x
25      in (p1' * p2 - p2' * p1) / p2 ^ 2
```

第一引数の f を数式としてパターンマッチしている。4-5 行目はシンボルの微分を定義している。もし、 f が x そのものであったら、1 を返し、 f が x 以外のシンボルであったら 0 を返すように定義されている。7-12 行目はいくつかの基本的な関数の適用について合成関数の微分 $\frac{df(g(x))}{dx} = \frac{df(g(x))}{dg(x)} \frac{dg(x)}{dx}$ を記述している。たとえば、7 行目は指数関数の微分をしている。このマッチ節の本体は $\frac{de^{g(x)}}{dx} = e^{g(x)} \frac{dg(x)}{dx}$ を表現している。また、8 行目は対数関数の微分をしている。このマッチ節の本体は $\frac{d\log(g(x))}{dx} = \frac{1}{g(x)} \frac{dg(x)}{dx}$ を表現している。14-15 行目は定数項に対する

微分を定義している。定数項は微分すると 0 になる。17-18 行目は、ここまでで微分を定義した項を掛け合わせた項についての微分を定義している。 x^n は nx^{n-1} に微分されることと、 fg は $f'g + fg'$ に微分されることを利用して定義している。ここでも合成関数の微分の公式が使われている。20 行目は多項式の微分が定義されている。ここまでの単項式についての微分が定義されているので、それらを足し合わせたものについては、それぞれの項について微分して足し合わせてやればよい。22-24 行目は多項式を分母と分子にもつ式に対する微分が定義されている。商の微分の公式を使って定義されている。

9.6 ベクトルや行列の計算 - 添字記法

Egison の数式処理システムとしての特徴に、ベクトルや行列、そしてそれらを一般化したテンソルを扱う計算を簡潔に記述できるというものがある。これは二十世紀初頭に微分幾何や相対性理論の研究の過程で発明されたテンソルの添字記法をプログラミング言語の機能として Egison がサポートしているためである。

添字記法は、テンソルに添字を付加することによって、さまざまな種類のテンソルの掛け算を表現する。たとえば、ベクトル同士の掛け算には、テンソル積・アダマール積・内積の三種類があるが、これらは以下のように添字を使い分けることによって表現される。

```
[| x1, x2 |]_i . [| y1, y2 |]_j -- [| [| x1 * y1, x1 * y2 |], [| x2 * y1, x2 * y2 |] |]
    _i_j
[| x1, x2 |]_i . [| y1, y2 |]_i -- [| x1 * y1, x2 * y2 |]_i
[| x1, x2 |]~i . [| y1, y2 |]_i -- x1 * y1 + x2 * y2
```

上記の例のように、Egison ではベクトルは成分を `[| |]` で囲むことにより表現する。また行列は、ベクトルのベクトルとして表現する。関数型プログラミングの視点で添字記法は、テンソルの掛け算関数のパラメータとして、テンソルだけでなく添字も追加で渡すことによって一つの関数に複数の役割を持たしていると説明できる。

添字記法を言語機能としてもつプログラミング言語はほかにも存在する。そのなかで Egison の特徴は、関数型プログラミングとうまく調和するかたちで添字記法を組み込んでいるところである。そのおかげで、高階関数と添字記法を組み合わせることができる。たとえば、 $g_{i_1 j_1} g_{i_2 j_2} \dots g_{i_n j_n}$ のような数式を `foldl` 関数を使って以下のように記述することができる。

```
foldl . 1 (map (\x -> g_[i_x]_[j_x]) [1..n])
```

添字記法は Egison の重要な言語機能であるため、第 11 章と第 13 章で詳しく紹介する。

9.7 数式の出力形式

“-M” オプションで出力形式を指定することができる。“egison -M latex”を実行すると、LaTeX 形式で結果を出力する。

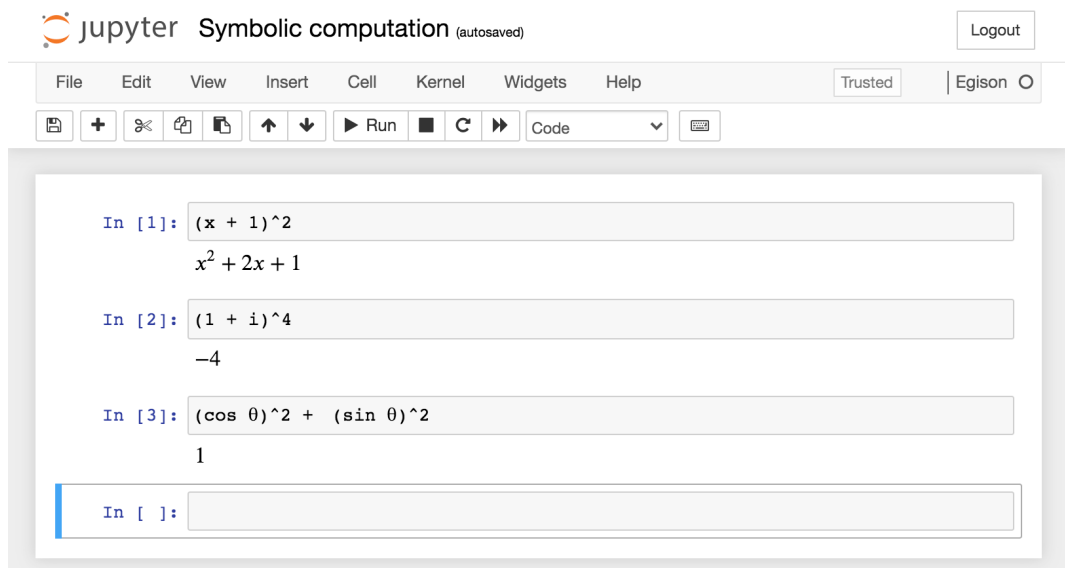


図 9.1: Jupyter Notebook 上で Egison を実行する様子

```
$ egison -M latex
Egison Version 3.7.14 (C) 2011-2018 Satoshi Egi
https://www.egison.org
Welcome to Egison Interpreter!
> (+ (sqrt a) b)
#latex|\sqrt{a} + b|#
```

“egison -M mathematica” を実行すると、Mathematica 形式で結果を出力する。

```
$ egison -M mathematica
Egison Version 3.7.14 (C) 2011-2018 Satoshi Egi
https://www.egison.org
Welcome to Egison Interpreter!
> (+ (sqrt a) b)
#mathematica|Sqrt[a] + b|#
```

LaTeX 出力は Jupyter Notebook と連携している。Jupyter Notebook の Egison プラグインを利用すれば、図 13.1, 図 13.4 のようにインタラクティブに計算結果が数式として表示される。

第 10 章

数式データの扱い

本章は、シンボリックな計算を扱うための Egison の機能を紹介する。

10.1 数式データの内部表現

数式は組み込みデータ型として Egison 処理系内部で Haskell の代数的データ型を用いて直接実装されている。fromMathExpr という組み込み関数を使うと、この数式データを Egison 上の代数的データ型に変換することができる。

```
fromMathExpr (x - sqrt 2)
-- Div (Plus [Term 1 [(Symbol x [], 1)], Term -1 [(Apply sqrt [Div (Plus [Term 2 []])
--      (Plus [Term 1 []]), 1])]) (Plus [Term 1 []])
```

本節は、数式データを表現するこの内部の代数的データ型について説明する。

上記の出力の代数的データ型からもわかるように、数式データ型は、分母と分子の両方に積和標準形の多項式をとる分数として定義されている。Divがこの分数を生成するためのデータコンストラクタで、Plusが多項式を生成するためのデータコンストラクタとなっている。そのため、たとえば分数同士の足し算の結果は一つの分数にまとめられる。

```
a / b + x / y
-- (a * y + x * b) / (b * y)
```

Plusの引数がリストであることからわかるように、多項式は、項のリストからなるデータ型として定義されている。そして、項は係数と因子のリストからなるデータ型として定義されている。項は Term データコンストラクタを使って生成される。Term は第一引数に係数、第二引数に因子のリストをとる。係数には整数、因子にはシンボルなどをとることができる。たとえば、 $3 * x^2 * \sqrt{2}$ という項は、係数として 3、因子のリストとして $[x^2, \sqrt{2}]$ をとる。因子にはシンボル以外に、 $\sqrt{2}$ のような項もとることができる。因子のリストは、底 (x^2 の場合 x) と冪指数 (x^2 の場合 2) の連想リストとなっている。冪指数には自然数しかとれないという制限がある。

```
fromMathExpr (3 * x^2 * sqrt 2)
```

```
-- Div (Plus [Term 3 [(Symbol x []), 2], (Apply sqrt [Div (Plus [Term 2 []]) (Plus [Term 1 []])], 1)]) (Plus [Term 1 []])
```

上記の例のように、因子は `Symbol` や `Apply` といったデータコンストラクタを使って生成される。`Symbol` データコンストラクタは、第一引数にシンボル自身を、第二引数に添字のリストをとる。`Symbol` データコンストラクタの第一引数にシンボル自身にしているのは、`fromMathExpr` の結果を扱いやすくするためである。シンボルを表現するのデータ構造のなかにシンボル自身があると定義が無限に循環してしまうため、Haskell によるシンボルの内部表現は `Symbol` データコンストラクタの第一引数は文字列をとっている。添字については、テンソルについて解説する第 11 章で説明する。`Apply` データコンストラクタは、第一引数に関数名のシンボルを、第二引数に引数の数式のリストをとる。その他に因子のデータコンストラクタに `Quote` と `Function` がある。`Quote` データコンストラクタについては、10.2 節で紹介する。`Function` データコンストラクタについては、第 12 章で紹介する。

`fromMathExpr` の逆に Egison 上の代数的データ型として表現された数式を Egison 処理系内部の数式データに変換する `toMathExpr` 関数も用意されている。`toMathExpr` 関数を使うと、Egison 上の代数的データ型として数式を処理したあとに、これらのデータを Egison 内部の数式データに戻すことができる。

```
toMathExpr (fromMathExpr (x - sqrt 2))
-- x - sqrt 2
```

ただし、`fromMathExpr` と `toMathExpr` のような組み込み関数は、Egison ライブラリで定義されている `mathExpr` マッチャーを使えば、ユーザーが触る必要はほぼない。`mathExpr` マッチャーの実装に `fromMathExpr` と `toMathExpr` は使われている。

数は複雑なデータ型である。自然数までであれば代数的データ型としてエレガントに定義できる。しかし、自然数の足し算の逆関数引き算を考えると、負の数という概念が得られる。さらに足し算を繰り返すことによって得られる掛け算の逆を考えると、有理数という概念が得られる。さらに掛け算を繰り返すことによって得られるべき乗という操作の逆を考えると、平方根や虚数、それらを含む代数的数の概念が得られる。

10.2 バッククオート (‘) による式展開の制御

Egison は数式を自動で積和標準形に展開すると 9.1 で述べたが、この展開をバッククオート (‘) により制御することができる。バッククオートに続く式は、ひとつのシンボルのように扱われる。

```
`(x + 1)^2
-- `(x + 1)^2

`(x + 1) + `(x + 1)
-- 2 * `(x + 1)
```



```
`(x + 1) + (x + 1)
-- `(x + 1) + x + 1
```

バッククオートが役に立つのは、主に下記のような多項式の割り算を含む計算のときである。

```
(x + 1)^2 / (x + 1)
-- (x^2 + 2 * x + 1) / (x + 1)

`(x + 1)^2 / `(x + 1)
-- (x + 1)
```

バッククオートによりクオートされた項は、Egison 内部の数式データのなかで因子として扱われる。Quoteは、Symbolや Applyと同列のデータコンストラクタとして定義されている。

```
fromMathExpr `(x + 1)^2)
-- Div (Plus [Term 1 [(Quote (Div (Plus [Term 1 [(Symbol x []), 1)], Term 1 [])] (Plus [Term 1 []])), 2])] (Plus [Term 1 []])fromMathExpr `(x + 1)^2
```

10.3 シングルクオート (') による関数適用の制御

Egison はシンボルを関数として適用するとシンボリックな結果を返す。例えば、未定義の変数 `f` についての関数適用は以下のように動作する。

```
f a -- f a
```

定義済みの関数の適用についても上記のようにシンボリックな結果を返したいことがある。たとえば、`sqrt`関数については以下のように動作してほしい。

```
sqrt 4 -- 2
sqrt 5 -- sqrt 5
```

このような動作を実現するために、Egison には組み込み構文としてシングルクオート (') が用意されている。シングルクオートには変数が続き、たとえその変数が定義済みであったとしても、その変数の名前のシンボルを返す。シングルクオートを関数の前に付加すれば、定義済みの関数に対してもシンボリックな関数適用を表現できる。

```
def f (x: MathExpr) : MathExpr := x + 1

f a -- a + 1
'f a -- f a
```

シングルクオートは平方根を計算する `sqrt`の実装以外にも、三角関数 `sin`や `cos`の実装などで Egison ライブラリ内で使われている。

```
fromMathExpr x
-- Div (Plus [Term 1 [(Symbol x [], 1)]] (Plus [Term 1 []]))
fromMathExpr 'x
-- Div (Plus [Term 1 [(Symbol x [], 1)]] (Plus [Term 1 []]))
```

10.4 withSymbols 式によるローカルシンボルの宣言

未定義の変数はシンボルとして扱われるという仕様は、すでに定義済みの変数をユーザーがシンボルとして扱おうとするミスを誘発しやすい。この問題を解決するために、withSymbols式というローカルシンボルを宣言するための構文が Egison には用意されている。withSymbols式は、第一引数に変数のリストをとり、これらの変数は第二引数の式の評価時にシンボルとして扱われる。

```
def i : Integer := 10

i -- 10
withSymbols [i] i -- i
i + withSymbols [i] i -- 10 + i
```

withSymbols式で宣言されたローカルなシンボルは、未定義変数から生成されるグローバルなシンボルとは独立したシンボルとして処理される。

```
j + withSymbols [j] j -- j + j
```

10.5 数式データに対するパターンマッチ

本節は、数式データに対して用意されているパターンを紹介する。数式に対するマッチャー mathExpr は Egison ライブラリに実装されている。mathExpr の定義は lib/math/expression.egi で確認できる。また数式に対するパターンを、より数式に近いかたちで記述するためにいくつかの糖衣構文が組み込みで実装されている。本節では、これらの糖衣構文も紹介する。数式に対するパターンマッチは、9.5 節で実演されている。パターンの具体的な使用例として、9.5 節のプログラムを参照してほしい。

10.5.1 因子に対するパターンマッチ

因子に対するパターンコンストラクタは、symbol・apply・quote・func の 4 つがある。symbol はシンボルに、apply は 9.2 節で紹介したシンボリックな関数適用に、quote は 10.2 節で紹介したバッククオートされた式に、func は第 12 章で解説する関数シンボルと呼ばれるオブジェクトにパターンマッチするためのパターンコンストラクタである。

`symbol`パターンコンストラクタの第一引数はシンボル自身に、第二引数は添字にパターンマッチする。添字については、テンソルについて解説する第 11 章で改めて紹介する。

```
match x as mathExpr with
| symbol $s _ -> s
-- x
```

シンボルに対しては `symbol`パターンコンストラクタを使ってパターンマッチすることもできるが、実際には値パターンを使ってパターンマッチすることが多い。

```
match x as mathExpr with
| #x -> "Matched"
| _ -> "Not matched"
-- "Matched"
```

`apply`パターンコンストラクタの第一引数は関数名のシンボルに、第二引数は引数の数式のリストにパターンマッチする。引数のパターンマッチには、`list mathExpr`マッチャーが使われるため、リストに対するパターンを使うことができる。

```
match ('f x 1 y) as mathExpr with
| apply $f ($x :: $xs) -> (f, x, xs)
-- (f, x, [1, y])
```

`quote`パターンコンストラクタは引数に数式に対するパターンをとる。このパターンは `mathExpr` マッチャーを使ってクオートの中身の式とパターンマッチされる。

```
match `(x + 1) as mathExpr with
| quote $e -> e
-- x + 1
```

10.5.2 項に対するパターンマッチ

項に対するパターンマッチには、`term`パターンコンストラクタを使う。`term`パターンコンストラクタの第一引数は係数に、第二引数は因子の連想リストにパターンマッチする。因子の連想リストのパターンマッチには、6.7 節で紹介した `assocMultiset` マッチャーが使われる。

```
matchAll 3 * x^2 * y as mathExpr with
| term $a ($x :: $xs) -> (a, x, xs)
-- [(3, x, [(x, 1), (y, 1)]), (3, y, [(x, 2)])]
```

`term`パターンコンストラクタには、より数式に近いパターンを記述するための糖衣構文がある。

```
matchAll 3 * x^2 * y as mathExpr with
| $a * $x * $xs -> (a, x, xs)
-- [(3, x, x * y), (3, y, x^2)]
```

```
matchAll 3 * x^2 * y as mathExpr with
  | $a * $x^#2 * $xs -> (a, x, xs)
-- [(3, x, y)]
```

10.5.3 多項式に対するパターンマッチ

有理式に対するパターンマッチには、polyパターンコンストラクタを使う。polyパターンコンストラクタは引数に項のリストにマッチするパターンをとる。

```
matchAll x + y + 1 as mathExpr with
  | poly ($x :: $xs) -> (x, xs)
-- [(x, [y, 1]), (y, [x, 1]), (1, [x, y])]
```

polyパターンコンストラクタにも、より数式に近いパターンを記述するための糖衣構文がある。

```
matchAll x + y + 1 as mathExpr with
  | $x + $xs -> (x, xs)
-- [(x, y + 1), (y, x + 1), (1, x + y)]
```

10.5.4 有理式に対するパターンマッチ

有理式に対するパターンマッチには、divパターンコンストラクタを使う。divパターンコンストラクタの第一引数は分子の多項式に、第二引数は分母の多項式にパターンマッチする。

```
matchAll (a + b) / c as mathExpr with
  | div $x $y -> (x, y)
-- [(a + b, c)]
```

divパターンコンストラクタにも、より数式に近いパターンを記述するための糖衣構文がある。

```
matchAll (a + b) / c as mathExpr with
  | $x / $y -> (x, y)
-- [(a + b, c)]
```

第 11 章

テンソル計算

Egison にはテンソル計算を便利に記述するために開発された機能が実装されている。本章はこれらの機能を紹介していく。

11.1 テンソルとは

テンソルとは、ベクトルや行列を一般化したものである。ベクトルは一階のテンソル、行列は二階のテンソルである。ベクトルは一次元配列、行列は二次元配列としてプログラムでは表現されることが多い。 n 階のテンソルは、 n 次元配列により表現される。

以下はそれぞれベクトル、行列、2 階のテンソルの例を数式で表現したものである。

$$\begin{aligned} & (a_1 \quad a_2 \quad a_3) \\ & \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix} \\ & \left(\begin{pmatrix} a_{111} & a_{112} & a_{113} \\ a_{121} & a_{122} & a_{123} \end{pmatrix} \begin{pmatrix} a_{211} & a_{212} & a_{213} \\ a_{221} & a_{222} & a_{223} \end{pmatrix} \begin{pmatrix} a_{311} & a_{312} & a_{313} \\ a_{321} & a_{322} & a_{323} \end{pmatrix} \right) \end{aligned}$$

上記の数式は Egison では下記のように表現される。Egison ではテンソルの成分を `[ | ]` で囲むことにより、テンソルを表現する。行列はベクトルのベクトルとして、2 階のテンソルは行列のベクトルとして表現される。これは C 言語などの多次元配列の表記法に似ている。

```
[| a_1, a_2, a_3 |]
```

```
[| [| a_11, a_12, a_13 |],  
  [| a_21, a_22, a_23 |] |]
```

```
[| [| [| a_111, a_112, a_113 |],  
      [| a_121, a_122, a_123 |] |],  
  [| [| a_211, a_212, a_213 |],  
      [| a_221, a_222, a_223 |] |] |]
```

```
[| [| a_311, a_312, a_313 |],
  [| a_321, a_322, a_323 |] |] |]
```

11.2 テンソルの添字記法

テンソル計算は、ベクトルについてのテンソル積・アダマール積・内積の組み合わせでほとんど表現できる。テンソル積は、ベクトルのすべての成分の組み合わせからなる行列を返す。アダマール積は、対応する成分同士のみからなるベクトルを返す。内積は、アダマール積の結果の成分を足し合わせた結果のスカラー値を返す。

$$(a_1 \ a_2) \otimes (b_1 \ b_2) = \begin{pmatrix} a_1 b_1 & a_1 b_2 \\ a_2 b_1 & a_2 b_2 \end{pmatrix}$$

$$(a_1 \ a_2) \circ (b_1 \ b_2) = (a_1 b_1 \ a_2 b_2)$$

$$(a_1 \ a_2) \cdot (b_1 \ b_2) = a_1 b_1 + a_2 b_2$$

たとえば、行列 A と行列 B の掛け算は、 A の行と B の列についてはテンソル積、 A の列と B の行については内積をとる演算と解釈できる。

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} = \begin{pmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{pmatrix}$$

ベクトルについては、掛け算の方法は、テンソル積・アダマール積・内積の3つだけだが、行列や行列よりさらに高階なテンソルとなると掛け算の方法が無数にある。これらの掛け算のすべてに名前をつけるのは大変である。そのため、数学者はシンボリックな添字をテンソルに付加し、その添字をパラメーターにしてこれらの掛け算を表現する。たとえば、それぞれのベクトルに異なるシンボルが付加されている場合はテンソル積に、それぞれのベクトルの同じ位置に同じシンボルが付加されている場合はアダマール積に、それぞれのベクトルの上下に同じシンボルが付加されている場合は内積になる。添字には上添字と下添字の二種類がある。この位置関係によってアダマール積となるか内積となるか区別する。このようにシンボリックな添字を使って、テンソルのかけあわせ方を指定する記法は、添字記法と呼ばれている。下記の数式は Egison で 9.6 節で示したプログラムで表現できる。

$$(a_1 \ a_2)_i (b_1 \ b_2)_j = \begin{pmatrix} a_1 b_1 & a_1 b_2 \\ a_2 b_1 & a_2 b_2 \end{pmatrix}_{ij}$$

$$(a_1 \ a_2)_i (b_1 \ b_2)_i = (a_1 b_1 \ a_2 b_2)_i$$

$$(a_1 \ a_2)^i (b_1 \ b_2)_i = a_1 b_1 + a_2 b_2$$

行列のかけ算も添字記法をつかってうまく表現できる．内積をとる A の列と B の行について、それぞれ上下を逆にし、同じシンボルにすれば、行列の掛け算は表現できる．

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}^i_j \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}^j_k = \begin{pmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{pmatrix}^i_k$$

11.3 添字記法をプログラミングに導入するためのアイデア

添字記法のプログラミングへの導入が難しい理由は、演算子ごとにシンボリックな添字のルールが異なることにある．特にテンソルの足し算と掛け算では、添字のルールが異なる．例を紹介する． u と v をそれぞれベクトルとする．

- $u_i + v_i$ はベクトルを返す．対して、 $u_i v_i$ は無効な式である．($u_i v_i$ は上下添字を自由に入れ替えることが可能なユークリッド空間では内積を返す．しかし、このような例外はここでは考えない.)
- $u^i + v_i$ は無効である．対して、 $u^i v_i$ は内積を返す．
- $u_i + v_j$ は無効である．対して、 $u_i v_j$ はテンソル積を返す．

このように演算子ごとに異なるシンボリックな添字のルールがある．そのため、テンソルを扱う関数を定義するたびに、シンボリックな添字のルールを指定する必要がある．これはわずらわしい作業である．それだけではなく、添字ルールを記述するための新しい構文が必要になる場合もある．このような新しい構文はプログラミング言語を複雑にする．

Egison は、シンボリックな添字のルールを簡略化することによって、この問題を解決する．添字のルールの簡略化によって、有効な数式の解釈は変わらない．ただし、無効であった数式が有効になる場合がある．例えば、 $v_i + v_j$ は無効な式であったが、新しい添字ルールでは有効な式になる．テンソルの足し算についての簡略化された添字ルールを図 11.1a で示す．添字ルールを緩めることによって、多くのテンソルについての演算をテンソルの足し算の変種として捉えることができるようになる．例えば、テンソルの掛け算は、足し算と同じ形で成分を掛け合わせたあとに縮約 (contraction) という追加の処理をする演算と捉えられる (図 11.1b)．テンソルの縮約とは、同じシンボルの上添字と下添字があるときに、対角成分を足し合わせる操作のことをいう．このような見方をすると、テンソルについての関数を定義するときの添字ルールの記述をへらすことができる．

$$\begin{pmatrix} a \\ b \end{pmatrix}_i + \begin{pmatrix} c \\ d \end{pmatrix}_i = \begin{pmatrix} a+c \\ b+d \end{pmatrix}_i \qquad \begin{pmatrix} a \\ b \end{pmatrix}_i \begin{pmatrix} c \\ d \end{pmatrix}_i = \text{contract} \begin{pmatrix} ac \\ bd \end{pmatrix}_i = \begin{pmatrix} ac \\ bd \end{pmatrix}_i$$

$$\begin{pmatrix} a \\ b \end{pmatrix}_i + \begin{pmatrix} c \\ d \end{pmatrix}_j = \begin{pmatrix} a+c & a+d \\ b+c & b+d \end{pmatrix}_{ij} \qquad \begin{pmatrix} a \\ b \end{pmatrix}_i \begin{pmatrix} c \\ d \end{pmatrix}_j = \text{contract} \begin{pmatrix} ac & ad \\ bc & bd \end{pmatrix}_{ij} = \begin{pmatrix} ac & ad \\ bc & bd \end{pmatrix}_{ij}$$

$$\begin{pmatrix} a \\ b \end{pmatrix}_i + \begin{pmatrix} c \\ d \end{pmatrix}^i = \begin{pmatrix} a+c & a+d \\ b+c & b+d \end{pmatrix}^i \qquad \begin{pmatrix} a \\ b \end{pmatrix}_i \begin{pmatrix} c \\ d \end{pmatrix}^i = \text{contract} \begin{pmatrix} ac & ad \\ bc & bd \end{pmatrix}^i = ac + bd$$

(a) 足し算の添字ルール

(b) 掛け算の添字ルール

図 11.1: 簡略化された添字ルール

11.4 スカラー仮引数とテンソル仮引数

テンソルの添字記法のプログラミングへの導入は、(i) 型システムによるスカラー関数とテンソル関数の区別と、(ii) 適切な添字の簡約ルールによってなされる。本節からこれらの概念の解説を始める。

まず、型システムによるスカラー関数とテンソル関数の区別を説明する。Egison では、仮引数の型注釈により、関数がスカラー関数かテンソル関数かが決まる。仮引数の型が `Tensor a` 型でない場合、その仮引数はスカラーとして扱われる。仮引数の型が `Tensor a` 型である場合、その仮引数はテンソルとして扱われる。スカラーとして扱われる仮引数にテンソルが引数として与えられると、テンソルの成分ごとに関数の処理がマップされる。テンソルとして扱われる仮引数にテンソルが引数として与えられると、テンソルがそのまま仮引数に渡される。以下の例では、型変数 `a` がスカラー型と推論され、テンソルが渡されると成分ごとに処理がマップされる。

```
(\u v -> (u, v)) [| a, b |]_i [| x, y |]_j
-- [| [| (a, x), (a, y) |],
--    [| (b, x), (b, y) |] |]_i_j
```

一方、明示的に `Tensor` 型を指定した場合、テンソルがそのまま渡される。

```
(\ (u: Tensor a) (v: Tensor a) -> (u, v)) [| a, b |]_i [| x, y |]_j
-- ([| a, b |]_i, [| x, y |]_j)
```

Egison の型システムは、**スカラー関数** (*scalar function*) と **テンソル関数** (*tensor function*) という 2 種類の関数を区別する。スカラー関数は、テンソルを引数にとったとき、成分ごとに処理をマップする関数のことをいう。たとえば、“+” や、“-”、“*”、“/”、“ ∂/∂ ” はスカラー関数である。


```
def min (x: a) (y: a) : a := if x < y then x else y
```

(a) min関数の定義（型注釈により，任意の比較可能な型 a に対して動作することを示している）

$$\min\left(\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}_i, \begin{pmatrix} 10 \\ 20 \\ 30 \end{pmatrix}_j\right) = \begin{pmatrix} \min(1, 10) & \min(1, 20) & \min(1, 30) \\ \min(2, 10) & \min(2, 20) & \min(2, 30) \\ \min(3, 10) & \min(3, 20) & \min(3, 30) \end{pmatrix}_{ij} = \begin{pmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{pmatrix}_{ij}$$

(b) 異なる添字をもつベクトルにたいする min関数の適用

$$\min\left(\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}_i, \begin{pmatrix} 10 \\ 20 \\ 30 \end{pmatrix}_i\right) = \begin{pmatrix} \min(1, 10) & \min(1, 20) & \min(1, 30) \\ \min(2, 10) & \min(2, 20) & \min(2, 30) \\ \min(3, 10) & \min(3, 20) & \min(3, 30) \end{pmatrix}_{ii} = \begin{pmatrix} \min(1, 10) \\ \min(2, 20) \\ \min(3, 30) \end{pmatrix}_i = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}_i$$

(c) 同じ添字をもつベクトルにたいする min関数の適用

図 11.2: スカラー関数の例として min関数の定義と適用例

テンソル関数は，テンソルを引数にとったとき，テンソルをテンソルとしてそのまま処理する関数のことをいう．たとえば，行列式を計算する関数や，テンソル同士のかけ算のための関数は，テンソルの成分ごとに自動でマップされては記述できないため，テンソル関数として定義する必要がある．関数がスカラー関数かテンソル関数かは型注釈によって決まる．仮引数の型が `Tensor a` 型でない場合，テンソルが引数として渡されると自動的に成分ごとに処理がマップされる（スカラー関数）．仮引数の型が `Tensor a` 型である場合，テンソルはテンソルとしてそのまま渡される（テンソル関数）．図 11.2 と図 11.3 でスカラー関数とテンソル関数の定義の例を説明する．

図 11.2 は，スカラー関数の例として，min関数の定義とその適用例を紹介している．仮引数の型が a 型（型変数）であるため，テンソルが引数として渡された場合，自動的に成分ごとに関数が適用される．成分ごとに関数が適用された結果，図 11.2b のように関数はテンソル積と同じ形で適用される．テンソルの添字の簡約ルールはシンプルである．図 11.2c のようにスカラー関数を適用した結果，同じシンボルの添字が 2 つ以上付加されたテンソルが生成された場合，その対角成分からなるテンソルに簡約されるというルールだけである．添字簡約のルールの詳しい仕様は次節で論じる．

図 11.3 は，テンソル関数の例として，テンソルのかけ算をする “.” 関数の定義と適用例を紹介している．仮引数の型が `Tensor a` 型であるため，テンソルが引数として渡された場合，テンソルとしてそのまま渡される．図 11.3b のように，Egison は上添字と下添字の両方を組み込みでサポートしている．上添字と下添字で同じシンボルが使われていた場合は，関数 `contractWith` を使って縮約することができる．

上添字は “ $\sim$ ”，下添字は “ $_$ ” を使ってプログラム上で表現される．たとえば，図 11.3b の一つ目の計算例は以下のように Egison で表現される．

```
def (.) (t1: Tensor a) (t2: Tensor a) : Tensor a := contractWith (+) (t1 * t2)
```

(a) “.”関数の定義（型注釈により，テンソルを受け取りテンソルを返すことを示している）

$$\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}_i \cdot \begin{pmatrix} 10 \\ 20 \\ 30 \end{pmatrix}_i = \text{contractWith}(+, \begin{pmatrix} 10 & 20 & 30 \\ 20 & 40 & 60 \\ 30 & 60 & 90 \end{pmatrix}_i) = 10 + 40 + 90 = 140$$

$$\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}_i \cdot \begin{pmatrix} 10 \\ 20 \\ 30 \end{pmatrix}_i = \text{contractWith}(+, \begin{pmatrix} 10 \\ 40 \\ 90 \end{pmatrix}_i) = \begin{pmatrix} 10 \\ 40 \\ 90 \end{pmatrix}_i$$

$$\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}_i \cdot \begin{pmatrix} 10 \\ 20 \\ 30 \end{pmatrix}_j = \text{contractWith}(+, \begin{pmatrix} 10 & 20 & 30 \\ 20 & 40 & 60 \\ 30 & 60 & 90 \end{pmatrix}_{ij}) = \begin{pmatrix} 10 & 20 & 30 \\ 20 & 40 & 60 \\ 30 & 60 & 90 \end{pmatrix}_{ij}$$

(b) “.”関数の適用

図 11.3: テンソル関数の例として “.”関数の定義と適用例

```
[| 1, 2, 3 |]~i . [| 10, 20, 30 |]_i
-- 140
```

11.5 三種類の添字

テンソルの添字記法を扱うために Egison には三種類の添字が導入されている．最初の二つは前節で紹介した上添字と下添字で $\sim$ と $_$ で表現される．三つ目は上下添字と呼ばれる添字で $\sim_$ で表現される．上下添字は，同一のシンボルをもつ上添字と下添字をもつテンソルに対してスカラー関数を適用した際に現れる．

```
[| 1 2 3 |]~i * [| 10 20 30 |]_i -- [| 10, 40, 90 |]_i
[| 1 2 3 |]~i * [| 10 20 30 |]~i -- [| 10, 40, 90 |]~i
[| 1 2 3 |]~i * [| 10 20 30 |]_i -- [| 10, 40, 90 |]~i
```

前節でも登場した `contractWith`関数は上下添字について対角成分をリストに変換し第一引数の関数で量み込む．

```
contractWith (+) [| 10, 40, 90 |]_i -- [| 10, 40, 90 |]_i
contractWith (+) [| 10, 40, 90 |]~i -- [| 10, 40, 90 |]~i
contractWith (+) [| 10, 40, 90 |]~i -- 130
```

`contractWith`関数は組み込み関数 `contract`を使って定義されたライブラリ関数である．組み込み関数 `contract`は上下添字の付加されたスロットの成分のリストを返す．

```

E({A, xs}) =
  if e(xs) = [] then
    {A, xs}
  elseif e(xs) = [{k,j}, ...] & p(k,xs) = p(j,xs) then
    E({diag(k, j, A), remove(j, xs)})
  elseif e(xs) = [{k,j}, ...] & p(k,xs) != p(j,xs) then
    E({diag(k, j, A), update(k, 0, remove(j, xs))})

```

図 11.4: 添字簡約の擬似コード

```

contract [| 1, 2, 3 |]~_i -- [| 1, 2, 3 |]
contract [| [| 11, 12 |], [| 21, 22 |] |]~_i~_j -- [| [| 11, 12 |]~_j, [| 21, 22 |]~_j |]
contract [| [| 11, 12 |], [| 21, 22 |] |]~_i~_j -- [| [| 11, 21 |]~_j, [| 12, 22 |]~_i |]
contract [| [| 11, 12 |], [| 21, 22 |] |]~_i~_j -- [| 11, 12, 21, 22 |]

```

11.6 添字の簡約規則

本節は、添字の簡約ルールを擬似コードを使って説明する。11.4 節で説明したように、テンソルをスカラー関数を適用したときは常にテンソル積のかたちで成分ごとに処理がマップされるため、ひとつのテンソルについての簡約ルールのみを定義すればよい。

図 11.4 は添字簡約をする擬似コードである。E(A,xs)は添字付きテンソルの簡約をする関数である。Aはテンソルの成分からなる配列であり、xsはAに付加されている添字のリストを表す。たとえば、E(A,xs)は添字として“~i~j~i”を持つテンソルに対して以下のように動作する。上添字、下添字、上下添字を表すフラグとしてそれぞれ 1, -1, 0 を使っている。

```

E({[| [| 1,2 |], [| 3,4 |] |]
  , [| 5,6 |], [| 7,8 |] |]
  , [{i,1}, {j,-1}, {i,-1}]) =
{[| [| 1,3 |], [| 6,8 |] |], [{i,0}, {j,-1}]}

```

擬似コード中で使われている補助関数を説明する。e(xs)はxs中に現れる同一のシンボルからなる添字のペアを見つける関数である。diag(k, j, A)はk番目とj番目のスロットについてAの対角成分からなるテンソルを計算する関数である。remove(k, xs)はリストxsからk番目の要素を取りのぞく関数である。p(k, xs)は連想リストxsからk番目の値を取得する関数である。update(k, p, xs)は連想リストxsのk番目の値をpに更新する関数である。これらの補助関数は以下のように動作する。

```

e([{i, 1}, {j, -1}, {i, -1}]) = [{1,3}]
diag(1, 2, [| [| 11,12 |], [| 21,22 |] |]) = [| 11,22 |]

```

```
p(2, [{i, 1}, {j, -1}])      = -1
remove(2, [{i, 1}, {j, -1}]) = [{i, 1}]
update(2, 0, [{i, 1}, {j, -1}]) = [{i, 1}, {j, 0}]
```

11.7 テンソル関数の適用

添字付きのテンソルがテンソル関数に渡されるとき、そのテンソルの添字ごとテンソル関数に適用される。図 11.3a で定義した “.” 関数でこの挙動を確認する。図 11.3b では、“.” 関数を適用をしたときなされる展開が段階的に表示されている。contractWith の内側に添字付きのテンソルがあらわれるが、これはテンソルの添字ごとテンソル関数に渡されることで可能になっている。この挙動のおかげで以下のようにテンソル関数についてテンソルの添字記法を使って関数適用できるようになっている。

```
[|1,2,3|]~i . [|10,20,30|]_i -- 140
[|1,2,3|]_i . [|10,20,30|]_j -- [| [|10,20,30|], [|20,40,60|], [|30,60,90|] |]_i_j
[|1,2,3|]_i . [|10,20,30|]_i -- [|10,40,90|]_i
```

テンソル関数の中で引数として渡されたテンソルに追加で添字を付加したいことがある。追加で新しい添字を付加するためには、以下のように...をテンソルと添字の間に挟めば良い。この機能は、11.13 節で紹介する subrefs と suprefs という組み込み関数を使った糖衣構文として実装されている。この機能の使用例は第 13 章で紹介する。

```
A..._i_j
```

11.8 省略された添字の補完

添字が省略されたテンソルが関数に適用された場合、Egison は自動で添字を補完する。本節はこの補完の規則を説明する。

以下、A, B を 2 階のテンソル、つまり行列であるとする。添字を省略した状態でスカラー関数に A, B を同時に適用すると、下記のように同じ組み合わせの添字が補完される。

```
A + B -- A_t1_t2 + B_t1_t2
```

関数適用の前に “!” が付加されていた場合は、下記のように違う添字が補完される。

```
A !+ B -- A_t1_t2 + B_t3_t4
```

添字が省略されたテンソルが、テンソル関数に適用された場合は、補完はおこなわれない。

上記の添字の補完規則には実は必然性がある。添字が省略された場合の添字補完規則を上記のように設定すると、微分形式のための演算子を簡潔に定義できる。微分形式の演算子の定義の具体例は、第 13 章で紹介する。あとで第 13 章でも述べるように、“!” が使われる必要があるのは、

微分形式の演算子の定義の中だけになる．そのため，基本的にユーザーは“!”については意識しなくてよい．

11.9 tensorMap 自動挿入の仕組み

本節は，型システムによる tensorMap 自動挿入の仕組みについて説明する．

11.9.1 基本原理

Egison の型システムは，関数適用時に仮引数の型と実引数の型を比較し，必要に応じて自動的に tensorMap を挿入する．tensorMap は，第一引数の関数を第二引数のテンソルの成分ごとに適用してテンソルを返す組み込み関数である．

挿入の判断基準は以下の通りである：

- 仮引数の型が Tensor a 型と unify できない（例：Integer，a 型に Num a などの型クラス制約がある場合）
- 実引数の型が Tensor a 型である

この両方の条件を満たす場合，自動的に tensorMap が挿入される．

11.9.2 具体例

図 11.2 で定義した min 関数を使って挙動を確認する．

```
def min (x: a) (y: a) : a := if x < y then x else y

min [|1,2,3|]_i [|10,20,30|]_j
-- 内部的に以下のように変換される:
-- tensorMap (\xi -> tensorMap (\yi -> min xi yi) [|10,20,30|]_j) [|1,2,3|]_i
-- 結果: [| [|1,1,1|], [|2,2,2|], [|3,3,3|] |]_i_j

min [|1,2,3|]_i [|10,20,30|]_i
-- 結果: [|1,2,3|]_i
```

min 関数の仮引数の型は型変数 a であり，Tensor a 型と unify できない．そのため，テンソルが引数として渡されると，自動的に tensorMap が挿入される．

一つ目の式の結果のテンソルの添字が “_i_j” となっている．tensorMap は，第一引数の関数に第二引数のテンソルの成分を適用した結果がテンソルである場合，その結果のテンソルの添字を tensorMap 全体の結果のテンソルの添字の末端に移動する．このために結果のテンソルの添字が “_i_j” となる．この仕組みがテンソルの添字記法を使ってスカラー関数を適用することを可能にしている．

11.9.3 型クラス制約との関係

型クラス制約がある場合も同様に `tensorMap` が挿入される。

```
def double {Num a} (x: a) : a := x + x

double [|1,2,3|]_i
-- Tensor Integer は Num のインスタンスでないため、
-- 自動的に tensorMap が挿入される:
-- tensorMap (\xi -> double xi) [|1,2,3|]_i
-- 結果: [|2,4,6|]_i
```

`Num a` という型クラス制約により、型変数 `a` は `Num` クラスのインスタンスでなければならない。`Tensor Integer` 型は `Num` クラスのインスタンスではないため、`Tensor a` 型と `unify` できない。そのため、テンソルが引数として渡されると、自動的に `tensorMap` が挿入され、テンソルの各成分 (`Integer` 型) に対して `double` 関数が適用される。

11.9.4 テンソル関数の場合

一方、仮引数の型が明示的に `Tensor a` 型である場合、`tensorMap` は挿入されない。

```
def (.) (t1: Tensor a) (t2: Tensor a) : Tensor a :=
  contractWith (+) (t1 * t2)

[|1,2,3|]~i . [|10,20,30|]_i
-- tensorMap は挿入されず、テンソルがそのまま渡される
-- 結果: 140
```

11.9.5 効率化のための工夫

実際の実装では、効率化のために `tensorMap2` という組み込み関数も使われる。`tensorMap` をネストして使う方法では、対角化するために対角成分しか必要ない場合でも対角成分以外の成分ももつテンソルを生成してしまう。`tensorMap2` は 2 引数関数と二つのテンソルを引数にとり、結果のテンソルに必要な成分だけを計算するように実装されている。

11.10 添字の反転と `flipIndices` 関数

偏微分をするために使われる ∂/∂ 関数はスカラー関数である。しかし、 ∂/∂ 関数はただのスカラー関数ではない。 ∂/∂ 関数は第二引数のテンソルの添字の上下を反転する。たとえば、 $(\partial/\partial \Gamma^{i_j k} x^{l_1})$ は、添字として “ $\sim i_j k_l$ ” をもつ四階のテンソルを返す。

このような動作を実現するために、Egison には `flipIndices` という組み込み関数が用意されている。 `flipIndices` は、その引数のテンソルのすべての添字の上下を反転させる。上下添字は反転させても上下添字のままである。

∂/∂ 関数は、以下のように `flipIndices` を使って定義される。

```
def  $\partial/\partial$  (f: Tensor MathExpr) (x: Tensor MathExpr)
  : Tensor MathExpr :=
  tensorMap2 (\f x ->  $\partial/\partial$  f x) f (flipIndices x)

--  $\partial/\partial$  はスカラー値に対する偏微分を計算する補助関数
def  $\partial/\partial$  (f: MathExpr) (x: MathExpr) : MathExpr := ...
```

`tensorMap2` により、`f` と `flipIndices x` の各成分に対して ∂/∂ 関数が適用される。 `flipIndices x` により第二引数の添字が反転されるため、結果のテンソルの添字も反転する。

以下は、 ∂/∂ の使用例である。

```
 $\partial/\partial$  [| (r * (sin  $\theta$ )), (r * (cos  $\theta$ )) |] _i [| r,  $\theta$  |] _j
-- [| |(sin  $\theta$ ), (r * (cos  $\theta$ )) |], [| (cos  $\theta$ ), (-1 * r * (sin  $\theta$ )) |] _i ~ _j
 $\partial/\partial$  [| (r * (sin  $\theta$ )), (r * (cos  $\theta$ )) |] _i [| r,  $\theta$  |] _i
-- [| (sin  $\theta$ ), (-1 * r * (sin  $\theta$ )) |] ~ _i
```

一つ目の例では、第二引数の添字が `_j` から `~j` に反転され、結果の添字が `_i~j` となる。

11.11 generateTensor 式によるテンソルの生成

`generateTensor` 式はテンソルの初期化をするために便利な組み込み構文である。 `generateTensor` 式は第一引数にリストを引数にとる関数、第二引数に生成したいテンソルのサイズをとる。たとえば、すべての成分が 1 であるサイズが (3, 4) の行列を生成するには以下のように `generateTensor` 式を書く。

```
generateTensor [2]#1 [3, 4]
-- [| [| 1, 1, 1, 1 |], [| 1, 1, 1, 1 |], [| 1, 1, 1, 1 |] |]
```

無名 `match` 関数と組み合わせると短い記述でいろいろなテンソルを生成できる。たとえば単位行列を生成するには以下のようにすればよい。

```
generateTensor (\match as list integer with
  | [$n, #n] -> 1
  | _ -> 0)
[3, 3]
-- [| [| 1, 0, 0 |], [| 0, 1, 0 |], [| 0, 0, 1 |] |]
```

11.12 テンソルの宣言

テンソルを変数に束縛する方法を解説する．まずテンソルは通常の値と同様に `def` 式や `let` 式で定義することができる．

```
def A : Tensor Integer := [| [| 11, 12 |], [| 21, 22 |] |]  
  
A -- [| [| 11, 12 |], [| 21, 22 |] |]
```

数学や物理では同じ変数名で添字の上下の型が違うテンソルにそれぞれ別の値を入れることがよくある．Egison でもその慣習に対応するために、添字の上下の型ごとに違うテンソルを定義できるようにしている．Egison は、変数を参照するとき、変数名と添字の型の両方を使う．変数定義のときに、変数名の後ろに添字の型を続けて記述することによって添字の型を指定する．

```
def g__ : Tensor MathExpr := [| [| 2, 2 |], [| 2, 2 |] |]  
def g~~ : Tensor MathExpr := [| [| 1 / 2, 1 / 2 |], [| 1 / 2, 1 / 2 |] |]  
  
g_1_1 -- 2  
g~1~1 -- 1 / 2
```

添字の型を指定してテンソルが束縛されている変数に参照するには、**ダミーシンボル** (*dummy symbol*) を使うのが便利である．ダミーシンボルは、`#` で表現され、すべてユニークなシンボルとして扱われる．ダミーシンボルは Egison に組み込みで実装されている機構である．ダミーシンボルを使うと、上記で定義した `g~~` と `g__` を以下のように参照できる．

```
g_#_# -- [| [| 2, 2 |], [| 2, 2 |] |]_#_#  
g~#~# -- [| [| 1 / 2, 1 / 2 |], [| 1 / 2, 1 / 2 |] |]~#~#
```

シンボルを使っても添字の型を指定してテンソルを参照することもできるが、ダミーシンボルを使う方が定義済みの変数をシンボルとして使おうとする間違いが起きにくい．

```
g_i_j -- [| [| 2, 2 |], [| 2, 2 |] |]_i_j  
g~i~j -- [| [| 1 / 2, 1 / 2 |], [| 1 / 2, 1 / 2 |] |]_i_j
```

ダミーシンボルはテンソルの添字として使う以外の用途はないように考えているが、ふつうのシンボルのように扱うことができる．以下は二つのダミーシンボルを足し合わせている．二つのダミーシンボルを足し合わせると、以下のようにまとめられないが、これはこの二つのダミーシンボルがそれぞれ異なるシンボルとして扱われるからである．

```
# + # -- # + #
```

数式でテンソルを定義するときのように、左辺の変数の添字としてシンボルを書くこともできる．


```
def A_i_j : Tensor Integer := [| [| 11, 12 |], [| 21, 22 |] |]_i_j
def B_i_j : Tensor Integer := [| [| 11, 12 |], [| 21, 22 |] |]_j_i

A_#_# -- [| [| 11, 12 |], [| 21, 22 |] |]_#_#
B_#_# -- [| [| 11, 21 |], [| 12, 22 |] |]_#_#
```

この機能は以下のような糖衣構文として Egison では実装されている.

```
def A_i_j := ...
-- => def A__ := withSymbols [i, j]
--      transpose [i, j] ...
```

transpose関数は、テンソルの成分を第一引数で指定された順番に転置する組み込み関数である.

同じ変数名と添字の型のテンソルが環境にない場合は、添字の型を末尾から順に取り除いて参照をおこなう. たとえば、下記の例のように $T\sim$ と $T_$ が定義されている状態で、 $T\sim\#_$ を参照すると、 $T\sim_$ が環境にないため、末尾の添字の型をひとつ取り除いて $T\sim$ を参照する. $T\sim$ は定義済みであるため、その値が参照される.

```
def T~ : Tensor Integer := [| [| 11, 12 |], [| 21, 22 |] |]
def T_ : Tensor Integer := [| [| -11, -12 |], [| -21, -22 |] |]

T~#_# -- [| [| 11, 12 |], [| 21, 22 |] |]
```

11.13 添字付加のために便利な構文

階数がパラメーター n によって変わるテンソル A について、 $A_{i_1\dots i_n}$ のように添字を付加したいことがある. このために `subrefs` と `subrefs!` という組み込み関数が Egison には用意されている. これらの関数は第一引数にテンソルに第二引数に添字のリストを付加する. `subrefs` は第一引数のテンソルの既存の添字に追加で第二引数に添字のリストを付加するのに対して、`subrefs!` は既存の添字を消して新たに添字を付加し直す.

```
subrefs! A (map 1#i_%1 [1..n])
subrefs A (map 1#i_%1 [1..n])
```

$A_{i_1\dots i_n}$ のような形のテンソルは数式でよく現れるため、上記の `subrefs!` と `subrefs` を使ったプログラムについては以下のような糖衣構文が用意されている.

```
A_(i_1)...(i_n)
A..._(i_1)...(i_n)
```

上記の内容は $A^{i_1\dots i_n}$ のような上添字をもつテンソルにも用意されており、このために `suprefs` と `suprefs!` という組み込み関数が Egison には用意されている.

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{12} & a_{22} & a_{23} \\ a_{13} & a_{23} & a_{33} \end{pmatrix}$$

(a) 対称なテンソルの例

図 11.5: テンソルの対称性と歪対称性

subrefsや suprefsのように既存の添字に追加で添字を付加する機能は、第 13 章で紹介する微分形式についての関数を定義するときに使われる。

11.14 テンソルの対称性・歪対称性の宣言

テンソルが**対称**であるとは、図 11.5a の例のように、上三角の成分と下三角の成分が一致することをいう。テンソルが**歪対称**であるとは、図 11.5b の例のように、上三角の成分と下三角の成分の符号が反転することをいう。テンソルが歪対称であるとき、対角成分は 0 となる。

数学や物理で現れるテンソルの多くが対称性をもつ。たとえば、13.2 節で計算するリーマン曲率テンソル R_{abcd} は $R_{abcd} = -R_{abdc}$, $R_{abcd} = -R_{bacd}$, $R_{abcd} = R_{cdab}$ という複数の対称性をもつ。数学者や物理学者はこれらの対称性を意識しながら適宜計算を省略する。Egison でも対称性を意識した計算の最適化を実装したいが、プログラミング言語でこの最適化をするには、明示的にテンソルの対称性を宣言する必要がある。Egison はテンソルの対称性を宣言するための記法が実装されている。本節は、この記法の使い方と実装を解説する。

11.14.1 テンソルの対称性を宣言する記法

リーマン曲率テンソルには、上記の対称性に加えて第一ビアンキ恒等式 $R_{abcd} + R_{acdb} + R_{adb c} = 0$ とよばれる対称性もある。この第一ビアンキ恒等式の表現として、 $R_{a[bcd]} = 0$ という表記がある。Egison に実装されているテンソルの対称性を宣言する記法は、この表記法に触発されて得られたものである。

具体例をみせながら、その記法を紹介する。Egison でリーマン曲率テンソル R_{abcd} は下記のように定義できる。右辺に複雑な式があるが、本節では無視していただいて問題ない。左辺だけに注目してほしい。

```
def R_a_b_c_d : Tensor MathExpr := withSymbols [i, j]
  g_a_i . (∂/∂ Γ~i_b_d x~c - ∂/∂ Γ~i_b_c x~d +
    Γ~j_b_d . Γ~i_j_c - Γ~j_b_c . Γ~i_j_d)
```

$R_{abcd} = -R_{abdc}$ という歪対称性を宣言するには、以下のように添字_a_bを波括弧{}で囲む。

```
def R{ _a_b }_c_d : Tensor MathExpr := withSymbols [i, j]
```

```
g_a_i . (∂/∂ Γ~i_b_d x~c - ∂/∂ Γ~i_b_c x~d +
        Γ~j_b_d . Γ~i_j_c - Γ~j_b_c . Γ~i_j_d)
```

上記のように R_{abcd} を定義すると、たとえば R_{21cd} を参照したとき、 R_{12cd} の計算結果の符号を反転した値を返すようになる。対称性を複数同時に宣言することもできる。 $R_{abcd} = -R_{abdc}$ と $R_{abcd} = -R_{bacd}$ を同時に宣言するには、以下のように添字_a_bと_c_dの両方を波括弧{}で囲む。

```
def R[{_a_b}{_c_d}] : Tensor MathExpr := withSymbols [i, j]
  g_a_i . (∂/∂ Γ~i_b_d x~c - ∂/∂ Γ~i_b_c x~d +
        Γ~j_b_d . Γ~i_j_c - Γ~j_b_c . Γ~i_j_d)
```

こうすると、たとえば R_{2121} を参照したとき、 R_{1212} の計算結果を返すようになる。

$R_{abcd} = R_{cdab}$ のような複雑な対称性も記述できる。この対称性を記述するためには添字_a_bと_c_dをそれぞれひとまとまりで扱うために丸括弧()で囲む。そして、これらを角括弧[]で囲む。波括弧{}が歪対称性を宣言するために使われるのに対し、角括弧[]は対称性を宣言するために使われる。

```
def R[({_a_b})(_c_d)] : Tensor MathExpr := withSymbols [i, j]
  g_a_i . (∂/∂ Γ~i_b_d x~c - ∂/∂ Γ~i_b_c x~d +
        Γ~j_b_d . Γ~i_j_c - Γ~j_b_c . Γ~i_j_d)
```

三つの対称性 $R_{abcd} = -R_{abdc}$, $R_{abcd} = -R_{bacd}$, $R_{abcd} = R_{cdab}$ を同時に宣言することもできる。

```
def R[{{_a_b}}{_c_d}] : Tensor MathExpr := withSymbols [i, j]
  g_a_i . (∂/∂ Γ~i_b_d x~c - ∂/∂ Γ~i_b_c x~d +
        Γ~j_b_d . Γ~i_j_c - Γ~j_b_c . Γ~i_j_d)
```

まとめると、Egison は対称性の宣言には三種類の括弧を使い分ける。角括弧[]は対称性を宣言するために使う。波括弧{}は歪対称性を宣言するために使う。丸括弧()は複数の添字をひとまとまりにするために使う。

対称なテンソルの添字は隣り合っていることが多いため、数学や物理で現れるテンソルの対称性は、この記法でほぼ宣言することができる。:=の右辺のプログラムを全く変更せずに対称性を宣言できるのはこの記法のよいところである。

11.14.2 対称性の宣言によってテンソルの定義が短くなる例

さきほど、この手法の利点として、:=の右辺のプログラムを全く変更する必要がないと述べたが、:=の右辺のプログラムが簡略化できるケースもある。たとえば、下記のようにテンソルの各ブロックごとに異なる式を使って値が定義されるテンソルがそのようなケースに該当する。

```
def R'_i_j_k_l : Tensor MathExpr :=
  generateTensor
```

```

(\match as list integer with
| [#1, #1, _, _] -> 0
| [_, _, #1, #1] -> 0
| [#1, $b, #1, $d] -> -1 * p^2 * δ~(b - 1)_(d - 1)
| [$a, #1, #1, $d] -> p^2 * δ~(a - 1)_(d - 1)
| [#1, $b, $c, #1] -> p^2 * g_(b - 1)_(c - 1)
| [$a, #1, $c, #1] -> -1 * p^2 * g_(a - 1)_(c - 1)
| [#1, $b, $c, $d] -> -1 * p * ∇J_(b - 1)_(c - 1)~(d - 1)
| [$a, #1, $c, $d] -> p * ∇J_(a - 1)_(c - 1)~(d - 1)
| [$a, $b, #1, $d] -> -1 * p * ∇J~(d - 1)_(a - 1)_(b - 1)
| [$a, $b, $c, #1] -> p * ∇J_(c - 1)_(a - 1)_(b - 1)
| [$a, $b, $c, $d] -> R_(a - 1)_(b - 1)_(c - 1)~(d - 1)
                        + -1 * p^2 * J_(b - 1)_(c - 1) * J_(a - 1)~(d - 1)
                        + p^2 * J_(a - 1)_(c - 1) * J_(b - 1)~(d - 1)
                        + 2 * p^2 * J_(a - 1)_(b - 1) * J_(c - 1)~(d - 1))

[5, 5, 5, 5]

```

対称性により条件分岐を大きく減らすことができる。

```

def R'[_i_j][_k_l] : Tensor MathExpr :=
  generateTensor
    (\match as list integer with
    | [#1, #1, _, _] -> 0
    | [#1, $b, #1, $d] -> -1 * p^2 * δ~(b - 1)_(d - 1)
    | [#1, $b, $c, $d] -> -1 * p * ∇J_(b - 1)_(c - 1)~(d - 1)
    | [$a, $b, $c, $d] -> R_(a - 1)_(b - 1)_(c - 1)~(d - 1)
                        + -1 * p^2 * J_(b - 1)_(c - 1) * J_(a - 1)~(d - 1)
                        + p^2 * J_(a - 1)_(c - 1) * J_(b - 1)~(d - 1)
                        + 2 * p^2 * J_(a - 1)_(b - 1) * J_(c - 1)~(d - 1))

[5, 5, 5, 5]

```

11.14.3 実装

テンソルの対称性を宣言する記法の実装方法について解説する。この記法は `generateTensor` 式を使う糖衣構文として実装されている。本節では、この変換を具体例をいくつか紹介することによって説明する。

まず対称性を宣言する角括弧 `[]` がどう展開されるか説明する。以下のプログラムの...の箇所にはなにか適当なテンソルを返すプログラムがはいる。

```
def X[_i_j] := ...
```

上記の形のプログラムは、以下のようなプログラムに展開される。

```
def X_i_j : Tensor a :=
```

```
let tmpX_i_j := ...
generateTensor
  (\i j -> if i > j then tmpX_j_i
           else          tmpX_i_j)
(tensorShape tmpX_#_#)
```

tmpXという新たな変数を導入し、定義したいテンソルを代入する。X自体は generateTensor式を使って定義する。この generateTensor式は、下半分の成分については、上半分の成分を参照するように定義されている。Egison はテンソルの成分の評価を遅延するため、tmpXの下半分の成分は計算されることはない。そのため、テンソル成分の計算コストが半分になる。

歪対称性を宣言する波括弧{}も同じように展開される。

```
def X_{i_j} := ...
```

下半分の成分の符号が上半分の成分に対して反転していることと、対角成分が 0 になるように generateTensor式の内部が変更されている。

```
def X_i_j : Tensor a :=
  let tmpX_i_j := ...
  generateTensor
    (\i j -> if i > j      then -tmpX_j_i
              else if i = j then 0
              else          tmpX_i_j)
  (tensorShape tmpX_#_#)
```

11.15 tensorMap 挿入の詳細な仕組み

Egison では、多相関数がテンソル引数に対して正しく動作するように、コンパイラが自動的に tensorMapや tensorMap2を挿入する。本節では、この自動挿入の仕組みの詳細を解説する。

11.15.1 処理フローとタイミング

tensorMapの挿入は、型推論 (Phase 5-6) の後、型クラス展開 (Phase 8-2) の前に実行される (Phase 8-1)。処理の流れは以下のとおりである：

1. Phase 5-6: 型推論により、すべての式に型が付与される。
2. Phase 8-1: TensorMap 挿入により、必要な箇所に tensorMap/tensorMap2が自動挿入される。
3. Phase 8-2: 型クラス展開により、型クラスメソッド（たとえば (+)）が辞書引きの形式 (dict_Num_("plus")) に変換される。

TensorMap 挿入を型クラス展開の前に実行する理由は、引数の型（スカラーかテンソルか）を先

に確定させることで、型クラス展開時にインスタンス選択が正しく行えるようにするためである。

11.15.2 挿入の基本原則

TensorMap 挿入は以下の基本原則に基づいて行われる：

原則 1: 型コンストラクタ内のスカラー型に対する演算への挿入

型コンストラクタ（リスト `[a]`、タプル `(a, b)` など）の内側にスカラー型または型クラス制約付き型変数がある場合、その型に対する演算には常に `tensorMap` または `tensorMap2` を挿入する。

たとえば、`xs: [a]` という型をもつ変数 `xs` から取り出された要素 `x` に対して、スカラー型を期待する演算 `(+)` を適用する場合、`tensorMap2 (+)` という形式に変換される。

原則 2: スカラーに対する `tensorMap` の動作

`tensorMap` と `tensorMap2` は、スカラー値に対してもそのまま適用できるように実装されている：

- `tensorMap f scalar` は `f scalar` を返す（テンソルでない場合は恒等的に動作）
- `tensorMap2 f s1 s2` は `f s1 s2` を返す

この性質により、実行時に引数がスカラーかテンソルかを判定する条件分岐を挿入する必要がなくなり、実装が大幅に簡略化される。

11.15.3 高階関数の場合の挿入

高階関数に関数を引数として渡す場合、その関数が二項関数であり、かつ両方のパラメータがスカラー型である場合、自動的に `tensorMap2` でラップされる。

二項関数の検出

関数の型が `a -> b -> c` の形式（ただし `c` は関数型でない）であり、`a` と `b` がともにスカラー型である場合、その関数は `tensorMap2` でラップすべき二項関数として検出される。

スカラー型の判定は、型が `Tensor a` と単一化（unification）できないかどうかで行われる：

- 型クラス制約付き型変数（たとえば `{Num a} a`）は、`Tensor` 型が `Num` のインスタンスをもたないため、`Tensor a` と単一化できず、スカラー型と判定される。
- 具象型（`Integer`、`Float` など）も `Tensor a` と単一化できず、スカラー型と判定される。
- `Tensor Integer` のようなテンソル型は `Tensor a` と単一化できるため、スカラー型とは判定されない。

tensorMap2 によるラッピング

二項関数 $f : a \rightarrow b \rightarrow c$ は以下のように変換される：

```
f ---> \x y -> tensorMap2 f x y
```

ここで、ラムダ式の引数 x と y はテンソル型 (Tensor a と Tensor b) をもち、結果もテンソル型 (Tensor c) となる。

イータ展開されたラムダの処理

型クラスメソッドは、TensorMap 挿入の段階ではまだイータ展開された形式で現れる。たとえば、 $(+)$ 演算子は以下のような形式になっている：

```
\etaVar1 etaVar2 -> dict_Num_("plus") etaVar1 etaVar2
```

この 2 引数ラムダ式の本体が 2 引数関数適用である場合、本体全体を tensorMap2 でラップする：

```
\etaVar1 etaVar2 -> tensorMap2 (\x y -> dict_Num_("plus") x y) etaVar1 etaVar2
```

この変換により、型クラスメソッドに対しても tensorMap 挿入が正しく適用される。

11.15.4 具体例: sum 関数

多相的な sum 関数の変換過程を追うことで、TensorMap 挿入の仕組みをより具体的に理解できる。

元の定義

```
def sum {Num a} (xs: [a]) : a := foldl1 (+) xs
```

この関数は、型クラス制約 $\{Num\ a\}$ をもつ多相的な型 $[a] \rightarrow a$ をもつ。

Phase 5-6: 型推論後

型推論により、型変数 a が具体的な型 (たとえば t_0) にインスタンス化される：

```
def sum : {Num t0} [t0] -> t0 :=  
  \xs ->  
    foldl1 ((+) : {Num t0} t0 -> t0 -> t0) (xs : [t0])
```

ここで、 xs の型は $[t_0]$ であり、型コンストラクタ $[]$ の内側に型クラス制約付き型変数 t_0 が存在する。また、 $(+)$ の型は $\{Num\ t_0\} \ t_0 \rightarrow t_0 \rightarrow t_0$ であり、スカラー型を引数にとる二項関数である。

Phase 8-1: TensorMap 挿入後

(+)は二項関数であり、両方のパラメータがスカラー型 (t0) であるため、tensorMap2でラップされる：

```
def sum : {Num t0} [t0] -> t0 :=
  \xs ->
    foldl1 (tensorMap2 (+)) xs
```

この時点で、xsの要素がテンソルである場合でも、tensorMap2により要素ごとの加算が正しく実行されるようになる。

Phase 8-2: 型クラス展開後

型クラスメソッド (+)が辞書引きの形式に展開される：

```
def sum : {Num t0} [t0] -> t0 :=
  \dict_Num xs ->
    foldl1 (tensorMap2 (dict_Num("plus"))) xs
```

ここで、dict_Numは Num t0 インスタンスのメソッド辞書である。

実行時の動作

この変換により、sum関数はスカラーとテンソルの両方に対して正しく動作する：

■スカラーの場合 (t0 = Integer):

```
sum [1, 2, 3]
-- foldl1 (tensorMap2 (+)) [1, 2, 3]
-- tensorMap2 (+) 1 2 --> (+) 1 2 --> 3
-- tensorMap2 (+) 3 3 --> (+) 3 3 --> 6
-- 結果: 6
```

スカラーに対しては、tensorMap2が恒等的に動作するため、通常の加算が行われる。

■テンソルの場合 (t0 = Tensor Integer):

```
sum [[1, 2], [3, 4]]
-- foldl1 (tensorMap2 (+)) [[1, 2], [3, 4]]
-- tensorMap2 (+) [1, 2] [3, 4] --> [1+3, 2+4] --> [4, 6]
-- 結果: [4, 6]
```

テンソルに対しては、tensorMap2が要素ごとに加算を適用する。

■混合ケース:

```
sum [[1, 2], 3]
-- tensorMap2 (+) [1, 2] 3 --> tensorMap (\x -> x + 3) [1, 2]
-- --> [1+3, 2+3] --> [4, 5]
-- 結果: [4, 5]
```


一方がテンソル，もう一方がスカラーの場合でも，`tensorMap2`が適切に処理する．

11.15.5 まとめ

TensorMap 挿入の仕組みにより，Egison では以下が実現されている：

- ユーザーは型クラス制約を使って多相的な関数を定義するだけで，テンソル演算が自動的にサポートされる．
- コンパイラが型情報をもとに `tensorMap`/`tensorMap2`を自動挿入するため，ユーザーが明示的に記述する必要がない．
- 実行時の型に応じて，スカラー演算とテンソル演算が自動的に切り替わる．
- 高階関数（`foldl1`，`map`など）に渡される関数引数に対しても，自動的に適切な変換が適用される．

この設計により，Egison の数式処理システムは，スカラーとテンソルを統一的に扱うことができ，かつユーザーにとって直感的で使いやすいインターフェースを提供している．

第 12 章

関数シンボル

名前が f , 引数が x, y と決まっているが, 中身は未定義の関数 $f(x, y)$ は, 紙の上では f と引数を省略して表記されることが多い. このシンボリックな関数の引数を省略する表記をプログラミングにも導入するための機能が関数シンボル (*function symbol*) である. 関数シンボル自体の動機はシンプルであるが, 関数シンボルは Egison 特有の言語機能であるため, 一つの独立した章を使ってその実装方法も含め紹介する.

12.1 関数シンボルの基本的な使い方

関数シンボルが特に力を発揮するのは, 微分を扱う計算のときである. 以下では x, y を引数にとる関数シンボル f を定義している. 関数シンボルは `MathExpr` 型をもつ数式として扱われる.

```
declare symbol x, y : MathExpr
```

```
def f : MathExpr := function (x, y)
```

関数シンボルを `show` で表示すると, 引数の値が常に表示される.

```
show f -- "f x y"
```

以下のように, 関数シンボル f を x や y について微分できる. 偏微分の結果の関数は, 引数の番号を `|` 記号で付加した表示になる. $f|1$ は「 f の第 1 引数に関する偏微分」, $f|1|2$ は「第 1 引数で偏微分した後さらに第 2 引数で偏微分した結果」を表す. “`|`” は, “`_`” や “`~`” と同様 Egison 組み込みの記号である. 以下に関数シンボルの微分を標準的な Egison 出力と LaTeX 形式の出力の両方の表示例を示す. “ ∂/∂ ” は Egison で定義されているライブラリ関数である.

```
show ( $\partial/\partial$  f x) -- "f|1 x y"
```

```
show ( $\partial/\partial$  ( $\partial/\partial$  f x) y) -- "f|1|2 x y"
```

f は x と y についての関数であるので, 以下のように z で微分すると 0 になる.

```
show ( $\partial/\partial$  f z) -- "0"
```

物理の計算では、 t, x, y, z の4つの引数をとる関数がよく登場する。もし関数シンボルのような仕組みがないと、式が長くなり、読み難くなるため、関数シンボルの仕組みはプログラムの読み書きしやすくする。

12.2 引数への値の代入

関数シンボルには、通常の関数適用と同じ構文で引数を代入できる。 `f 0 1`と書くと、`f`の第1引数に `0`、第2引数に `1`を代入した数式が得られる。引数の個数が合わない場合はエラーになる。

```
show (f 0 1) -- "f 0 1"
show (f x 0) -- "f x 0"
```

引数を代入した結果も `MathExpr`型の数式であるため、さらに微分できる。引数に具体的な数値が入った場合、その引数に対する微分は `0`になる。

```
∂/∂ (f 0 1) x -- 0
```

`V.substitute`関数を使っても引数を置き換えられる。

```
V.substitute [|x, y|] [|0, 0|] f
-- f 0 0
```

12.3 合成関数の微分（連鎖律）

合成関数の微分にも対応していることが、Egison の関数シンボルの重要な特徴である。 `function` 式の仮引数に変数名ではなく式をとるのは、合成関数の微分に対応するためである。さきほどと同様に g を、 $g(x, y)$ の省略であるとする。また、 $x = r \cos \theta, y = r \sin \theta$ という関係式もあるとする。このとき、 g を r で微分すると、連鎖律により $\frac{\partial g}{\partial x_1} \cos \theta + \frac{\partial g}{\partial x_2} \sin \theta$ という結果を返している。

```
declare symbol r, θ : MathExpr

def x : MathExpr := r * cos θ
def y : MathExpr := r * sin θ
def g : MathExpr := function (x, y)

show g
-- "g (r * 'cos θ) (r * 'sin θ)"

show (∂/∂ g r)
-- "g|1 (r * 'cos θ) (r * 'sin θ) * 'cos θ + g|2 (r * 'cos θ) (r * 'sin θ) * 'sin θ"
```

偏微分の結果に含まれる `g|1`、`g|2` は、それぞれ $\frac{\partial g}{\partial x_1}$ 、 $\frac{\partial g}{\partial x_2}$ を表す記号である（引数の番号を | 記号で付加する）。この表示には引数の値が常に含まれるため、どの点での偏微分係数であるかが明示される。

12.4 関数シンボルを成分にもつテンソル

数学や物理では、テンソルの成分として関数が出てくることがある。たとえば、ベクトル場やテンソル場を表現するときにそのような場面がある。このような場面に対応するため、テンソルの成分として関数シンボルを使うことを Egison は許している。以下の例のようにテンソルの成分が関数シンボルであった場合、テンソルを束縛している変数名に成分の位置の添字を付加した名前をもつ関数シンボルが生成される。

```
declare symbol x, y, z : MathExpr

def g_i_j : Tensor MathExpr := generateTensor
  (\match as list integer with
    | [$n, #n] -> function (x, y, z)
      | _ -> 0)
  [3, 3]

show (withSymbols [i, j] g_i_j)
-- "[ [ [ g_1_1 x y z, 0, 0 ], [ 0, g_2_2 x y z, 0 ], [ 0, 0, g_3_3 x y z ] ] ]"

show (withSymbols [i, j] ( $\partial/\partial$  g_i_j x))
-- "[ [ [ g_1_1|1 x y z, 0, 0 ], [ 0, g_2_2|1 x y z, 0 ], [ 0, 0, g_3_3|1 x y z ] ] ]"
```

12.5 関数シンボルの内部表現とパターンマッチ

関数シンボルは Function データコンストラクタを使って Egison 内部では表現されている。Function データコンストラクタは二つの引数をとる。第一引数には関数シンボルの名前 (ScalarData 型) がはいる。第二引数には関数シンボルの引数の値のリスト ([ScalarData] 型) がはいる。

```
declare symbol x, y : MathExpr
def f : MathExpr := function (x, y)

fromMathExpr f
-- Div (Plus [Term 1 [(Function f [x, y], 1)]] (Plus [Term 1 []]))
```

func パターンコンストラクタを使って関数シンボルにパターンマッチできる。このパターンは関数名と引数の値のリストを抽出する。

```
match f as mathExpr with
| func $name $args -> (name, args)
-- (f, [x, y])
```

連鎖律の実装では、引数の数を利用して各引数に対する偏微分係数を整数インデックスで取り出している。

```
def ∂/∂ $f *$x :=
  match f as mathExpr with
  ...
  | func _ $args ->
    sum (map2 (\s r -> (userRefs f [s]) * ∂/∂ r x)
              (between 1 (length args))
              args)
  ...
```

ここで `between 1 (length args)` は `[1, 2, ..., n]` という整数のリストを生成する。 `userRefs f [1]` は `f` の名前シンボルに整数インデックス `1` を付加した数式 `f|1` を返す。

12.6 関数シンボルの実装方法

関数シンボルの実装で注意すべきことは、関数シンボルは自身の名前を保持することである。そのために Egison インタプリタは現在定義しようとしている変数の名前を管理している。また、[12.4 節](#)で解説したテンソルの成分に現れる関数シンボルをサポートするためには、現在テンソルのどの位置の成分を定義しているのか管理する必要がある。そのために Egison インタプリタは現在定義しようとしているテンソルの成分の位置も管理している。これらは Egison インタプリタの実装の `eval` 関数の追加引数として管理されている。

第 13 章

Egison で計算する微分幾何

本章は、数式処理システムとしての Egison の実践的な応用例として微分幾何への応用を紹介する。Egison を使うと、さまざまな多様体の曲率が定義の数式に近いプログラムを記述するだけで計算できたり、外微分やホッジ作用素のような微分形式のための作用素もプログラムで簡潔に定義できる。本章の解説は読者に微分幾何の知識を仮定している。微分幾何に興味を持つ読者には、小林昭七による『曲線と曲面の微分幾何』（裳華房）がおすすめである。

13.1 微分幾何と Egison

微分幾何は手計算がとくに大変な分野である。例えば、曲率が自明ではないもっとも単純な多様体は球面 S^2 であるが、そのリーマン曲率テンソルを手計算すると、初めての場合、30 分くらいかかる。続けて、その次に単純な多様体であるトーラス T^2 のリーマン曲率テンソルを計算すると、またしても 30 分以上はかかる。実際の研究のためには、より高次元の複雑な多様体の曲率を計算することが多い。このような場合、計算に一ヶ月以上かかることもある。そのため、微分幾何は数式処理システムがもっとも応用しやすい分野でもある。

数式処理システムを使えば、 S^2 のリーマン曲率テンソルは、球面座標系、リーマン計量、クリストッフェル記号、リーマン曲率テンソルの定義をプログラムとして記述すれば、数秒で計算できる。 T^2 については、 S^2 のリーマン曲率テンソルを計算をするプログラムを書いた後であれば、座標系の設定以外の部分は使いまわせるため、数分の作業で計算できる。より高次元の複雑な多様体についても、計量の設定を変更するだけで、計算することができる。

Egison は、9.5 節で解説した微分演算子、第 11 章で解説したテンソルのための言語機能、第 12 章で解説した関数シンボルを組み合わせ、従来の数式処理システムよりも数学の記法に近い形で微分幾何の計算をプログラムとして記述できる。また、11.8 節で触れたように、外微分やホッジ作用素のような微分形式のための作用素もプログラムで簡潔に定義するための仕組みも提供している。本章では、これらの機能を実演し、微分幾何の研究・教育に Egison が役に立つことを紹介する。

13.2 リーマン曲率テンソルの計算

本節は、球面について、リーマン計量や、クリストッフェル記号、リーマン曲率テンソル、リッチ曲率、スカラー曲率といった様々な微分幾何的な量を Egison を使って計算するプログラムを解説する。図 13.1 は、Jupyter Notebook 上で球面のリーマン曲率テンソルを計算する Egison プログラムとその実行結果の出力である。さまざまな幾何学的不変量は、曲率から計算される。本節で解説するプログラムの計量の部分だけを変えれば、あらゆる多様体の曲率を計算できる。そのため、本節のプログラムを理解すれば、実際の微分幾何の研究にも Egison が使える。本節は、このプログラムの各箇所について順番に説明していくことによって進める。

13.2.1 球座標系 - [1]-[2]

球座標系は、 r と θ 、 ϕ の 3 つのパラメーターからなる。 r は原点からの距離をあらわす。 θ は z 軸との角度、緯度をあらわす。 ϕ は xy 平面における x 軸からの回転量、経度をあらわす。球座標系の r の値を固定すれば、球面の座標系を得ることができる。球座標系とデカルト座標系は、 $x = r \cos \theta \sin \phi$, $y = r \cos \theta \cos \phi$, $z = r \sin \theta$ という関係式で結ばれている。以上が、図 13.1 の [1] と [2] で表現されている内容である。

13.2.2 接ベクトルとリーマン計量 - [3]-[7]

球面上の θ 、 ϕ によって定まる各点の上の接空間の基底となるベクトルをまず計算している。[3] でそのようなベクトルを計算し、[4] でそのベクトルを表示している。[4] の出力の 1 行目と 2 行目が基底となるベクトルの成分である。

リーマン計量は、各点における接ベクトルの大きさや向きの情報をもつ行列として表現できる。[5] でベクトルの内積を計算することによって基底ベクトルの大きさが計算されている。[6] では、[5] で定義したリーマン計量の値を表示している。球座標系においては、座標から自然に定まる基底の大きさが、各点で異なる。たとえば、経度 (ϕ) 方向の基底ベクトルの大きさが、緯度 (θ) によって変わる。この情報は、 g_{ij} の 2, 2 成分の $r^2 \sin^2 \theta$ にあらわれている。

[7] では、逆行列を計算する関数 `M.inverse` を使って、上添字がつくリーマン計量を求めている。リーマン計量には下添字がつく g_{ij} と上添字がつく g^{ij} の 2 種類がある。

13.2.3 クリストッフェル記号とリーマン曲率テンソル - [8]-[12]

[8] と [9] は、第一種クリストッフェル記号と第二種クリストッフェル記号の数式による定義をプログラムとして記述したものである。数式に非常に近いかたちでプログラムとして表現されている。直感的には、クリストッフェル記号 Γ^i_{jk} は、基底ベクトル e_j を e_k 方向に動かしたときの e_i 方向への変化の具合を表現する。

```

In [1]: def x := [ | 0, ϕ | ]

In [2]: def X := [ | r * sin θ * cos ϕ, r * sin θ * sin ϕ, r * cos θ | ]

In [3]: def e_i := ∂/∂ X x_i

In [4]: e_#
      ( rcos(θ)cos(ϕ)  rcos(θ)sin(ϕ)  -rsin(θ) )
      ( -rsin(θ)sin(ϕ)  rsin(θ)cos(ϕ)    0 )_#

In [5]: def g_i_j := generateTensor (\[a, b] -> e_a_i . e_b_i) [2, 2]

In [6]: def g~i~j := M.inverse g_#_#

In [7]: (g_#_#, g~#~#)
      ( ( r^2      0 ) , ( 1/r^2      0 ) )
      ( 0  r^2sin(θ)^2 )_## ( 0  1/r^2sin(θ)^2 )_##

In [8]: def Γ_i_j_k := (1 / 2) * (∂/∂ g_i_k x~j + ∂/∂ g_i_j x~k - ∂/∂ g_j_k x~i)

In [9]: def Γ~i~j_k := withSymbols [m] g~i~m . Γ_m_j_k

In [10]: (Γ_1_#_#, Γ_2_#_#)
      ( ( 0      0 ) , ( 0      r^2sin(θ)cos(θ) ) )
      ( 0 -r^2sin(θ)cos(θ) )_## ( r^2sin(θ)cos(θ)      0 )_##

In [11]: def R~i~j_k_l := withSymbols [m]
      ∂/∂ Γ~i~j_l x~k - ∂/∂ Γ~i~j_k x~l + Γ~m~j_l . Γ~i~m_k - Γ~m~j_k . Γ~i~m_l

In [12]: (R~1_2_#_#, R~2_1_#_#)
      ( ( 0      sin(θ)^2 ) , ( 0  -1 ) )
      ( -sin(θ)^2      0 )_## ( 1  0 )_##

In [13]: def Ric_i_j := withSymbols [m] sum (contract R~m~i_m_j)

In [14]: Ric_#_#
      ( 1      0 )
      ( 0  sin(θ)^2 )_##

In [15]: def scalarCurvature := withSymbols [i, j] g~i~j . Ric_i_j

In [16]: scalarCurvature
      2
      r^2

```

図 13.1: 球面のリーマン曲率テンソルの計算

$$\Gamma_{ijk} = \frac{1}{2} \left(\frac{\partial g_{ij}}{\partial x^k} + \frac{\partial g_{ik}}{\partial x^j} - \frac{\partial g_{kj}}{\partial x^i} \right) \quad \Gamma^i_{jk} = g^{im} \Gamma_{mjk}$$

リーマン曲率テンソルも，[11] のように，その数式による定義をそのままプログラムとして記述できている．直感的には，リーマン曲率テンソル R^i_{jkl} は， e_k と e_l によって作られる閉路に

沿って e_j を動かしたときの e_i 方向への変化の具合を表現する．

$$R^i_{jkl} = \frac{\partial \Gamma^i_{jl}}{\partial x^k} - \frac{\partial \Gamma^i_{jk}}{\partial x^l} + \Gamma^m_{jl} \Gamma^i_{mk} - \Gamma^m_{jk} \Gamma^i_{ml}$$

13.2.4 リッチ曲率とスカラー曲率 - [13]-[16]

[13]-[14] でリッチ曲率を，[15]-[16] でスカラー曲率を計算している．それぞれ下記の数式に対応している．

$$Ric_{ij} = {}^m_{imj} \quad S = g^{ij} Ric_{ij}$$

13.2.5 計量を変えてみる

本節で解説したプログラムの計量の部分を変えるだけで，いろいろな多様体のクリストッフェル記号やリーマン曲率テンソルを計算できる．

たとえば，トーラス T^2 のリーマン曲率テンソルを計算するには，球座標系をトーラスの座標系に変更すればよい．トーラスの座標系はたとえば，図 13.2 の [1]-[2] のように設定できる．バッククォート (“”) が，この式の中で使われている．10.2 節で解説したように，バッククォートのついた数式は，Egison 処理系に一つのシンボルと同じように扱われる．ここでは，バッククォートを適切に挿入することより，計算プロセスや，最終的な計算結果が単純になっている．

また，計量を以下のように定義される Schwarzschild 計量に変えれば，原点を中心に一つだけ質量をもつ天体がある宇宙についての一般相対性理論の計算ができる．この計算結果は，「時空の曲がり=重力」ということをあらわしている．

```
def g_i_j : Tensor MathExpr := [| [| ` (c^2 * r - 2 * G * M) / c^2 * r, 0, 0, 0 |]
    [| 0, -1 * c^2 * r / ` (c^2 * r - 2 * G * M), 0, 0 |]
    [| 0, 0, -1 * r^2, 0 |]
    [| 0, 0, 0, -1 * r^2 * (sin ` $ \theta $ )^2 |] |]
```

13.3 曲率形式の計算 - 微分形式を使った計算 (1)

11.7 節や，11.8 節，11.13 節で言及したように，Egison のテンソル関連の機能は微分形式を使った計算を簡潔に表現できるように設計されている．微分形式を使うと，テンソルに関する演算をより抽象的に記述できるようになる．

本節では，微分形式の計算の例として，曲率形式の計算を紹介する．曲率形式は，リーマン曲率テンソルを一般化したもので，曲率形式の公式 $\Omega^i_j = d\omega^i_j + \omega^i_k \wedge \omega^k_j$ というリーマン曲率テンソ

```

In [1]: def x := [| θ, φ |]

In [2]: def X := [| ~(a * cos θ + b) * cos φ, ~(a * cos θ + b) * sin φ, a * sin θ |]

In [3]: def e_i := ∂/∂ X x_i

In [4]: e_#
      (
      -asin(θ)cos(φ)    -asin(θ)sin(φ)    acos(θ)
      -(acos(θ)+b)sin(φ) (acos(θ)+b)cos(φ)    0
      )_#

In [5]: def g_i_j := generateTensor (\[a, b] -> e_a_i . e_b_i) [2, 2]

In [6]: def g-i-j := M.inverse g_#_#

In [7]: (g_#_#, g~#-#)
      (
      (
      a^2      0
      0      (acos(θ)+b)^2
      )_##,
      (
      1/a^2      0
      0      1/(acos(θ)+b)^2
      )_##
      )

In [8]: def Γ_i_j_k := (1 / 2) * (∂/∂ g_i_k x-j + ∂/∂ g_i_j x-k - ∂/∂ g_j_k x-i)

In [9]: def Γ~i_j_k := withSymbols [m] g-i-m . Γ_m_j_k

In [10]: (Γ_1_#_#, Γ_2_#_#)
      (
      (
      0      0
      0      (acos(θ)+b)asin(θ)
      )_##,
      (
      0      -(acos(θ)+b)asin(θ)
      -(acos(θ)+b)asin(θ)    0
      )_##
      )

In [11]: def R~i_j_k_l := withSymbols [m]
      ∂/∂ Γ~i_j_l x-k - ∂/∂ Γ~i_j_k x-l + Γ~m_j_l . Γ~i_m_k - Γ~m_j_k . Γ~i_m_l

In [12]: (R~1_2_#_#, R~2_1_#_#)
      (
      (
      0      (acos(θ)+b)cos(θ)/a
      -(acos(θ)+b)cos(θ)/a    0
      )_##,
      (
      0      -acos(θ)/(acos(θ)+b)
      acos(θ)/(acos(θ)+b)    0
      )_##
      )

In [13]: def Ric_i_j := withSymbols [m] sum (contract R~m_i_m_j)

In [14]: Ric_#_#
      (
      acos(θ)/(acos(θ)+b)      0
      0      (acos(θ)+b)cos(θ)/a
      )_##

In [15]: def scalarCurvature := withSymbols [i, j] g-i-j . Ric_i_j

In [16]: scalarCurvature
      2cos(θ)
      (acos(θ)+b)a

```

図 13.2: トーラスのリーマン曲率テンソルの計算

ルの公式よりも簡潔な形の式を使って計算できる。上記の公式にあらわれる ω は接続形式と呼ばれ、微分形式において第二種クリストッフェル記号に対応するものである。

```

In [1]: def x := [| θ, φ |]

In [2]: def g_i_j := [| [| r^2, 0 |], [| 0, r^2 * (sin θ)^2 |] |]_i_j

In [3]: def g~i~j := [| [| 1 / r^2, 0 |], [| 0, 1 / (r^2 * (sin θ)^2) |] |]~i~j

In [4]: def Γ_i_j_k := (1 / 2) * (∂/∂ g_i_k x~j + ∂/∂ g_i_j x~k - ∂/∂ g_j_k x~i)

In [5]: def Γ~i~j_k := withSymbols [m] g~i~m . Γ_m_j_k

In [6]: def ω~i~j := Γ~i~j_#

In [7]: def d %t := !(flip ∂/∂) x t

In [8]: infixl expression 7 ^

In [9]: def (^) %x %y := x !. y

In [10]: def Ω~i~j := withSymbols [k]
antisymmetrize (d ω~i~j + ω~i~k ∧ ω~k~j)

In [11]: Ω~1_2

$$\begin{pmatrix} 0 & \frac{\sin(\theta)^2}{2} \\ \frac{-\sin(\theta)^2}{2} & 0 \end{pmatrix}$$


In [12]: Ω~2_1

$$\begin{pmatrix} 0 & \frac{-1}{2} \\ \frac{1}{2} & 0 \end{pmatrix}$$


```

図 13.3: 曲率形式の計算

13.3.1 ウェッジ積・曲率形式

図 13.3 は S^2 の曲率形式を Egison で計算したプログラムである。[1]-[5] で球面座標系のパラメータと、リーマン計量、クリストッフエル記号を設定している。[6] で接続形式 ω を第二種クリストッフエル記号を使って定義している。[7] で外微分演算子を定義している。[8]-[9] でウェッジ積を定義している。ウェッジ積は中置演算子として定義されている。[10]-[12] で曲率形式を計算して出力している。

13.3.2 オイラー形式とオイラー数

オイラー類は、積分したらオイラー数が得られる特性類として有名な特性類である。以下のプログラムは、トーラス T^2 のオイラー形式を計算している。

```

def eulerForm : Tensor MathExpr := (1 / (2 * π)) * (Ω~1_2 - Ω~2_1)

eulerForm -- [| [| 0, (cos θ) / (* 2 π) |], [| ((-1 * (cos θ)) / (* 2 π)), 0 |] |]

```

```

In [1]: def N := 2

In [2]: def x := [|r, 0|]

In [3]: def g_i_j := [| [| 1, 0 |], [| 0, r^2 |] |]_i_j

In [4]: def g-i-j := [| [| 1, 0 |], [| 0, 1 / r^2 |] |]~i-j

In [5]: def d %A := !(flip ∂/∂) x A

In [6]: def hodge %A :=
  let k := dfOrder A in
  withSymbols [i, j]
  (sqrt (abs (M.det g_#_#))) *
  (foldl1 (.) ((ε' N k)_(i_1)..._(i_N) . A..._(j_1)..._(j_k))
    (map 1#g~(i_#1)~(j_#1) [1..k]))

In [7]: def δ %A :=
  let k := dfOrder A in
  -1^(N * (k + 1) + 1) * (hodge (d (hodge A)))

In [8]: def Δ %A :=
  match dfOrder A as integer with
  | #0 -> δ (d A)
  | #N -> d (δ A)
  | _ -> d (δ A) + δ (d A)

In [9]: def f := function (r, θ)

In [10]: Δ f

```

$$\frac{-\frac{\partial^2 f}{\partial \theta^2} - r \frac{\partial f}{\partial r} - r^2 \frac{\partial^2 f}{\partial r^2}}{r^2}$$

図 13.4: 極座標のホッジ・ラプラシアン の計算

積分のための関数が Egison には実装されていないため、手計算でこれを積分した結果を記す。積分すると T^2 のオイラー数である 0 が得られる。

$$\chi(T^2) = \int_{T^2} d\theta d\phi \frac{\cos \theta}{2\pi} = \int_0^{2\pi} d\theta \cos \theta = [\sin \theta]_0^{2\pi} = \sin 2\pi - \sin 0 = 0$$

13.4 ホッジ・ラプラシアン の計算 - 微分形式を使った計算 (2)

本節では、微分形式を使った計算をプログラムする例を紹介する。

図 13.4 は、極座標のラプラシアンをホッジ・ラプラシアンの公式 $\Delta = d\delta + \delta d$ を使って計算する Egison プログラムとその実行結果である。本節の前半は、このプログラムの解説をしながら進む。

13.4.1 外微分・ホッジ作用素

図 13.4 の前半 [1]-[4] は，極座標系のパラメーターと計量を設定している．その後，外微分・ホッジ作用素・外微分の随伴作用素という微分形式の基本的な演算子を [5]-[8] で定義している．[6] のホッジ作用素の定義には以下の公式を使っている．この定義のなかで使われている `subrefs` は，第二引数のシンボルのリストを，第一引数のテンソルの下添字として順番に付加する組み込み関数である．

$$*A = \sqrt{\det |g|} \cdot \epsilon_{i_1 \dots i_n} \cdot A_{j_1 \dots j_k} \cdot g^{i_1 j_1} \dots g^{i_k j_k} \cdot e^{i_{k+1}} \wedge \dots \wedge e^{i_n}$$

13.4.2 ホッジ・ラプラシアン

微分形式を使うと，ラプラシアンは $\Delta = d\delta + \delta d$ と座標系に依存しないように定義できる．この公式を使って表現されるラプラシアンはホッジ・ラプラシアンと呼ばれる．[8] においてこの公式が `Egison` で表現されている．[8] の定義において，0 形式と 2 形式の場合が特別扱いされている．これは，2 次元座標系における 2 形式にたいして d が，0 形式にたいして δ が，厳密には未定義であるためである．

[9] で定義した関数シンボルに，[10] でホッジ・ラプラシアンを適用している．13.2 節で解説したリーマン曲率テンソルと同様，ホッジ・ラプラシアンについても計量を変えるだけで，いろいろな座標系のラプラシアンを計算できる．

13.5 その他の微分形式についての演算子 - 内部積・リー微分

内部積やリー微分も，`Egison` で簡潔に定義できることを紹介しておく．これらの演算子は，たとえば，流体力学の質量保存の法則やナビエ・ストークス方程式の微分形式による表現をするときに使う．内部積は以下のように定義できる．

```
def ι (X: Tensor MathExpr) (Y: Tensor MathExpr) : Tensor MathExpr :=
  withSymbols [i] dfOrder Y * X...~i . (antisymmetrize Y..._i)
```

リー微分は以下のように定義できる．

```
def Lie (X: Tensor MathExpr) (Y: Tensor MathExpr) : Tensor MathExpr :=
  match dfOrder Y as integer with
  | #0 -> ι X (d Y)
  | #N -> d (ι X Y)
  | _ -> ι X (d Y) + d (ι X Y)
```

第 III 部

Egison の今後

第 14 章

Egison 開発の目標と方針

本章では、今後の Egison 開発についてどのようなことを考えているのか、「処理系の開発」・「アプリケーションの提示」・「Egison の先にある研究」と三つのテーマにわけて紹介する。

14.1 処理系の開発

Egison 処理系の開発については、

1. 独立したプログラミング言語としての Egison 処理系の開発
2. Egison の機能の他言語への移植

の 2 つを進めている。14.1.1 節で、Egison 本体の実装について、14.1.2 節で多言語への移植について述べる。

14.1.1 Egison 本体

Egison 本体には、Egison コミュニティで発明された言語機能をフルに実装されている。言語機能は豊富であるが、実行速度に関してはあまり気にせず実装されており、インタプリタ実装しかもたない。これは、新しい言語機能をスムーズに実装できるようにするためである。インタプリタは Haskell で実装されている。

独自の新機能の追求

プログラミング言語に今まで実装されたことがない新しい機能を Egison に実装し動いたときが、Egison 開発で一番楽しい瞬間である。新しい言語機能のアイデアは実際に現実的な問題について手を動かしてプログラムを書いているときに生まれる。ループ・パターンや、シーケンシャル・パターン、テンソルの対称性の宣言、関数シンボルなどの機能はそうにして生まれた。今後もどんどん新しい言語機能を発案・実装していきたいと考えている。

静的型システムやコンパイラの実装

静的型システムやコンパイラのような既存の言語機能を Egison に組み込むことも重要な課題である。既存の言語機能を Egison に組み込むためには、新しい工夫が必要になることが多い。たとえば、静的型システムを Egison に組み込むには、パターンやマッチャーについて適切な型規則を設計する必要がある。

Egison に静的型システムを導入する研究は進んでおり、河田旺さんによる Typed Egison [2] と Formalized Egison [1] がある。Typed Egison は、Egison インタプリタからパターンマッチに関する最小限の機能を切り出し、静的型システムを実装したものである。Typed Egison は Egison version 3 をベースにしている。Formalized Egison は、Typed Egison の型安全性を証明するために Coq で実装されたインタプリタである。Egison 本体に静的型システムを導入するには、比較的大きな手間がかかるため、まだ着手されていない。Egison Version 5 の課題である。

Egison パターンマッチをコンパイル手法についても研究は進んでいる。2020 年 3 月に小川広水さんによって Egison パターンマッチをバックトラッキング・モノドを使った Haskell プログラムに変換する手法が発案され、Sweet Egison[4] という Haskell ライブラリとして実装された。ただ、Egison 本体については、新機能をすばやく実装できる proof of concept のプログラミング言語ということもあり、コンパイラを実装することはまだ先になりそうである。

■Egison Version 4 のリリース 2020 年 4 月にリリースした Egison version 4 から、多くのユーザーにとって馴染みやすいように文法を一新した。Egison Version 4 の新しい文法は中置記法 ($:=$, $++$, $::$, $+$ など) をサポートしたおかげで、必要な括弧の数が種類も減っており、視認性が高まっている。以下は、素数の無限リストから双子素数を抽出するパターンマッチを記述したプログラムを Egison version 3 と Egison version 4 で書きくらべたものである。

Listing 14.1: Egison version 3

```
(define $twin-primes
  (match-all primes (list integer)
    [<join _ <cons $p <cons ,(+ p 2) _>>> [p (+ p 2)])])

(take 5 twin-primes) ; => {[3 5] [5 7] [11 13] [17 19] [29 31]}
```

Listing 14.2: Egison version 4

```
def twinPrimes :=
  matchAll primes as list integer with
    | _ ++ $p :: #(p + 2) :: _ -> (p, p + 2)

take 5 twinPrimes -- => [(3, 5), (5, 7), (11, 13), (17, 19), (29, 31)]
```


新文法の親しみやすさは、特に数式を書いたときに発揮される。リーマン曲率テンソルの公式という微分幾何で有名な公式を Egison version 3 と Egison version 4 で書きくらべてみる。リーマン曲率テンソルの公式は以下のような数式では表現される。

$$R^i_{jkl} = \frac{\partial \Gamma^i_{jl}}{\partial x^k} - \frac{\partial \Gamma^i_{jk}}{\partial x^l} + \Gamma^m_{jl} \Gamma^i_{mk} - \Gamma^m_{jk} \Gamma^i_{ml}$$

数式の記述には中置記法が多用されるため、中置記法をサポートした新文法のほうが数式に近い記述になる。以下は、実際にリーマン曲率テンソルの公式を Egison version 3 と Egison version 4 で書きくらべたものである。

Listing 14.3: Egison version 3

```
(define $R~i_j_k_l
  (with-symbols {m}
    (+ (- (∂/∂ Γ~i_j_l x~k) (∂/∂ Γ~i_j_k x~l))
      (- (. Γ~m_j_l Γ~i_m_k) (. Γ~m_j_k Γ~i_m_l)))))
```

Listing 14.4: Egison version 4

```
def R~i_j_k_l := withSymbols [m]
  ∂/∂ Γ~i_j_l x~k - ∂/∂ Γ~i_j_k x~l + Γ~m_j_l . Γ~i_m_k - Γ~m_j_k . Γ~i_m_l
```

14.1.2 他のプログラミングへの Egison 言語機能の移植

2019 年からは既存のメジャーなプログラミング言語に Egison の機能を実装することもはじまった。Egison の言語機能を既存プログラミング言語へ移植することには、

1. 移植先のプログラミング言語のユーザーが気軽に Egison の機能を試すことができるようになる。
2. 移植先のプログラミング言語の処理系で実行されるために、ナイーブな実装の Egison 本体よりも、プログラムの実行速度が格段に高速になる。

というメリットがある。ただし、[14.1.1](#) で述べたとおり、既存言語に Egison の言語機能を移植するためには、既存言語の他の機能との兼ね合わせを考慮する必要があるために、すべての機能が移植できないことがある。たとえば、現在のところ、ループ・パターンの移植はまだできていない。また、既存言語の仕様や実装に詳しくなる必要があるために、これらのメリットを享受するためには比較的大きな開発コストがかかるというデメリットもある。そのため、Egison 本体も並行して開発されている。

Egison パターンマッチの Scheme ライブラリ実装（実装済み）

2018 年 12 月に Scheme という言語自体がシンプルでかつ強力なメタプログラミング機能（言語を拡張するための言語機能）をもつ関数型プログラミング言語に Egison のパターンマッチを提供するライブラリを実装した。言語の拡張機能（メタプログラミングのための機能）が強力でかつ言語仕様が簡潔な言語をほど、言語機能の移植が進めやすい。そのため、最初の移植先言語として、Scheme が選ばれた。このライブラリの実装については論文を執筆して Scheme Workshop 2019 で発表した。このライブラリ実装は Scheme 処理系によってコンパイルされるために、Egison のパターンマッチを使ったプログラムを、Egison 処理系よりも 50 倍近く高速に実行できる。Scheme 処理系には Gauche を選んだ。Deep embedding と呼ばれる手法を用いて、このライブラリは実装されている。

Egison パターンマッチの Haskell ライブラリ実装その 1（miniEgison として実装済み）

上記の Egison パターンマッチの Scheme 移植が完成した直後から、同様の手法で Egison パターンマッチを Haskell に移植するための Haskell ライブラリの開発を開始した。Scheme が動的型付けプログラミング言語であるのにしたいして、Haskell は静的型システムをもつ言語であり、型システムを意識した移植の手法を考える必要があることが実装の難所だった。この実装は 2019 年 9 月頃に完成した。

Egison パターンマッチの Haskell ライブラリ実装その 2（Sweet Egison として実装済み）

miniEgison には、miniEgison を使って書いたプログラムと同等のプログラムを Haskell で通常の関数型プログラミングスタイルで書いたプログラムをくらべたときに miniEgison によるプログラムのほうが数倍遅いという問題があった。この問題を解決するために、shallow embedding という手法で、Sweet Egison という Haskell ライブラリを 2020 年 3 月から新しく実装しなおした。Sweet Egison は、Egison パターンマッチを使って記述されたプログラムを、同等の意味の関数型プログラミングスタイルで記述されたプログラムに変換して実行する。そのため、Egison パターンマッチを用いた簡潔なプログラムの記述と通常の Haskell プログラムと同等の実行速度を併せ持つ。

Egison パターンマッチを GHC 拡張として実装（未実装）

ライブラリにより DSL として言語機能を実装すると、冗長な記述をしないといけない箇所が出てくることが多い。Egison パターンマッチを実装した Haskell ライブラリである miniEgison や Sweet Egison にもこの問題があり、マッチ節やマッチャー定義の記述に少し冗長な記述が必要になる箇所がある。この問題を解決するには、Haskell の標準的なコンパイラである GHC を直接編集して組み込みの構文として言語機能を実装すればよい。今後、GHC を直接拡張して Egison のパターンマッチを実装する試みをする予定である。ただ実装するだけでなく、GHC に取り込まれ

るためには GHC コミュニティとコミュニケーションをとる必要がある。GHC に新機能を提案するための仕組みとして GHC Proposal というものがある。ある程度実装がすすんだらドキュメントをまとめてそこで提案したい。

システム・プログラミング言語 Rust (未実装)

ユーザーがメモリの管理を操作できるプログラミング言語への Egison パターンマッチの実装は面白い課題である。現在のところ、以下のような疑問をもっている。

- メモリを効率的に操作することにより、効率的なパターンマッチ機能が実装できるか？
- メモリを効率的に操作するために、パターンマッチ機能を制限する必要があるか？
- パターンマッチ機能を制限したとして応用はあるか？

メタプログラミング機能が豊富といわれる Rust への実装を考えている。

14.2 アプリケーションの提示・開発

Egison の提供する言語機能のアプリケーションは、多くのプログラマにとって自明ではない。Egison の応用例を提示・開発することは、世のプログラマに Egison が実用に役に立つことに納得してもらい、さらに新しい応用例を考えてもらうために重要である。

14.2.1 アルゴリズムの実装 (SAT ソルバー)

さまざまなアルゴリズムを Egison で実装してみることは、Egison の機能が役に立つ場面が明確になったり、新しい機能の発明につながることが多い。第 4 章で SAT ソルバーのアルゴリズムの 1 つである Davis-Putnam アルゴリズムの実装を紹介したが、より複雑であるが効率的な CDCL アルゴリズムも Egison で実装している。より効率的なアルゴリズムを実装しようとするメモリの使い方まで管理したくなり、14.1.2 節で言及したように、Rust のようなシステム・プログラミング言語で Egison のパターンマッチを使いたいニーズも生まれる。

14.2.2 数式処理システム

本書の第 II 部で紹介した数式処理システムは、Egison パターンマッチの重要な応用である。数式に対するパターンマッチ・エンジンが簡潔に実装できるために、数式処理システムを少ない手間で実装することができた。そのために、数式処理システムの拡張も簡単で、テンソルの添字記法のサポートや関数シンボルなど新しい機能の実装を簡単に実験できた。

14.2.3 Egison のパターンマッチを応用した定理証明記述言語

将来はコンピュータ上で数学の証明は記述し検査されるようになる．現在これがされていないのは，コンピュータ上で数学の証明を紙の上のように簡潔に表現できないためである．Egison はプログラミング言語としてアルゴリズムの表現を簡潔にする機能を開発したが，同様に証明の記述を簡潔にすることもできるはずである．たとえば，Egison のパターンマッチは証明のための条件分岐を記述するために役に立つはずである．そのような考えをもとに，Egison のパターンマッチをもつ証明支援システムを設計している [6]．

14.2.4 クエリ言語（パターンマッチの応用）

Egison のパターンマッチはさまざまな種類のデータベースのクエリ言語として使うことができる．例として，ソーシャルネットワークを管理するデータベースに対して，ユーザー名 "Egison_Lang" のユーザからフォローされているが，このユーザをフォローし返していないユーザー一覧を取得するクエリを考える．このクエリは，matchAll式を使って記述できる．リレーショナル・データベースのテーブルを集合として，パターンマッチしている．

```
1 matchAll (users, follows, users) as (set user, set follow, set user) with
2   ((and (Name #"Egison_Lang") (ID $uid)) : _),
3   (and (FromID #uid) (ToID $fid)) : !((and (FromID #fid) (ToID #uid)) : _),
4   (and (ID #fid) (Name $fname)) : _) -> (fid, fname)
```

この matchAll 式は，ユーザテーブル (users) と，フォローテーブル (follows)，ユーザテーブルのタプルに対してパターンマッチする．それぞれのテーブルは集合としてパターンマッチされる．2 行目はユーザテーブルに対するパターンマッチを記述している．Name と ID は，それぞれのレコードのフィールドの値を取得するためのパターン・コンストラクタである．2 行目のユーザテーブルに対するパターンは，ユーザ名が "Egison_Lang" であるユーザの ID をパターン変数 \$uid に束縛する．3 行目はフォローテーブルに対するパターンマッチを記述している．FromID と ToID は，フォローとフォロイーを取得するためのパターン・コンストラクタである．FromID のユーザが ToID のユーザをフォローしている．ユーザ IDuid のユーザをフォロー返ししていないユーザは not パターンを使ってパターンマッチされている．4 行目はユーザ IDfid のユーザの名前を取得するために，ユーザテーブルのパターンマッチをしている．結果として，タプル (fid, fname) を返している．

Egison パターンマッチにはクエリの記述が簡潔であるという利点もある．たとえば，上記と同等のクエリを SQL で記述すると複雑になる．非線形パターンのかわりに WHERE 節のなかに条件を記述し，not パターンのかわりにサブクエリを使うためである．パターンマッチ指向によるクエリの記述は左から右に一度で読めるが，SQL のクエリはそうのように読むことができない．

```
1 SELECT DISTINCT ON (user.name) user.name
```

```

2 FROM user AS user1, follow AS follow1, user AS user2
3 WHERE user1.name = 'Egison_Lang' AND follow1.from_id = user1.id AND user2.id = follow1.
   to_id
4 AND NOT EXISTS
5     (SELECT '' FROM follow AS follow2
6         WHERE follow2.from_id = follow1.to_id AND follow2.to_id = user1.id)

```

14.2.5 機械学習（テンソル添字記法の応用）

テンソルを扱う機械学習のアルゴリズムは、添字記法を使って簡潔に表現できる。その例として、Egison によるニューラル・ネットワークの実装を考えている。ニューラル・ネットワークの実装には下記のような数式が現れる。 x_i は前の層から現在の層への入力、 y_j は現在の層から次の層への出力、 w_j^i は重み、 b_j はバイアスを表す。

$$y_j = w_j^i x_i + b_j$$

1 階のテンソルであるベクトルや 2 階のテンソルである行列までしか登場しないニューラル・ネットワークのアルゴリズムの場合、微分幾何の計算にくらべると添字記法の恩恵は薄れるが、行列の転置のミスなどがなくなるため、添字記法は役に立つ。ニューラル・ネットワークのアルゴリズムには、ステンスル計算や、平均プーリングやマックス・プーリングといったプーリング処理のような数学のテンソル計算では現れない処理が必要になるため、このような処理を表現する方法について研究する必要がある。

14.3 Egison の先にある研究

Egison を開発している間に考える始めるようになった解決することがむずかしそうな大きな未解決問題を紹介する。

14.3.1 二次元以上のデータの表現

改行やインデントを使って、二次元的にプログラムを視覚化することができるが、基本的にプログラムは一次元の文字列データである。プログラムは構文木により内部的にはツリー構造によって表現される。プログラムを表現する文字列は、このツリー構造を深さ優先の順番で並べたものである。そのため、ツリーの形に落とし込めないデータやプログラムの構造は、きれいにプログラムとして表現できない。

ツリーの形に落とし込めないデータには、たとえばグラフがある。グラフを一次元の文字列に落とし込むと、二次元の紙面の上に図示したグラフとくらべて、どうしても読みにくいものになる。その他にもオセロ盤や碁盤も一次元の文字列に落とすのがむずかしいデータである。オセロの挟むという概念や、碁の囲むという概念は、直感的には簡単な概念であるのにプログラム上で表

現するのはむずかしい。

ツリーの形にきれいに落とし込めないプログラムの構造には、複数の引数をとって複数の結果を返す関数の適用がある。一般的なプログラムでは、たいていの関数は複数の引数をとるが一つの返り値を返す。そのため、 $f(g(x), h(y))$ のように、ある関数の結果を別の関数の結果にわたすような関数適用のネストがきれいに書ける。一つの引数を取り、複数の結果を返す関数の適用もツリーの形にきれいに落とし込める。実はこの構文がパターンマッチである。パターン $\$x :: \$y :: _$ は一つの引数をとって二つの返り値を返す関数 $::$ の適用をネストしたものと考えることができる。複数の引数をとって複数の結果を返す関数のネストした適用は、これらのようにツリーの形にきれいに落とし込めない。

複数の引数をとって複数の結果を返す関数が頻出する分野に量子アルゴリズムがある。量子アルゴリズムの記述のために、量子回路が記述されることが多いが、基本的に量子回路への q ビットの入力と出力は同数である。量子アルゴリズムの紹介には、量子回路の図が付随することが多いが、この図のほうがプログラムより理解しやすい。

これらの問題を解決するには、ツリー構造以外を使って内部的にプログラムを表現する、文字列以外を使ってプログラムを記述するなどといった根本的な変化が必要であるようにみえる。プログラムを二次元の図として記述する手法には、ヴィジュアル・プログラミングという分野の手法が知られている。このような視点で新しいプログラミング言語を考えることを面白そうに感じている。

14.3.2 数以外のユーザー定義データ型もシンボリックに処理できる数式処理システム

Egison を含む多くの数式処理システムは、数の計算をシンボリックに扱うことができる。しかし、シンボリックに扱いたい対象は数だけではない。リストや、ツリー、グラフなどもっとさまざまなユーザー定義データ型をシンボリックに扱いたいこともある。Egison 上の数式処理システムは数のシンボリックな扱いが組み込みで実装されており、任意のデータ型にシンボリックな計算を拡張できるようになっていない。そこで、シンボリックな計算が柔軟にできるもっと一般的な数式処理システムを作れたら面白いと感じている。

14.3.3 複数のプログラミング・スタイルの表現力の差をくらべるための理論

ことなるプログラミング・スタイルで記述された複数のプログラムの読みやすさ・書きやすさを客観的にくらべる手法は確立されていない。このような手法がないことは、新しい記法の開発があまり活発でない原因の一つであると考えられる。また、もしこのような手法があれば Egison プログラムの読みやすさ・書きやすさを客観的に示すことができ、Egison を広めやすくなると考えている。そのようなわけで、あるプログラムの読みやすさ・書きやすさを測る手法について考察している。本節では、この問題に関する現時点の考えを書く。

プログラムの読みやすさ・書きやすさとは、プログラムの理解のしやすさと同じである。プログラムの理解のしやすさを定義するには、プログラムを理解するとはどういうことか定義する必要がある。プログラムを理解するとはどういうことか定義できれば、あとはそれにかかる計算量がプログラムの理解のしやすさということになる。

プログラムの理解には無数の段階がある。たとえば、マージソートを実装したプログラムについて、

- リストを引数にとってリストを返すプログラム
- 順序に沿って並べ直したリストを返すプログラム
- リストの長さを n としたとき、時間計算量 $n \log(n)$ で結果を返すプログラム

などいろいろな視点から解析し理解できる。これらのプログラムの性質は、プログラムの型として表現でき、型が合っているか検査したり、型を推論できる場合があることが知られている。

そこで、プログラムの理解のしやすさを計測するために型検査に必要なステップ数を使えるのではと考えている。新しいプログラミング・スタイルの優位性を示すには、そのプログラミング・スタイルによってある特定のプログラムの性質が示しやすくなることを示す。実際、計算量のよくな複雑なプログラムの性質については、Egison のパターンマッチを用いたプログラムは、通常の間数型のプログラムよりも、少ない手間で検査・推論できそうな直感がある。

付録 A

パターンマッチ指向プログラミング問題集

パターンマッチ指向プログラミングを体験していただくために、クイズ形式で問題を用意した。

member 関数

下記の空白をうめて member 関数を完成させよ。

```
1 def member x xs :=
2   match xs as list eq with
3   |  -> True
4   | _ -> False
5
6 member 1 [1,2,3,4]
7 -- True
8
9 member 5 [1,2,3,4]
10 -- False
```

concat 関数

下記の空白をうめて concat 関数を完成させよ。

```
1 def concat xss :=
2   matchAllDFS xss as  with
3   |  -> 
4
5 concat [[1,2],[3],[4,5]]
6 -- [1,2,3,4,5]
```


四つ子素数

四つ子素数を列挙するプログラムを記述せよ。四つ子素数とは、 $(p, p + 2, p + 6, p + 8)$ という形の素数の四つ組のことをいう。

```
1 take 4 (matchAll primes as list integer with
2   |  -> (p, p + 2, p + 6, p + 8))
3 -- [(5, 7, 11, 13), (11, 13, 17, 19), (101, 103, 107, 109), (191, 193, 197, 199)]
```

差が6の素数のペア

下記の空白をうめて素数のペア $(p, p + 6)$ を抽出せよ。

```
1 take 6 (matchAll primes as list integer with
2   |  -> (p, p + 6))
3 -- [(5, 11), (7, 13), (11, 17), (13, 19), (17, 23), (23, 29)]
```

intersect 関数

下記の空白をうめて、2つのコレクションの共通部分を抽出する intersect関数を完成させよ。

```
1 def intersect xs ys :=
2   matchAllDFS (xs, ys) as  with
3   |  -> 
4
5 intersect [1,2,3,4] [2,4,5]
6 -- [2,4]
```

difference 関数

下記の空白をうめて、第二引数のコレクションに含まれていない第一引数のコレクションの要素のみを抽出する difference関数を完成させよ。

```
1 def difference xs ys :=
2   matchAllDFS (xs, ys) as  with
3   |  -> 
4
5 difference [1,2,3,4] [2,4,5]
6 -- [1,3]
```

doubles 関数

コレクション中にちょうど 2 回現れる要素のみを抽出する doubles 関数を完成させよ。

```
1 def doubles xs :=
2   matchAllDFS xs as  with
3   |  -> 
4
5 doubles [1,2,3,2,1,1,4,5,3]
6 -- [2,3]
```

tails 関数

tails 関数をパターンマッチを使って定義せよ。ループ・パターンを使う回答とそうでない回答の 2 つを考えてみよ。

```
1 def tails xs :=
2   matchAll xs as list something with
3   | 
4   -> ts
5
6 tails [1,2,3,4]
7 -- [[1, 2, 3, 4], [2, 3, 4], [3, 4], [4], []]
```

多重集合の同値性チェック

多重集合の同値性をチェックする 2 引数述語 equalMultiset をパターンマッチを使って定義してみよ。

```
1 def equalMultiset xs ys :=
2   match (xs, ys) as (list eq, multiset eq) with
3   | ([], [])
4   -> True
5   | 
6   -> equalMultiset xs' ys'
7   | _
8   -> False
9
10 equalMultiset [1,1,2] [2,1,1]
11 -- True
12
13 equalMultiset [1,1,2] [1,2,2]
```

```
14 -- False
```

ループ・パターンを使って再帰関数を使わずに書いてみよ.

```
1 def equalMultiset xs ys :=
2   match (xs, ys) as (list eq, multiset eq) with
3   | 
4   -> True
5   | _
6   -> False
```

ジョーカーの存在を考慮するポーカーの役判定

3章 3.2.2 節のポーカーの役判定のためのパターンマッチ (poker関数) の実装を一切変更せずに, cardマッチャーの定義だけを変更して, ジョーカーの存在を考慮するポーカーの役判定をできるようにせよ.

```
1 def card := matcher
2   | card $ $ as (suit, mod 13) with
3     | Card $s $n -> [(s, n)]
4     | Joker -> 
5   | $ as something with
6     | $tgt -> [tgt]
7
8 poker [Card Spade 5, Card Spade 6, Joker, Card Spade 8, Card Spade 9]
9 -- "Straight flush"
10 poker [Card Spade 5, Card Diamond 5, Joker, Card Club 5, Card Heart 7]
11 -- "Four of a kind"
```

付録 B

数式処理システムとしての Egison の演習問題

本付録の問題にあるプログラムをいくつか書いてみることによって数式処理システムとしての Egison を使いこなせるようになる。プログラムを書くむずかしさよりも、数学の知識を求められる問題が多いがぜひ挑戦してみしてほしい。

三次方程式と四次方程式の解

9.4 節の内容を発展させて、三次方程式と四次方程式の解を求めるプログラムを書け。三次方程式と四次方程式の解を求めるアルゴリズムには、それぞれカルダノの方法、フェラーリの方法が知られている。

1 の n 乗根

1 の n 乗根は代数的に求めることができることが知られている。いくつかの n について 1 の n 乗根を計算するプログラムを書いてみよ。1 の 5 乗根, 7 乗根, 9 乗根, 17 乗根については Egison レポジトリにサンプルコードが用意されている。

リーマン曲率の計算

13.2 節のプログラムを参考にして、円柱のリーマン曲率, S^3 のリーマン曲率を計算するプログラムを書け。

球座標系のホッジ・ラプラシアン

13.2 節の極座標のホッジ・ラプラシアンを計算するプログラムを参考にして，球座標系のホッジ・ラプラシアンを計算するプログラムを書け．

参考文献

- [1] akawashiro/formalized-egison. <https://github.com/akawashiro/formalized-egison>, 2019. [Online; accessed 2020-02-09].
- [2] egison/typed-egison. <https://github.com/egison/typed-egison>, 2019. [Online; accessed 2020-02-09].
- [3] egison/backtracking: Backtracking monad in Haskell. <https://github.com/egison/backtracking>, 2020. [Online; accessed 2021-05-06].
- [4] egison/sweet-egison: Haskell library for non-deterministic pattern matching. <https://github.com/egison/sweet-egison>, 2020. [Online; accessed 2021-05-22].
- [5] S. Egi. Loop Patterns: Extension of Kleene Star Operator for More Expressive Pattern Matching against Arbitrary Data Structures. In *The Scheme and Functional Programming Workshop*, 2018.
- [6] S. Egi. Pattern-match-oriented proof writing language. In *Conference Companion of the 4th International Conference on Art, Science, and Engineering of Programming*, pages 223–224, 2020.
- [7] S. Egi and Y. Nishiwaki. Non-linear Pattern Matching with Backtracking for Non-free Data Types. In *Asian Symposium on Programming Languages and Systems*, pages 3–23. Springer, 2018.
- [8] J. Harrison. *Handbook of Practical Logic and Automated Reasoning*. Cambridge University Press, 2009.
- [9] J. M. Spivey. Algebras for combinatorial search. *Journal of Functional Programming*, 19(3-4):469–487, 2009.

索引

#, 10
++, 9
--, 9
, 91
::, 9
?, 10
~, 92

algebraicDataMatcher, 18
and パターン, 13
as, 9

contract, 92

flipIndices, 97

inductive pattern 宣言, 42

matchAll, 8
matchAllDFS, 12

not パターン, 13

or パターン, 13

primes, 10

with, 9

値パターン, 10
型クラス, 54
関数シンボル, 108
組み込み関数, 58
原始データパターン, 44
原始パターンパターン, 44
コレクション, 11

コンス・パターン, 9
シーケンシャル・パターン, 16
述語パターン, 10
ジョイン・パターン, 9
スカラー関数, 90
添字付き変数, 14
タプル・パターン, 17
ダミーシンボル, 98
テンソル関数, 90
パターン関数, 17
パターンのアドホック多相性, 11
パターン・フュージョン, 48
パターンマッチ指向, 4
変数パターン, 17
マッチャー, 9
無名パラメーター関数, 51
ループ・パターン, 14
ワイルドカード最適化, 49